



UNIVERSIDAD
NEBRIJA

Master's Degree in Quantum Computing
Higher Polytechnic School

Quantum Error Correction Decoders
for 2D Color Codes:
Systematic Simulation and Comparison

Author: Antonio Morales García

Scientific Advisor: Dr. Miguel Ángel Martín-Delgado Alcántara
Tutor: Prof. Francisco Gálvez Ramírez

Madrid 2026

To Nines, Pedro, Antonio, and Carlos.
The ones I love the most, for being my primum movens.
The ones who love me the most, for the immense love this work consumed.

Acknowledgments

I would like to express my deepest gratitude to everyone who made this Master's Thesis possible.

First and foremost, I want to thank my parents, Frasco and María José, who instilled in me freedom, curiosity, and the courage to challenge the status-quo. They allowed me to grow free from the weight of expectations and always valued and celebrated my achievements. My gratitude also goes to my godfather, Nono, for teaching me perseverance, personal pride (understood as the pursuit of excellence), and the profound desire to leave the world a better place than I found it; and to my grandparents, who taught me the true value of knowledge, wisdom, sacrifice, and hard work. And last, but not least among my roots, I want to thank Lolo, who taught me that there are many different, yet equally valid, ways to approach life, and that the journey is more important than the destination.

My wife, Nines, and our children, Pedro, Antonio, and Carlos also deserve all my gratitude. They are not only my primary motivation, but they have also sacrificed as much as I have for this project. Pursuing a Master's degree in one's forties, and doing it while working full-time, demands immense family sacrifices, and I could not have reached this finish line without their patience, understanding, and love.

I owe much to my professors in this Master's program, and by extension to all my mentors and teachers, who have built the foundation that has allowed me to reach this point: Those who teach us are the invisible architects of our identity. Among them, I want to especially thank Sergio Boixo; his talk first inspired me to delve into the world of quantum computing, and his responses when I asked for advice helped me take my first steps in this field. I also owe a great debt of gratitude to Greg Quillard, a good friend who happens to be the person who has fostered my professional growth the most throughout my career.

Last, but certainly most important, I would like to express my sincere appreciation to the scientific advisor of this work, Miguel Ángel. It has been an absolute privilege to have this thesis directed by such an eminence in the field, and it is a unique honor to write about 2D color codes under the guidance of their co-creator. I am deeply grateful for his time, his insight, and this extraordinary opportunity. Additionally, I extend my thanks to my academic tutor, Paco, for providing a refreshing introduction to classical error correction, for his help in coordinating the administrative details and for connecting me with Miguel Ángel, making this

opportunity a reality.

Since many of the people mentioned above do not speak English, I have taken the liberty of including a Spanish translation of these acknowledgments below.

Me gustaría expresar mi más profundo agradecimiento a todos los que han hecho posible este Trabajo de Fin de Máster.

En primer lugar, quiero dar las gracias a mis padres, Frasco y María José, que me inculcaron la libertad, la curiosidad y el valor para cuestionar lo establecido. Mis padres me permitieron crecer libre del peso de las expectativas y siempre valoraron y celebraron mis logros. Estoy igualmente agradecido a mi padrino, Nono, por enseñarme la perseverancia, el amor propio (entendido como la búsqueda de la excelencia) y el profundo deseo de dejar el mundo mejor de lo que lo encontré; y a mis abuelos, que me enseñaron el verdadero valor del conocimiento, la sabiduría, la renuncia y el trabajo duro. Y por último, pero no menos importante entre mis raíces, quiero dar las gracias a Lolo, que me enseñó que hay muchas formas distintas, pero igualmente válidas, de entender la vida, y que el camino es más importante que el destino.

Mi mujer, Nines, y nuestros hijos, Pedro, Antonio y Carlos, también merecen toda mi gratitud. No solo son mi principal motivación, sino que han sacrificado tanto como yo por este proyecto. Cursar un Máster pasados los cuarenta, y hacerlo trabajando a tiempo completo, exige inmensos sacrificios familiares, y no lo habría logrado sin su paciencia, comprensión y amor.

Debo mucho a mis profesores de este Máster y, por extensión, a todos mis mentores y maestros, que han construido los cimientos que me han permitido llegar hasta aquí: aquellos que nos enseñan son los arquitectos invisibles de nuestra identidad. Entre ellos, quiero dar las gracias especialmente a Sergio Boixo; una charla suya fue lo que me inspiró por primera vez a adentrarme en el mundo de la computación cuántica, y sus respuestas cuando le pedí consejo me ayudaron a dar mis primeros pasos en este campo. También debo toda mi gratitud a Greg Quillard, un buen amigo que resulta ser la persona que más ha impulsado mi crecimiento profesional a lo largo de mi carrera.

Por último, pero sin duda lo más importante, me gustaría expresar mi sincero agradecimiento al director científico de este trabajo, Miguel Ángel. Ha sido un

privilegio absoluto que este Trabajo Fin de Máster haya sido dirigido por una eminencia en la materia, y es un honor único escribir sobre los códigos de color 2D bajo la tutela de su cocreador. Estoy profundamente agradecido por su tiempo, su visión y por esta extraordinaria oportunidad. Además, hago extensivo mi agradecimiento a mi tutor académico, Paco, por brindarme una refrescante introducción a la corrección de errores clásica, por su ayuda en la coordinación de los trámites administrativos y por ponerme en contacto con Miguel Ángel, haciendo realidad esta oportunidad.

Structure of this thesis

Indexes and Tables of Content

A brief Table of Contents is provided at the beginning of this work (see page [xi](#)), outlining only the chapters and main sections.

A comprehensive Table of Contents can be found at the end of the thesis (page [I](#)), alongside the List of Figures (page [V](#)) and the Bibliography (page [IX](#)).

Source code and repository structure

Chapters [5](#) and [6](#) show the result of multiple simulations and code execution. Chapter [7](#) produces statistical comparisons of the results obtained in Chapter [6](#).

The complete code required to execute the simulations presented in this work is available in a git repository at https://github.com/othermore/TFM_QEC_2D_color_code_decoders, alongside the simulation result data, plots and the latex sources for this work. Specifically for the developed code, small code snippets are shown in this document to illustrate how the different tools and libraries work, and the line numbers in the original files for each snippet are displayed to facilitate locating them in the source files.

The code is located in the `src` directory. Each chapter has its own directory and python notebook. The specific code for each simulation is located in separate files (starting with `s` and the number of the simulation) and included in the notebook.

Libraries and resources required for multiple simulations within a given chapter, are included in the chapter's directory.

The plots, circuits and lattices are stored in the **figures** directory, organized by chapters, and the remaining figures of this work are stored in **thesis/figures**. Finally, the simulation data is stored in the **data** directory, similarly organized by chapters.

Overview

1	Introduction: Classical error correction and foundations of quantum error correction	1
1.1	Project Goals and Scope	3
1.2	Classical error correction	4
1.3	Shor code	6
1.4	The Stabilizer Formalism	12
1.5	The Threshold Theorem	17
2	State of the art of QEC codes: Surface codes, color codes and QLDPC codes	19
2.1	Introduction	19
2.2	Transversality and the Eastin-Knill Theorem	20
2.3	Surface Codes	22
2.4	Color Codes	26
2.5	Non-topological QLDPC Codes	31

3	Modeling quantum error	35
3.1	Types of quantum noise	35
3.2	Error Models	37
3.3	Relationship between the models	38
4	QEC decoders for color codes	41
4.1	Minimum Weight Perfect Matching (MWPM)	42
4.2	Union-find	45
4.3	BP+OSD	47
4.4	Projection Decoder	51
4.5	The Restriction Decoder	57
4.6	Möbius Decoder	60
4.7	Concatenated MWPM	65
4.8	Correlated matching decoder	69
5	Implementation and simulation of QEC codes: Framework setup and baseline models	73
5.1	Simulating QEC codes and decoders with <code>stim</code>	74
5.2	Beyond <code>stim</code> : Experimental setup for implementation and simulation	86
5.3	Automated noise injection: Extracting error models from Qiskit Fake Providers	91
6	Simulating 2D triangular color codes with a 4.8.8 lattice geometry and a code-capacity noise model	95
6.1	Syndrome Graph Construction and Decoding	95
6.2	Implementing a 2D color code	97
6.3	Simulation Architecture and Evaluation Metrics	101
6.4	BP+OSD	104

6.5	Projection decoder	108
6.6	Restriction Decoder	112
6.7	Möbius Decoder	117
6.8	Concatenated MWPM Decoder	118
6.9	Correlated matching decoder	120
7	Comparative analysis and discussion	123
7.1	Determination of error thresholds (p_{th}) for 4.8.8 color codes	124
7.2	Logical failure rate (p_L) scaling as a function of physical error rate (p) and code distance (d)	125
7.3	Sub-threshold scaling analysis and theoretical suppression limits .	127
7.4	Computational complexity and empirical decoding time	128
7.5	Final evaluation	132
8	Potential improvements and future work	135
8.1	Potential improvements	135
8.2	Future work	136
	List of Figures	V
	Bibliography	XI

Introduction: Classical error correction and foundations of quantum error correction

The field of quantum computing is evolving rapidly. It has transitioned from a theoretical curiosity to being generally accepted as a frontier technology. The fundamental premise of this discipline is that information processing based on the laws of quantum mechanics can solve problems that remain computationally out of reach for classical computing [1]. This paradigm began in 1982, when Richard Feynman pointed out that classical computers are not well equipped to simulate quantum systems due to the exponential growth of space required to represent a quantum state. He proposed that a computer based on quantum mechanical principles would be required to model nature effectively [2].

The Quantum Advantage and the Noise Barrier

The true potential of this technology was demonstrated in 1994 by Peter Shor, who developed a quantum algorithm for integer factorization in polynomial time [3]. Shor's work proved that quantum computing

could break widely used cryptographic schemes, such as RSA. That is a theoretical proof of what has been called “Quantum Advantage”. However, this assumes the existence of quantum hardware that has not been realized yet.

The realization of such a computer is difficult because of the fragility of quantum states. Qubits are vulnerable to noise from environmental interactions and to inaccuracies in the operations we make with them. When this noise affects the qubits, the information they store is lost.

We refer to this reality as the era of Noisy Intermediate-Scale Quantum (NISQ) technology. In this era, error rates in quantum gates are several orders of magnitude higher than their classical counterparts. Without a robust way to protect quantum information, even the most advanced algorithms fail before reaching a meaningful result.

The Evolution of Quantum Error Correction

As we will see later in this chapter, unlike classical bits, quantum information cannot be protected by just copying it, due to the no-cloning theorem. To overcome this, the field of Quantum Error Correction (QEC) was pioneered by Peter Shor [4] and Andrew Steane [5]. They demonstrated that a single “logical” qubit can be represented with multiple “physical” qubits. More formally, the logical qubit is encoded in a larger Hilbert space of multiple physical qubits, allowing for the detection and correction of errors without destroying the underlying quantum state.

While Shor’s solution (known as Shor’s code) proves that QEC can be theoretically implemented, it is unable to effectively mitigate error due to the large number of additional data and ancilla qubits that it adds as distance scales, and the number of qubits monitored by each detector. This added complexity of the code causes more errors than it can prevent, specially under realistic noise models.

A significant milestone in QEC was the introduction of topological codes by Alexei Kitaev [6], which will be discussed in Chapter 2. Kitaev’s Toric Code and subsequent Surface Codes presented a new paradigm of local, lattice-based architectures where errors are detected by measuring local parities (stabilizers). These types of codes became leading candidates for

scalable hardware due to their high error thresholds and two-dimensional physical layouts, which map well to the most common realizations of quantum computers.

1.1 Project Goals and Scope

This Master’s Thesis focuses on the simulation and comparative analysis of decoders for the 2D triangular color code. Introduced by Bombín and Martin-Delgado [7], color codes represent an evolution over surface codes that offers significant advantages, particularly regarding the transversal implementation of logical gates, which simplifies fault-tolerant operations.

The performance of various QEC protocols and their corresponding decoders has been extensively studied by multiple authors. However, the community currently lacks accessible, consolidated resources that establish a clear methodology and a unified framework for evaluating these results. To address this gap, this work aims not merely to provide a performance analysis, but to serve as a detailed methodological guide. By offering a structured evaluation framework, it seeks to ease the steep learning curve for new researchers and graduates starting their work in quantum error correction.

Scope

The scope of this work is defined by three specific constraints regarding the lattice geometry, how to evaluate decoding algorithms, and the noise model. The foundations of these concepts are detailed in the following sections and chapters, starting with an overview of classical error correction principles. Below, we outline the specific constraints.

First, as we will formally define in Chapter 2, color codes can be implemented through several geometries (4.8.8, 6.6.6 and 4.6.12). In this work, we will specifically adopt a **4.8.8 lattice geometry**. This choice is motivated by its lower qubit requirements (fewer qubits are needed per distance) and its properties as a doubly-even code, which enables transversal implementation of the full Clifford group, including H , $S^\dagger$ and CNOT [7].

As this introduction establishes the boundaries of the research, it references advanced concepts (like topological geometries or noise models) that will be formally defined in subsequent chapters.

Second, our objective is to quantify the performance of various decod-

ing strategies, from classical **Projection decoders** to state-of-the-art **Möbius** and **Concatenated MWPM** approaches. Using advanced simulation and sampling tools such as `stim` [8] and `sinter`, in Chapter 7 we will compare their efficiency through standard metrics: physical error thresholds (p_{th}), logical failure rate (p_L) scaling as a function of physical noise, theoretical sub-threshold suppression limits, and the trade-off between decoding accuracy and computational time.

In quantum physics, **decoherence** is the process by which a system loses its quantum properties (like phase or superposition) due to continuous interactions with the environment. By assuming that all logical operations and measurements are perfectly instantaneous, the code-capacity model effectively isolates this environmental decoherence—which corrupts data qubits as they sit idle in the quantum memory—from the operational noise introduced by the physical execution of circuits.

Finally, as we will discuss in Chapter 3, various sources of noise affect real quantum hardware. While realistic hardware-aware simulations of QEC codes require accounting for all of them—including imperfect gates and measurement errors—this study focuses strictly on **code-capacity noise**. This foundational model isolates the decoherence of data qubits over time by assuming perfect syndrome extraction. Adopting this approach allows us to evaluate the strengths of the 4.8.8 lattice and the algorithmic efficiency of the decoders, without the confounding variables introduced by circuit-level operations.

Similarly, different error types (X, Y, Z) can appear in a quantum circuit, and the ratios between them may differ (some types may be more likely). We will simulate both **bit-flip** and **depolarizing** noise, with the former being the standard in the literature and the one we will use to compare our results.

1.2 Classical error correction

Before discussing how to protect quantum information, it is essential to understand the basic mechanisms of classical error correction. While classical bits are physically robust, they are still susceptible to occasional errors (e.g. caused by hardware degradation or electromagnetic interference). Since classical systems are naively discrete (information is binary), a classical error is strictly discrete: a bit flips from 0 to 1, or from 1 to 0.

To detect and correct these bit-flips, classical computing relies on the concept of **redundancy**. By adding extra bits to the original message, we can deduce if an error has occurred and, in many cases, revert it. Two of the most foundational techniques used to achieve this are repetition codes and parity checks. These classical concepts serve as the baseline

for building analogous quantum error correction solutions.

Repetition Codes and Parity Checks

The simplest approach to redundancy is the **repetition code**. If we want to send a single logical bit, we copy it multiple times. For example, in a 3-bit repetition code, a logical 0 is encoded as 000, and a logical 1 as 111.

If a noise event flips one of the physical bits (e.g., the state becomes 010), the receiver can apply a **majority vote** to determine the original message. Since there are more zeros than ones, the system concludes the intended logical state was 0, effectively correcting the error.



Figure 1.1. Mechanism of a classical 3-bit repetition code correcting a single bit-flip error.

A more efficient method is the use of **parity checks**. Instead of copying the entire message, we calculate a parity bit based on the sum of the data bits. For instance, we can add a bit to ensure that the total number of 1s in a string is always an even number. If a single bit flips during transmission, the overall parity becomes odd, immediately alerting the system to the presence of an error. By structuring multiple parity checks across different overlapping blocks of data, classical systems can locate and correct errors with minimal overhead.

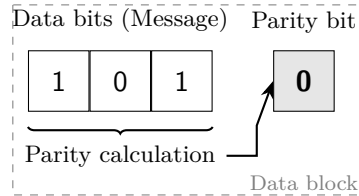


Figure 1.2. A parity check bit appended to a 3-bit message. The parity bit (0) is calculated to ensure that the total number of '1's in the data block is even.

For the 3-bit repetition code to be useful, the probability of a single bit flipping, p , must be low enough that the chance of two or three bits flipping simultaneously ($3p^2(1-p) + p^3$) is negligible.

This overlapping block parity concept is the foundation of classical Hamming codes, which can mathematically pinpoint exactly which bit flipped without needing massive repetition.

The Three Major Challenges of Quantum Error Correction

It is natural to assume we could simply apply these classical techniques to quantum computers. However, translating repetition and parity concepts to the quantum realm is not straightforward. Quantum mechanics imposes three fundamental constraints that classical computing does not face:

1. **The No-Cloning Theorem:** Classical repetition codes require copying the information. In quantum mechanics, it is fundamentally impossible to create an identical copy of an unknown arbitrary quantum state [9]. Therefore, we cannot simply duplicate a qubit to create redundancy.
2. **Measurement Destroys Information:** Classical parity checks require reading the bits to see if an error occurred. If we attempt to measure a qubit in a superposition state to check for errors, the superposition collapses into a definitive classical state. Measuring the data to find the error destroys the quantum information we are trying to protect.
3. **Continuous Errors:** Classical computers are binary (a bit is either 0 or 1) and discrete (there are no intermediary values). This property applies to classical errors (a bit is either right or wrong). Qubits, however, exist in a continuous state space, the complex Hilbert space. This means that there is an infinite number of possible errors that can occur, making it apparently impossible to design a code that accounts for every potential continuous deviation.

The solution to these three fundamental barriers was formulated by Peter Shor, leading to the creation of the first true quantum error correction code, which we explore in the next section.

1.3 Shor code

The transition from classical to quantum error correction was made possible by Peter Shor's 1995 proposal [4]. His approach demonstrated that the three barriers of quantum mechanics mentioned above could be bypassed

A qubit state, $|\psi\rangle$, is a vector in a two-dimensional complex Hilbert space, defined by continuous amplitudes $\alpha|0\rangle + \beta|1\rangle$, with α and β being complex numbers. Such state can be represented in the Bloch Sphere. Because of this continuity, an error can be an arbitrarily small rotation.

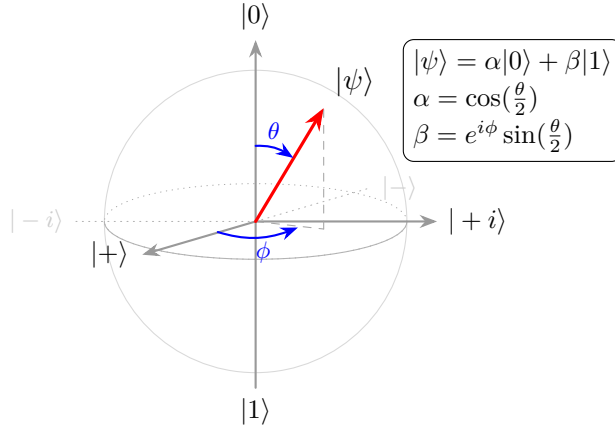


Figure 1.3. Geometric representation of a qubit on the Bloch sphere. The parameters θ and ϕ define the continuous orientation of the state vector $|\psi\rangle$ relative to the computational basis states $|0\rangle$ and $|1\rangle$.

using entanglement, parity-based measurements and concatenation of QEC codes.

The 3-qubit Code: Solving Cloning and Measurement

The first step is understanding that while we cannot copy an unknown state $|\psi\rangle$, we can **distribute** its information across an entangled system. In the 3-qubit bit-flip code, the encoding does not create copies; it creates a multi-particle state where the logical information is stored in a correlation of these particles' states.

$$|0\rangle_L = |000\rangle, \quad |1\rangle_L = |111\rangle$$

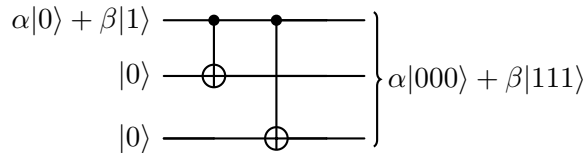


Figure 1.4. Encoding circuit for the bit-flip code. Entanglement distributes the logical information without violating the no-cloning theorem.

To solve the measurement problem, Shor introduced the concept of **syndrome extraction**. Instead of measuring each qubit (which would

The 3-qubit codes exposed here, which are simple repetition codes, are the main components of the 9-qubit Shor code. It is important to note that in Shor's original paper [4] these 3-qubit codes were not introduced as independent, standalone protocols.

destroy the superposition $\alpha|000\rangle + \beta|111\rangle$), we compare pairs of these qubits to determine whether they are equal or different without gaining any information about their actual state.

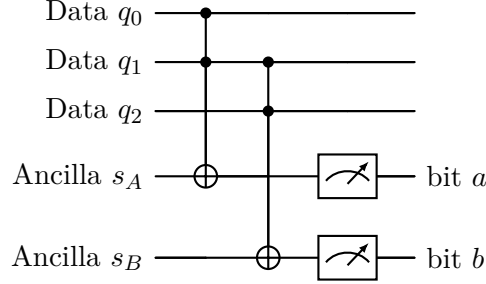


Figure 1.5. Syndrome measurement: The ancilla qubits are activated when the two qubits that they monitor are different. Depending on which of the two ancilla qubits are activated, we can determine which of the qubits has flipped. This is known as a Z-type detector

As shown in Figure 1.5, measuring the ancilla qubits does not provide information about the state of each individual qubit, but it provides information about two qubits being different. The following table shows how the measuring of these ancilla qubits maps to specific identified errors.

Syndrome (s_A)	Syndrome (s_B)	Error Location	Correction
0	0	None	No action
1	0	q_0	Apply X_0
0	1	q_2	Apply X_2
1	1	q_1	Apply X_1

A Code to Detect Phase-flip Errors

The bit-flip code addresses X errors. The other fundamental quantum error is the phase-flip (Z), which changes the relative phase (the sign) in a superposition: $Z(\alpha|0\rangle + \beta|1\rangle) = \alpha|0\rangle - \beta|1\rangle$. This is strictly a quantum phenomenon and has no classical counterpart.

To correct a phase-flip, we can leverage a key property of quantum gates: a Hadamard gate (H) transforms a Z (phase-flip) error into an X error

Formally, the Hadamard gate maps the standard computational basis $\{|0\rangle, |1\rangle\}$ to the superposition basis $\{|+\rangle, |-\rangle\}$. In terms of operators, $HZH = X$.

(bit-flip). Using the H gate, we can reuse the logic of the bit-flip code, where an X -parity check (i.e. comparing pairs of qubits) in the logical space effectively functions as a Z -check in the physical space.

The logical states are thus defined as:

$$|0\rangle_L = |+++\rangle, \quad |1\rangle_L = |--\rangle$$

The encoding circuit entangles the qubits using the CNOT structure of the bit-flip code and then applies a global Hadamard transformation to switch the entire block into the relevant basis.

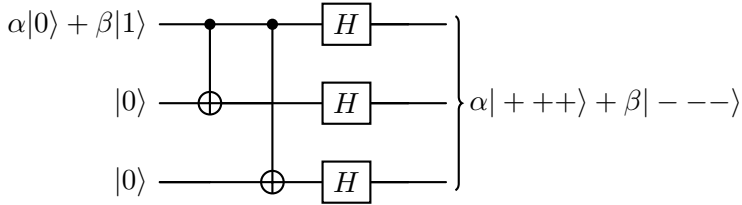


Figure 1.6. Encoding circuit for the 3-qubit phase-flip code. The information is first distributed via entanglement, and then Hadamard gates transform the final state into the $\{|+\rangle, |-\rangle\}$ basis.

Applying the same measuring technique to these three qubits in pairs, allows determining the syndrome, which in this case will point to a Z error.

The 9-qubit Shor Code: Correcting Arbitrary Errors

The 3-qubit code provides a solution to the first two challenges, but it only corrects one type of error (either bit-flip or phase-flip). To be able to correct flips in any of the three axis, Shor's proposed the **concatenation** of a bit-flip code and a phase-flip code. By nesting them, we create a 9-qubit block that protects against both bit-flips (X) and phase-flips (Z) simultaneously.

The logical basis states are constructed by taking three clusters of qubits. Each cluster is in a superposition ($|000\rangle \pm |111\rangle$) to handle phase errors, while each individual qubit within the cluster is tripled to handle bit-flip errors:

To measure the relative phases of the data qubits (implementing a X -type detector), the phase-kickback technique is used: the ancilla is put on a $|+\rangle$ state and CNOT gates are run with the ancilla as control qubit.

$$|0\rangle_L = \frac{(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)}{\sqrt{8}}$$

$$|1\rangle_L = \frac{(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)}{\sqrt{8}}$$

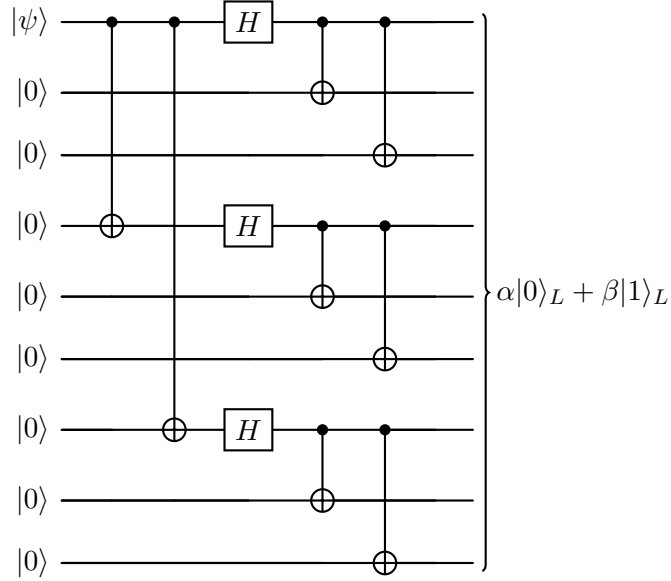


Figure 1.7. Full encoding circuit for the 9-qubit Shor code. The initial CNOT and Hadamard gates encode the logical state into a 3-qubit phase-flip code. Subsequently, each of these three qubits is encoded into a 3-qubit bit-flip code, resulting in the final 9-qubit state.

To extract the syndrome of the 9-qubit Shor code, a total of 8 ancilla qubits are needed, 6 of them monitoring pairs of qubits to detect bit flips inside the clusters (Z-type detectors), and 2 of them monitoring full clusters to detect phase flips between two clusters (X-type detectors). The Table 1.1 shows the ancilla qubits and the data qubits they monitor.

The Table 1.2 shows some examples of possible measured syndromes, the corresponding errors and the necessary recovery operation.

The Discretization of Quantum Errors

We have seen that the 9-qubit Shor code enables correcting both X and Z , but how can we correct an infinite number of possible small rotations?

Ancilla	Detector type	Targeted qubits	Error detected
a_1	Z-type	q_1, q_2	Bit-flip (X) in Block 1
a_2	Z-type	q_2, q_3	Bit-flip (X) in Block 1
a_3	Z-type	q_4, q_5	Bit-flip (X) in Block 2
a_4	Z-type	q_5, q_6	Bit-flip (X) in Block 2
a_5	Z-type	q_7, q_8	Bit-flip (X) in Block 3
a_6	Z-type	q_8, q_9	Bit-flip (X) in Block 3
a_7	X-type	$q_1, q_2, q_3, q_4, q_5, q_6$	Phase-flip (Z) in clusters 1 or 2
a_8	X-type	$q_4, q_5, q_6, q_7, q_8, q_9$	Phase-flip (Z) in clusters 2 or 3

Table 1.1. Ancilla qubits to extract the syndrome in the 9-qubit Shor code.

The solution is surprisingly simple. It relies on the mathematics of quantum measurement. As proven by Knill and Laflamme [10], we do not actually need to correct any arbitrary error. Any arbitrary single-qubit error can be expressed as a linear combination of the Pauli matrices: I (identity, no error), X (bit-flip), Z (phase-flip) and Y (both bit and phase-flip).

By designing a quantum error correction code capable of detecting and correcting independent X and Z errors, such as the one designed by Shor, the act of measuring the error (the syndrome extraction) forces the continuous, arbitrary error to collapse into one of the discrete Pauli operators (I , X , Y and Z). Since I means no error and the Pauli Y operator is simply a combination of X and Z ($Y = iXZ$), **correcting X and Z is sufficient to correct any arbitrary error**. This phenomenon, known as the discretization of quantum errors, effectively reduces the continuity quantum error problem back to a discrete problem analogous to classical bit-flips.

Error Distinguishability

The fundamental requirement for a quantum error correction code is that errors that require different recovery operations must result in distinct syndromes. In formal terms, this means that the errors must map the logical state into orthogonal, distinguishable subspaces. If two physical errors produce the same syndrome but result in fundamentally different

It is important to note that phase-flip correction can be applied to any qubit of the cluster for which we detected the error. This is because the phase is encoded in the relative phase between the $|000\rangle$ and $|111\rangle$ components of the cluster, which changes by changing the phase of any of the three qubits. To correct the Y error, the combination of X and Z can be used.

Syndrome (Z-type) (a_1a_2, a_3a_4, a_5a_6)	Syndrome (X-type) (a_7, a_8)	Error Type	Correction
(00, 00, 00)	(0, 0)	None	I (No error)
(10, 00, 00)	(0, 0)	X_1	Apply X_1
(11, 00, 00)	(0, 0)	X_2	Apply X_2
(01, 00, 00)	(0, 0)	X_3	Apply X_3
(00, 00, 00)	(1, 0)	Z in cluster 1	Apply Z_1
(00, 00, 00)	(1, 1)	Z in cluster 2	Apply Z_4
(00, 00, 00)	(0, 1)	Z in cluster 3	Apply Z_7
(10, 00, 00)	(1, 0)	Y_1	Apply Y_1

Table 1.2. Decoding the syndrome in the 9-qubit Shor code.

corrupted states, the decoder will not know which correction to apply, leading to a logical failure.

Any qubit within a cluster suffering a phase-flip error in the 9-qubit Shor code results in the same syndrome. However, an operation on any of the three qubits of the cluster will fix the state of the logical qubit. This is known as degeneracy, and we say that the three phase-flip errors that can affect a cluster are **degenerated errors**.

The 9-qubit Shor code works well precisely because it separates bit-flip and phase-flip errors into distinct, non-overlapping parity checks. For example, any bit-flip error in a single qubit will activate either one or two of the first 6 ancillas (see Table 1.2), which can map uniquely to the specific qubit that flipped.

Analyzing these subspaces manually becomes unsustainable for larger, more complex quantum codes. This scalability problem motivates the need for a robust, algebraic framework.

1.4 The Stabilizer Formalism

As quantum codes scale, representing the logical state as a vector of 2^n amplitudes becomes computationally intractable. Tracking the evolution of a highly entangled state qubit by qubit is unsustainable. The Stabilizer Formalism, introduced by Daniel Gottesman in 1997 [11], offers an elegant workaround: instead of describing the quantum state directly, we describe the set of operators that leave the state unchanged. More specifically, we focus on the operators that leave unchanged a valid state (i.e. a state where no error has occurred) but modify an erroneous state.

Stabilizer Operators

The core concept of this framework relies on a simple mathematical condition. We say that an operator S **stabilizes** a quantum state $|\psi\rangle$ if applying S to the state does not alter it in any way:

$$S|\psi\rangle = |\psi\rangle \quad (1.1)$$

In practice, a stabilizer operator is simply a combination of Pauli matrices (X , Y , Z) acting on a specific subset of qubits. For example, an operator Z_1Z_2 monitors the phase relationship of the first and second qubits. These operators are what we referred to as “syndrome detectors” or “parity checks” in the previous section when discussing the Shor code.

Note that an error affecting the qubits can also be represented as an operator (e.g. a bit-flip is a X Pauli matrix applied to the flipping qubit).

We can **measure** a stabilizer operator. Such measurement is implemented as a parity check on the data qubits that are connected to this operator, with the result being reflected in the state of an ancilla qubit:

- Z -type stabilizers, which detect bit-flip errors, are measured with a CNOT gate that has the data qubits as source and the ancilla qubit as target.
- X -type stabilizers, which detect phase-flip errors, are measured with a CNOT gate that has the data qubits as target and the ancilla qubit as source, which produces a *phase kickback*.

We must also introduce the concept of **commutation**. In the mathematics of Pauli matrices, two operators either commute (meaning their order of application does not matter) or **anti-commute** (meaning swapping their order introduces a minus sign). Because both stabilizers and errors are Pauli operators, we can determine commutation properties for them.

The algebraic property of commutation is the core mechanism of error detection. If a quantum system is stabilized by an operator S , we can periodically measure S to check the health of the qubits connected to it:

- If no error has occurred, or if an error occurs that *commutes* with S , the state remains a valid $+1$ eigenstate. The measurement returns

In linear algebra, an eigenvector $|\psi\rangle$ of an operator S satisfies $S|\psi\rangle = \lambda|\psi\rangle$, where λ is a constant called the eigenvalue. In the stabilizer formalism, we strictly look for states where $\lambda = +1$. Because the state remains unchanged, measuring S will yield a deterministic outcome of $+1$, indicating no error. It’s important to note that measuring the $+1$ usually results in a 0 value for the syndrome bit.

Mathematically, two operators A and B commute if $AB = BA$, and anti-commute if $AB = -BA$. For Pauli matrices, identical types commute (e.g., an X error commutes with an X stabilizer), but different types acting on the same qubit anti-commute (e.g., an X error anti-commutes with a Z stabilizer).

+1, telling the decoder that those specific qubits are safe (or not affected by an error detectable by S).

- If a physical error occurs that *anti-commutes* with S , the error flips the sign of the state. The measurement will return -1 , acting as a localized alarm that an error has corrupted the monitored qubits.

We can see that if we can construct a set of operators that can detect all the possible errors affecting our physical qubits, we can determine the operations required to correct them.

The Stabilizer Group and Generators

To build these stabilizer operators, we use the n -qubit Pauli group, denoted as $\mathcal{P}_n$. This group consists of all possible tensor products of the standard Pauli matrices (I, X, Y, Z) applied to n qubits, multiplied by a global phase factor $(\pm 1, \pm i)$.

Under this formalism, a quantum error correction code is defined by its **Stabilizer Group** ($\mathcal{S}$), which is a commuting subgroup of $\mathcal{P}_n$, that is, a subset of $\mathcal{P}_n$ for which all its members commute between them.

The reason why this is a strict requirement is that, for a QEC code to be valid, all the operators in the group must share a common set of eigenvectors with eigenvalue $+1$. These eigenvectors are the logical states of the QEC code, and are not affected by any of the stabilizer operators. Only commuting operators can share a common set of eigenvectors.

Additionally, for $\mathcal{S}$ to define a valid code, the operator $-I$ must not be an element of the group. If $-I$ was an element of $\mathcal{S}$, then we would have $-I|\psi\rangle = |\psi\rangle$. This only holds if $|\psi\rangle = 0$, meaning the code would have no valid quantum states.

We've defined the $\mathcal{S}$ as a set of commuting operators. However, we do not need every single operator in $\mathcal{S}$ to define the code. We only need a small, independent set of operators called **generators**. Just as a three basis vectors can define an entire 3D space, multiplying these generators together produces all the other stabilizers in the group. This set of independent operators represents the set of detectors that we will have to implement in the quantum circuit.

For an n -qubit system, choosing $n - k$ independent commuting generators uniquely defines a QEC code capable of encoding k qubits. Such generators create a protected logical subspace with 2^k logical states.

Logical Operators and Code Construction

Once the stabilizer group $\mathcal{S}$ is defined, we need a way to manipulate the encoded quantum information without leaving the protected logical subspace. This is achieved through **logical operators**.

A logical operator must commute with all the stabilizers in $\mathcal{S}$, ensuring that applying it does not trigger any error detection alarms (it leaves the state in the $+1$ eigenspace of all stabilizers). However, unlike stabilizers, non-trivial logical operators are not part of $\mathcal{S}$ itself. Furthermore, pairs of logical operators acting on the same logical qubit (such as a logical X_L and a logical Z_L) must anti-commute with each other, mimicking the standard behavior of physical Pauli operators.

A crucial property of the stabilizer formalism is that logical operators are not unique. Because any stabilizer $S \in \mathcal{S}$ acts identically as the identity operator on valid codewords, we can multiply any logical operator L by any stabilizer S to obtain a new operator $L' = LS$. Both L and L' will have the exact same effect on the logical state. We say that L and L' are **equivalent logical operators**.

This multiplicity leads to a rigorous definition of a code's robustness. The **distance** of a stabilizer code is defined as the minimum weight (the minimum number of physical qubits affected) among all possible non-trivial logical operators. In other words, out of all equivalent ways to perform a logical operation, the distance is the size of the "shortest" one. If an error chain happens to match this shortest logical operator, it will perform a valid logical operation without triggering any syndrome, effectively corrupting the data silently.

This implies that we cannot just pick any random combination of commuting Pauli strings to build a quantum code. A bad choice of $n - k$ generators might result in a code with a very small distance (e.g., $d = 1$ or $d = 2$), meaning it provides little to no protection.

To systematically select a generator group that guarantees the required

commutation and anti-commutation rules and achieves a high distance d , researchers map the problem to mathematical structures. These frameworks allow us to generate families of codes that can be scaled to arbitrary distances systematically. Prominent examples include **topological codes** (where qubits and stabilizers are arranged in geometric lattices, such as the Surface Code or the Color Code that we will explore in this work) and codes based on algebraic structures with commutative properties, such as polynomial rings used in **quantum Low-Density Parity-Check (qLDPC)** codes.

Standard $[[n, k, d]]$ Notation

Note that the parameter n does not include the number of ancilla qubits required to extract the syndrome, as it depends on the actual implementation.

In the field of Quantum Error Correction, stabilizer codes are conventionally described using the $[[n, k, d]]$ notation, which provides a summary of the code's cost and capacity:

- **n (Physical Qubits):** The total number of physical qubits used to construct the code.
- **k (Logical Qubits):** The number of logical qubits encoded.
- **d (Code Distance):** The minimum number of physical errors required to cause an undetected logical failure.

Under this convention, the Shor code is formally classified as a $[[9, 1, 3]]$ code, meaning it uses nine physical qubits to protect one logical qubit with a distance of three, and requires eight $(n - k)$ stabilizers.

The distance directly determines the error-handling capabilities of the code through two fundamental rules:

1. **Detection:** A code can detect any error affecting up to $d - 1$ physical qubits.
2. **Correction:** A code can correct up to t physical errors:

$$t = \left\lfloor \frac{d - 1}{2} \right\rfloor \quad (1.2)$$

For the Shor code ($d = 3$), these rules imply that the code can detect up to 2 errors and correct $t = 1$ error. If two errors occur simultaneously, the syndrome would cause a wrong correction. This is a permanent trade-off

in QEC: increasing the correction threshold t , requires increasing the distance d , which requires more physical qubits n .

1.5 The Threshold Theorem

For a QEC code to effectively mitigate the errors, the error introduced by the overhead QEC in the circuit must be lower than the error effectively corrected, so the resulting logical error is below the physical error rate.

The distance of a code relates to the capacity of the code to reduce the error rate, but it is also correlated with the code overhead: to increase the distance, we need to add more physical qubits, n , to codify the same number of logical qubits, k .

As we increase the complexity of quantum computations, we need quantum circuits that are wider (more qubits) and deeper (more steps in the calculation). Because the probability of error is constant through the execution, during the execution of an arbitrarily deep circuit, a non-recoverable error will eventually happen.

Proven by Aharonov and Ben-Or [12], the Threshold Theorem gives us a framework for arbitrarily long quantum computations.

It proves that when we increase the distance of a code, hence introducing overhead but improving the code capacity, the logical noise rate that we obtain behaves differently depending on the physical error rate that our hardware has:

- If the physical error rate, p is below a certain value, p_{th} , which depends on the correction protocol itself, increasing the distance reduces the logical noise. That is, the benefit of increasing the code capacity is greater than the drawback of increasing the circuit overhead.
- If p is above p_{th} , increasing the distance increases the logical noise. That is, the benefit of increasing the code capacity is smaller than the drawback of increasing the circuit overhead.

The value at which the behavior changes is what we call the Threshold, p_{th} , of a QEC code. We can conclude that, if we have hardware whose physical error rate p is below the threshold p_{th} of a certain code, we can

Note that the threshold, p_{th} , of a given code is not an isolated property of the code. It depends on the actual implementation of the code in a certain hardware. In Chapter 2 we will show how mapping a given code to a given system (a set of qubits with a given inter-connectivity) adds overhead that change the Threshold of the result.

arbitrarily increase the distance of such code to reduce the effective noise. This compensates the increase in the chances of a non-recoverable error (when a circuit depth increases) by increasing the distance of the code.

In Chapter 5, we will simulate various QEC codes to visualize this phase transition. By plotting the logical error rate p_L as a function of the physical error rate p for different distances, we will numerically identify the threshold p_{th} as the precise crossing point where scaling the distance inverts its effect on logical noise.

State of the art of QEC codes: Surface codes, color codes and QLDPC codes

2.1 Introduction

In Chapter 1, we established the fundamental principles of quantum error correction (QEC), learning how we can encode logical information into multiple physical qubits to protect it from noise. However, finding the right code for a real-world quantum computer is a delicate balancing act. A good QEC code must offer high fault-tolerance thresholds and error suppression capabilities, require hardware connectivity that is actually physically possible to build, and maintain a reasonable encoding rate so that we do not need a very large number of physical qubits for each logical one we encode. Additionally, the decoding process must run within the bounds imposed by the quantum gate execution times in the actual quantum computer to allow a real-time decoding and correction of errors.

This chapter explores the evolution of these codes, focusing on three

major families: Surface codes, Color codes, and Quantum Low-Density Parity-Check (QLDPC) codes.

2.1.1 About terminology: Topological vs. LDPC Codes

Before analyzing specific codes, we must clarify the terminology we use to classify them:

- **Topological Codes:** In a topological code, qubits are arranged on a physical lattice (usually in 2D or 3D). Crucially, the stabilizer checks only interact with strictly local, geometrically adjacent qubits. Surface codes and 2D color codes are the quintessential examples of topological codes.
- **Quantum Low-Density Parity-Check (QLDPC) Codes:** A code is considered LDPC if the number of qubits involved in any single parity check (its weight), and the number of checks acting on any single physical qubit, are both strictly bounded by a constant, regardless of the size of the code. Mathematically, surface codes and color codes perfectly fit this definition.
- **Non-topological QLDPC Codes:** These codes sacrifice the strict 2D geometric locality in favor of long-range connections. Doing so allows them to break theoretical limits and achieve asymptotically “good” encoding rates (where the number of logical qubits scales linearly with the physical ones).

2.2 Transversality and the Eastin-Knill Theorem

A central goal in Quantum Error Correction (QEC) is executing logical gates fault-tolerantly. A gate is called *transversal* if applying the physical gate independently to each physical data qubit results in the logical version of that gate being applied to the encoded state. Transversal gates are the holy grail of fault-tolerance because they never propagate errors between qubits within the same code block. Because the operations are performed strictly on isolated pairs of physical qubits (qubit-by-qubit), an error cannot spread to other physical qubits within the same logical

In modern literature, when researchers say “QLDPC codes”, they are often referring colloquially to *non-topological* QLDPC codes.

block. Thus, a single physical error remains a single physical error, which the error-correcting code can safely detect and fix.

However, the **Eastin-Knill Theorem** [13] states a severe theoretical limitation: *No quantum error-correcting code can have a universal set of transversal logical gates.* This means we cannot build a universal quantum computer relying solely on these simple, inherently safe transversal operations.

When a required logical gate is non-transversal, we cannot simply interact the physical qubits of one logical block with another pairwise. Instead, we must rely on complex fault-tolerant protocols that interact deeply with the internal structure and boundaries of the logical qubits. These protocols are challenging to implement and are prone to causing error propagation. Depending on the nature of the target operation, two main techniques are employed:

Multi-Qubit Gates: Lattice Surgery

To perform non-transversal operations between two distinct logical qubits (for example, a logical CNOT in codes that do not support it transversally), the standard approach is **Lattice Surgery**. Rather than applying physical gates between individual pairs of qubits across the blocks, lattice surgery temporarily “merges” two distinct logical qubits into one larger logical block by measuring multi-qubit parity checks along the spatial boundary separating them.

After the logical operation is completed, the blocks are “split” again. While this protocol is fault-tolerant in theory, it is practically demanding. It requires executing delicate measurement cycles along the boundaries of the code patches, demanding extra routing and exposing the logical qubits to complex error propagation dynamics during the merging and splitting phases.

Single-Qubit Gates: Magic State Injection and Distillation

To implement non-transversal single-qubit gates—most notably the non-Clifford T -gate, which is essential for universal quantum computation—we use **Magic State Injection** [14]. Instead of applying the gate directly

In Lattice Surgery, merging two logical qubits means activating ancillas between the borders of the two logical qubits and creating two qubit gates between these ancillas and data qubits of the two adjacent logical qubits, effectively creating a chain of stabilizers through the boundary of both qubits. The subsequent measurement of these stabilizer chain is equivalent to measuring the joint logical operator for both qubits. The act of measuring it is what causes the projection that leaves both logical qubits in an entangled state.

to our data qubit, we prepare an auxiliary logical qubit in a special state called a “magic state”. We then entangle our data qubit with this magic state (often using lattice surgery) and measure the auxiliary block, effectively teleporting the non-Clifford operation onto our data.

The critical issue is that preparing a magic state fault-tolerantly directly from physical qubits initially yields a “noisy” state. If we inject a noisy state into our computation, the logical algorithm will fail. Therefore, before injection, these states must be purified using a process called **Magic State Distillation**.

Distillation is a highly iterative and wasteful protocol that consumes several noisy magic states to produce a single, higher-fidelity state. For instance, a standard distillation protocol might consume 15 noisy states to yield just 1 usable state. Because useful quantum algorithms require large numbers of T -gates, a fault-tolerant quantum computer must continuously manufacture these states in real-time.

To achieve this, quantum processors must dedicate enormous, specialized regions—often referred to as *Magic State Factories*—exclusively to this task. This immense spatial and temporal overhead is precisely what motivates the search for alternative QEC families. For instance, some configurations of color codes [7] inherently support a much richer set of transversal gates, significantly reducing or entirely bypassing the need for extensive magic state distillation.

2.3 Surface Codes

2.3.1 From 1D Repetition to 2D Topologies

In Chapter 1, we learned that a 1D linear repetition code can only protect against one specific type of error (either bit-flips or phase-flips). Historically, protecting against both required complex concatenation schemes, like Shor’s 9-qubit code. The surface code solves this elegantly by expanding into a second spatial dimension. By distributing qubits across a 2D grid, the code can interleave two different types of parity checks simultaneously, monitoring both X and Z errors without the recursive and inefficient overhead of concatenation.

An example of the distillation overhead is visible in the 2021 article by Gidney & Ekerå[15]. In this paper, they propose the factorization of a 2048 bit RSA integer using a quantum processor with 20 million qubits. The article estimates that 25% of these qubits should be dedicated to Magic State distillation.

2.3.2 Origins: The Toric Code

This 2D topological approach was famously introduced by Alexei Kitaev in 1997 via the **Toric Code** [6]. It is typically defined on a 2D square lattice where the data qubits reside on the edges. The parity checks (stabilizers) are defined on the vertices (X -checks) and the faces or plaquettes (Z -checks). In this geometric arrangement, every data qubit is continuously monitored by four ancilla qubits: two responsible for X -checks and two for Z -checks. Conversely, every ancilla qubit monitors exactly four adjacent data qubits.

This specific topology is not arbitrary; it is meticulously designed to satisfy the fundamental requirement of stabilizer quantum codes: all parity checks must commute. Because X and Z Pauli operators anticommute on a single qubit, an X -check and a Z -check will only commute if they overlap on an *even* number of shared data qubits. In this lattice geometry, any vertex (X -check) and any adjacent face (Z -check) will always share exactly two data qubits, ensuring that all operators commute globally and the checks can be measured simultaneously without collapsing the logical state.

To avoid the mathematical complexity of handling open boundaries at the edges of the grid, Kitaev imagined a lattice with periodic boundary conditions. In this model, the top edge of the lattice connects to the bottom edge, and the left edge connects to the right edge. Topologically, joining these opposing edges forms a continuous closed surface in the shape of a torus.

2.3.3 Observables, Capacity, and Code Distance

In topological codes, logical operations (logical gates and measurements of the qubit) are not applied to single physical qubits. Instead, logical observables are defined as continuous chains or *strings* of Pauli operators applied sequentially along a path of adjacent data qubits.

A closed string of operations that forms a small, trivial loop on the surface is simply a combination of stabilizers and has no effect on the encoded logical state. However, a string that wraps entirely around the torus—a *non-contractible loop*—fundamentally changes the logical state.

As formulated by Bombín & Martín-Delgado [16], surface codes can also be represented as a lattice with colored faces. In this configuration, data qubits reside on the vertices and stabilizers are defined on the faces, with the color of each face indicating the type of stabilizer. This is known as the rotated surface code representation and is now widely adopted in the literature.

These global, non-contractible strings represent our logical operators (X_L and Z_L). Figure 2.1 shows this.

When visualizing this topology on a 2D grid, these two distinct non-contractible loops correspond to the orthogonal directions of the lattice (East-West and North-South). On a torus, for each of these two directions, two operators, X_L and Z_L , can be applied, effectively codifying 2 qubits (each with 2 operators).

The number of logical qubits k a code can store depends strictly on the topology (the genus) of the surface. A torus has exactly two independent paths to wrap a string (one around the central hole, and one through the tube), meaning the toric code natively encodes exactly $k = 2$ logical qubits, regardless of its physical size. We can therefore characterize this toric code as a $[[2d^2, 2, d]]$ code.

To increase the protection of the code, we simply increase the density of the grid. By adding more physical qubits, the topological shape (the torus) remains identical, but the length of the shortest non-contractible string grows. This minimum string length is exactly the **code distance** (d). As the grid grows, it takes a longer chain of consecutive physical errors to accidentally form a logical operator, thereby increasing the fault-tolerance while preserving the underlying topological properties.

2.3.4 Planar Implementation and Rotated Surface Codes

While the toric code is mathematically beautiful, manufacturing a quantum processor shaped like a 3D torus is physically impractical, particularly for 2D planar architectures like superconducting circuits.

To implement this on a real chip, we must “cut” the torus and lay it flat. This physical cut transforms the Toric Code into the planar **Surface Code**. However, cutting the torus exposes physical borders. To preserve the ability to correct both X and Z errors, the planar patch must have two types of alternating boundaries: “rough” boundaries (where Z -strings terminate) and “smooth” boundaries (where X -strings terminate). Because the closed loops of the torus are now severed into linear strings that span from one boundary to the opposite, the storage capacity drops: a standard planar patch encodes only $k = 1$ logical qubit.

To further optimize the mapping onto real hardware, the planar surface code is almost universally implemented in its **rotated** version. While this rotated representation perfectly preserves the underlying topological properties, it drastically improves spatial efficiency. Because the code distance d is defined purely by the minimum length of the string connecting opposite boundaries, rotating the orientation of these boundaries by 45 degrees eliminates the wasted corner qubits present in the standard planar cut. Consequently, the rotated surface code requires significantly less physical qubits to achieve the exact same distance d (see Figure 2.1).

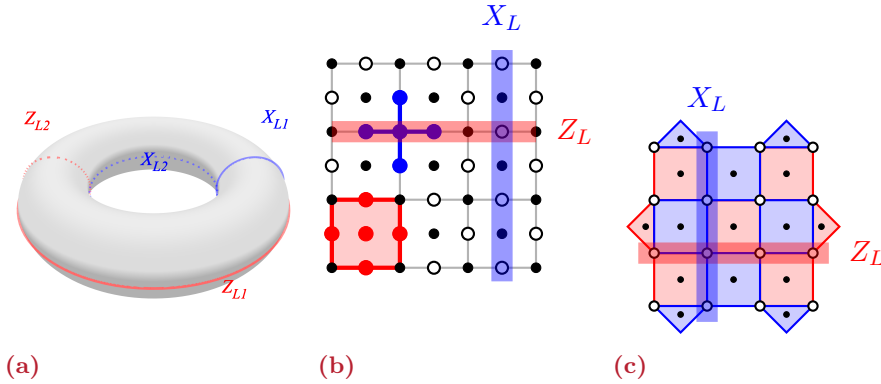


Figure 2.1. Evolution and representations of the surface code. (a) The toric code representation, illustrating how logical observables correspond to non-contractible continuous strings wrapping around the two fundamental cycles of the torus. (b) The original planar formulation by Kitaev ($d = 4$), with data qubits on the edges and checks on vertices and faces. The red square represents a Z -stabilizer and the blue cross represents a X -stabilizer. A total of 49 physical qubits are needed. (c) The equivalent rotated surface code, placing data on vertices and alternating checks on colored faces to significantly reduce physical overhead. Red faces represent Z -stabilizers and blue faces represent X -stabilizers. The blue and red lines represent the minimum string for the respective operators. In this case, only 31 physical qubits are required. In figures (b) and (c) for a $d = 4$ code, white circles represent data qubits and black circles represent ancilla qubits. The blue and red lines represent the minimum strings for the respective logical operators.

2.3.5 Advantages and Disadvantages

The surface code's primary advantage is its high error threshold (around $\sim 1\%$ for circuit-level noise, see [17]) and its locality, relying on strictly

nearest-neighbor 2D grid connectivity. This spatial locality is perfectly aligned with the 2D fabrication constraints of modern planar architectures, making it the industry standard baseline.

However, its main disadvantage is its terrible encoding rate. A standard planar surface code patch encodes exactly one logical qubit ($k = 1$), regardless of how much we scale its distance d . To achieve higher distances and exponentially better protection, the physical footprint grows quadratically ($n \approx d^2$). This massive spatial overhead limits the scalability of surface codes for algorithms requiring thousands of logical qubits when large distances are needed to mitigate physical error.

2.4 Color Codes

2.4.1 Origins and Topologies

Introduced by Bombín and Martin-Delgado [7], topological color codes represent a powerful evolution in the landscape of quantum error correction. They are defined on 2D lattices that strictly satisfy two geometric properties: they must be 3-valent and 3-colorable (typically utilizing red, green, and blue).

Unlike the surface code, which physically separates X -checks and Z -checks by placing them on vertices and faces respectively, color codes place both X and Z checks on the same faces of the lattice, while data qubits reside exclusively on the vertices.

This overlapping check structure is mathematically robust due to the inherent geometry of these lattices. In any 3-valent, 3-colorable lattice, every face is guaranteed to have an even number of vertices. Consequently, an X -check and a Z -check defined on the exact same face will overlap on an even number of data qubits, guaranteeing that they commute. Furthermore, any two adjacent faces will always share exactly one edge (which contains exactly two vertices). Thus, an X -check on one face will always share two data qubits with a Z -check on an adjacent face, ensuring global commutativity across the entire code block, as required by the Stabilizer Formalism presented in Section 1.4.

A trivalent lattice is a lattice in which each vertex is connected to exactly three edges. A 3-colorable lattice is one where, using only 3 colors, faces can be colored so that two adjacent faces never share the same color.

2.4.2 2D Color Code Configurations

Because of the strict mathematical requirement that the lattice must be 3-valent and 3-colorable, there is only a finite number of uniform tilings (Archimedean tilings) in the 2D plane that can support a color code, see [18]. Specifically, there are exactly three configurations, shown in Figure 2.2:

- **6.6.6 Lattice:** A regular lattice composed entirely of hexagons. This lattice can be characterized as $\left[\left[\frac{3d^2+1}{4}, 1, d\right]\right]$.
- **4.8.8 Lattice:** A semi-regular lattice where each vertex connects a square and two octagons. It can be characterized as $\left[\left[\frac{d^2+2d-1}{2}, 1, d\right]\right]$.
- **4.6.12 Lattice:** A more complex semi-regular lattice composed of squares, hexagons, and dodecagons, characterized as $\left[\left[\frac{3d^2-6d+5}{2}, 1, d\right]\right]$. While theoretically interesting, its extremely high-weight stabilizers (weight-12) make it practically nonviable for current hardware.

While there are only three uniform tilings, an infinite number of 3-valent, 3-colorable lattices can be defined if uniformity is not required. These are rarely studied in the literature because they are more difficult to scale and fit on a quantum chip with a regular pattern and because their error correction properties are less predictable.

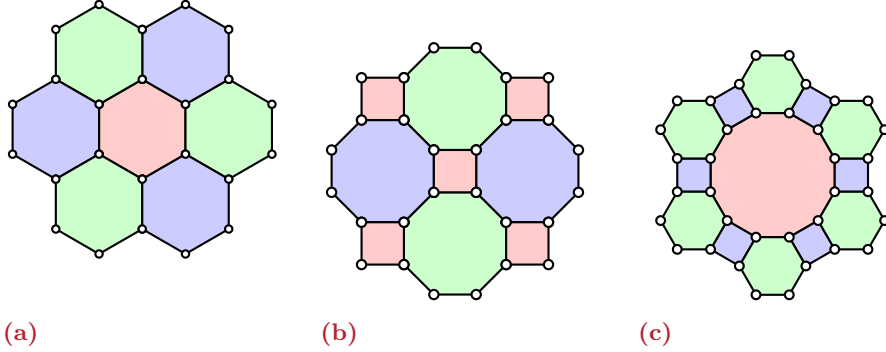


Figure 2.2. The three valid 2D configurations for topological color codes: the 6.6.6 hexagonal lattice, the 4.8.8 square-octagonal lattice, and the 4.6.12 lattice. The faces are 3-colored (e.g., red, green, blue) such that no two adjacent faces share the same color. Data qubits are represented with white circles. There are two ancilla qubits in each colored face, which are not represented in the image, which can actually be implemented only with one ancilla if they are reused for Z and X detectors.

The 4.8.8 lattice presents unique properties that make it particularly interesting, which motivates our decision of performing our simulations

with this lattice:

- Lower qubit requirements: For the 4.8.8, 6.6.6, and 4.6.12 color code geometries, the number of data qubits n as a function of the distance d is given by $n_{4.8.8} = \frac{d^2+2d-1}{2}$, $n_{6.6.6} = \frac{3d^2+1}{4}$, and $n_{4.6.12} = \frac{3d^2-6d+5}{2}$, respectively, with the 4.8.8 lattice being the one with best asymptotical ratio.
- As a doubly-even code, the 4.8.8 lattice enables a transversal implementation of the full Clifford group, including H , $S^\dagger$ and CNOT. While all 2D color code lattices allow for transversal Clifford group gates, only the 4.8.8 geometry allows the S gate to be implemented not only transversally, but uniformly (i.e., equally implemented) across all physical qubits. Lattices containing faces whose number of sides is not a multiple of four (such as the hexagons in the 6.6.6 and 4.6.12 codes) require the transversal operation to alternate between S and $S^\dagger$ gates (i.e. a non-uniform implementation).

A classical binary linear code is *doubly-even* if the Hamming weight of every codeword is a multiple of 4. In quantum CSS codes, this ensures all stabiliser generators act on a multiple of 4 qubits, which is the necessary condition to enable the uniform transversality of the phase gate S .

2.4.3 Topological Observables: Colored Strings and Nets

Just as in the surface code, logical observables in color codes are formed by continuous paths of Pauli operators acting on the data qubits. However, the 3-colorability of the lattice introduces a difference, as two qubits of the string can be connected through an edge or through a face.

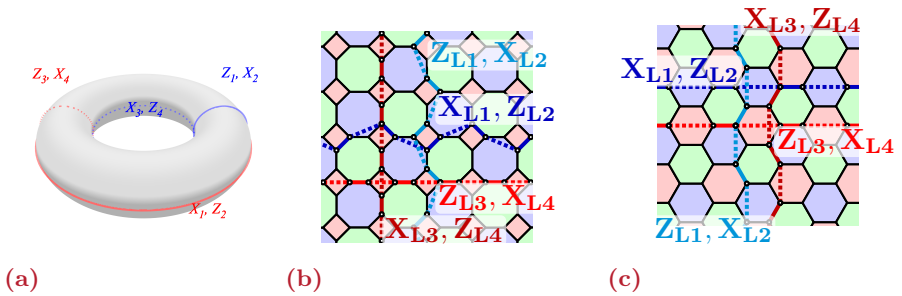


Figure 2.3. Logical observables in a toric color code. (a) Logical observables correspond to color-independent non-contractible continuous strings wrapping around the two fundamental cycles of the torus. Color codes allow us to codify two logical operators in the same string, with the logical observables for each qubit being orthogonal. (b) The 4 independent strings in a 4.8.8 lattice. (c) The 4 independent strings in a 6.6.6 lattice.

In a color code, strings are *colored*. A string can only be a border between faces of the other two colors, but it can go through faces of its color. For example, a “red” string of Pauli operators will strictly run along edges that separate green and blue faces, and will cross red faces.

This color property leads to a unique topological feature: **string branching**. A string of one color can “unfold” or branch into two strings of the remaining colors at any vertex. For instance, a red string can split into a green string and a blue string, forming a structure known as a *net*. This topological flexibility means that logical operators do not have to be simple linear loops; they can be complex, branching networks of operators that span the lattice.

These strings and nets define the logical operators. When two non-contractible nets of X and Z operators intersect at an odd number of data qubits, they anticommute, forming a valid logical qubit. Because of this rich combinatorial structure, a color code is twice as dense as a surface code. While a surface code wrapped on a torus natively encodes $k = 2$ logical qubits, a color code on the exact same toroidal surface encodes $k = 4$ logical qubits (see Figure 2.3). For color codes planar implementations, triangular patches are typically used, which encode $k = 1$ logical qubit but offer remarkably different boundary symmetries compared to the square patches of surface codes (see Figure 2.4). A square patch color code would encode $k = 2$ logical qubits.

Despite the lattice being three-colorable and supporting strings of three distinct colors, only two of these colors are strictly independent from an error-correction perspective. In a 2D color code, every data qubit is located at a vertex shared by exactly three faces—one red, one green, and one blue. Consequently, any single error will always trigger a defect in exactly one face of each color. Because these syndrome defects are inherently created in triplets, the parity information gathered from any two colors fully determines the state of the third.

2.4.4 Advantages: Transversality and 3D Topologies

Color codes offer massive theoretical advantages over surface codes, primarily regarding logical gate execution. As discussed earlier in this chapter, finding transversal gates is highly desirable to prevent error

propagation.

Color codes natively possess a fully transversal Clifford gate set, meaning that critical operations like the Phase gate (S), Hadamard (H), and CNOT can be applied strictly pairwise, without the need for complex and noisy lattice surgery.

Because of the Eastin-Knill Theorem, no QEC code can provide a universal set of transversal gates. Consequently, 3D color codes must sacrifice the transversal H gate (which are easier to distill) in order to support a transversal T gate. To circumvent this and achieve full universality without distillation, techniques like *dimensional jumping* are employed. This allows the code to transition between 2D and 3D topologies, applying transversal H gates while in the 2D configuration.

Furthermore, the color code topology can be generalized into three dimensions (3D). In 3D configurations, certain color codes allow the transversal execution of non-Clifford gates, such as the T -gate. This is a very significant advantage, as it bypasses the need for Magic State Distillation factories required by the surface code.

2.4.5 Disadvantages and Processor Mapping

Despite their theoretical superiority, 2D color codes face significant physical implementation hurdles. Their circuit-level threshold is generally lower than that of the surface code [18], meaning they require underlying hardware with significantly lower baseline noise to begin correcting errors effectively.

Moreover, the stabilizers acting on the faces involve a high number of qubits. For instance, in the 4.8.8 lattice, the octagonal faces represent weight-8 parity checks. Measuring a weight-8 stabilizer dynamically without exposing the data to correlated errors is a mapping challenge on modern hardware, which typically features restricted 2D grid connectivity.

Implementing these codes on real processors requires highly optimized, specialized layouts. For the **4.8.8 lattice**, Fowler [19] demonstrated that it is possible to map this topology onto a standard 2D square architecture, although it requires replacing the ancilla qubit in octagonal faces with 5 ancilla qubits, 4 of them connected to two data qubits each, and the fifth one connected to the 4 ancilla qubits ((see Figure 2.4)).

Similarly, the **6.6.6 hexagonal lattice** maps naturally to certain architectures, but it is still a non-trivial problem. Recent publications by Lacroix et al. [20] outline an optimized mapping for the 6.6.6 code.

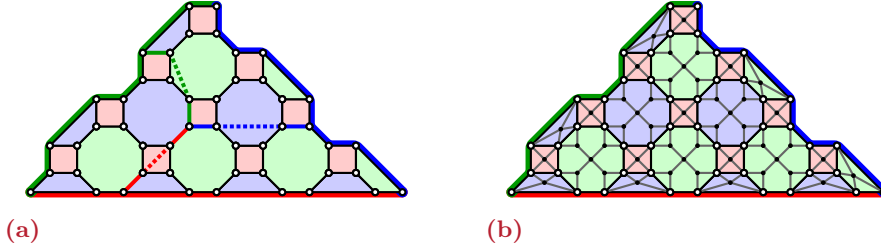


Figure 2.4. Planar topology of a 4.8.8 lattice color code. **(a)** Definition of the triangular patch for a planar topology, in which we must use string-nets to define the logical observables. Each string-net can encode the logical X and Z operators in the color code, but the individual strings must terminate at a boundary of their own color, which inherently requires branching. **(b)** Mapping of the color code patch to a quantum processor with a square grid of qubits. For the octagonal faces, we must use 5 ancillas to perform the stabilizer measurements. Ancilla qubits in the tetrahedrons that appear in the boundaries do not map the square grid and are intentionally simplified in this diagram.

2.5 Non-topological QLDPC Codes

While 2D topological codes are very hardware-friendly, their reliance on local 2D planes strictly limits their scalability. The fundamental bottleneck of local 2D codes is governed by the Bravyi-Poulin-Terhal (BPT) bound [21]. To build future quantum supercomputers with millions of qubits and increasing distances for longer computations, this geometric restriction imposes a prohibitive physical overhead.

This fact has drawn massive attention to non-topological Quantum Low-Density Parity-Check (QLDPC) codes. By sacrificing strictly local 2D connectivity and embracing long-range physical connections, these codes break the BPT bound. In a topological code, increasing the distance requires making the 2D patch larger, which squares the number of physical qubits. QLDPC codes, instead, can maintain a constant encoding rate while the distance grows linearly with the number of physical qubits. This removes the quadratic penalty for fault tolerance.

The BPT bound mathematically proves that for any quantum code restricted to local interactions in 2D, the parameters must satisfy $kd^2 \leq O(n)$. This implies that the physical overhead per logical qubit (n/k) must grow at least as d^2 . If higher error protection is needed (higher d), the physical footprint explodes quadratically.

2.5.1 Bivariate Bicycle Codes (BBC)

A prominent recent breakthrough in the practical domain is the Bivariate Bicycle Code (BBC) family, introduced by Bravyi et al. [22]. These codes prove that we can encode dozens of logical qubits with high distances using only a fraction of the physical qubits required by topological codes. In 2D topological codes like the Surface Code, the number of physical qubits n scales quadratically with the distance ($n \propto d^2$), which means that encoding k logical qubits requires scaling the hardware footprint dramatically as $O(kd^2)$. Non-topological QLDPC codes, and specifically BBCs, achieve a constant encoding rate $k/n \sim \text{const}$, meaning they scale linearly: the distance grows linearly with the number of physical qubits, bypassing the quadratic penalty.

To understand the massive savings: a typical BBC presented in [22] encodes $k = 12$ logical qubits with a distance of $d = 12$ using $n = 144$ data qubits (and 144 ancillas, totaling 288 physical qubits). To achieve the exact same protection and capacity ($k = 12, d = 12$) using standard surface codes, one would need roughly $12 \times (12^2 + 12^2) = 3456$ physical qubits. This represents more than a $10\times$ reduction in hardware footprint.

The core ingenuity of BBCs lies in how they guarantee that X -type and Z -type stabilizers commute, which is the most difficult challenge when designing any quantum code. In the stabilizer formalism, an X -stabilizer and a Z -stabilizer commute if and only if they overlap on an even number of physical qubits.

Instead of trying to manually guess which qubits to connect to satisfy this rule, BBCs use a clever mathematical trick. The physical data qubits are divided into two equal groups (the two “bicycles”, typically referred to as the Left and Right blocks). Each block is mathematically treated as a virtual 2D grid of dimensions $\ell \times m$ (rows and columns), containing $\ell \cdot m$ qubits. The parity check matrices are then constructed using two commutative matrices, A and B , which are derived from polynomials with two variables, x and y . The X -type parity check matrix is defined as $H^X = [A \mid B]$, and the Z -type parity check matrix is defined as $H^Z = [B^T \mid A^T]$.

Because the matrices A and B are built from commutative polynomials,

A **parity check matrix** is a mathematical representation of the stabilizer group. In this matrix, each row represents a single stabilizer generator (a parity check) and each column represents a physical data qubit. A 1 at position (i, j) means that the i -th check operator acts on the j -th qubit.

they commute ($AB = BA$). When computing the overlap between X and Z checks, the mathematical structure yields $H^X(H^Z)^T = AB + BA$, which trivially equals 0 (mod 2). This elegantly and automatically guarantees that any X -stabilizer and Z -stabilizer will always intersect on an even number of qubits, ensuring commutation without manual tweaking.

Physically, the variables x and y in the polynomials simply represent cyclic geometric shifts: x acts as an instruction to “shift one position to the right” and y means “shift one position down” on a virtual grid. A polynomial like $A = 1 + x + y^2$ acts as a wiring instruction that tells the hardware: “to build this stabilizer, apply the operator to a given qubit (1), the qubit to its right (x), and the qubit two steps down (y^2)”.

By keeping these “wiring instructions” very short (e.g., using exactly three terms for A and three terms for B), the total weight of every parity check is strictly bounded to exactly $3 + 3 = 6$ qubits, regardless of how large the total code (the grid) becomes. This naturally fulfills the Low-Density Parity-Check (LDPC) requirement, which is essential to prevent error propagation during syndrome measurements.

Their physical implementation, however, is still an open engineering challenge. Because the shifts dictated by the polynomials can connect qubits that are far apart on the grid, these codes require high-fidelity, long-range couplers. Current promising proposals to realize this non-local connectivity include dynamic optical interconnects or neutral atoms that are physically moved with optical tweezers. Therefore, non-topological QLDPC codes represent the ultimate frontier for next-generation fault-tolerant architectures.

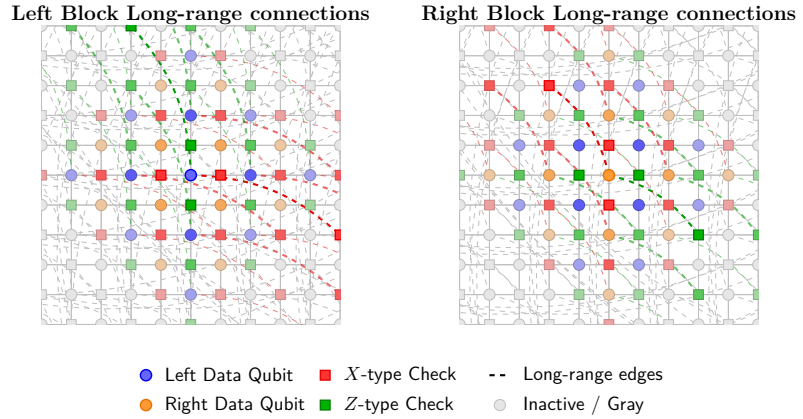


Figure 2.5. Construction of Bivariate Bicycle Codes. The lattice is composed of two identical blocks of data qubits (Left and Right), which are shown side-by-side to independently illustrate their long-range connections. To visualize this periodic connectivity clearly, a central “principal” data qubit is highlighted in each block: its long-range connections to the X and Z checks are drawn with full opacity. The connections for its surrounding neighbors are drawn with a fading gradient (progressively lighter) to help visualize the periodic structure.

Modeling quantum error

In order to design, evaluate, and compare Quantum Error Correction (QEC) protocols, we must first establish a framework to describe how errors occur in quantum systems. A decoder cannot be effectively designed or benchmarked without a clear definition of the noise it is expected to correct.

Simulating continuous noise for large-scale systems is computationally intractable. Fortunately, as discussed in Section 1.2, quantum errors can be discretized into a finite set of Pauli errors (X , Y , and Z). Therefore, in the context of QEC simulation, we can safely model noise as discrete probabilistic Pauli operations.

3.1 Types of quantum noise

Before deciding where errors occur in a circuit, we must define the probability distribution of the errors themselves. The choice of noise type drastically affects the performance of both the QEC code and the decoder.

3.1.1 Depolarizing noise

The depolarizing channel is widely used for benchmarking decoders because it provides a “worst-case” scenario of symmetric uncertainty. If a decoder performs well under depolarizing noise, it is generally robust across different hardware platforms.

The depolarizing channel is the most standard noise model used in QEC literature due to its symmetry and mathematical simplicity. Under this model, a qubit has a probability p of suffering an error, and when an error occurs, it is equally likely to be an X (bit-flip), Z (phase-flip), or Y (both) error. Thus, each specific Pauli error occurs with probability $p/3$.

3.1.2 Biased noise

Real quantum hardware rarely exhibits perfectly symmetric noise. In many physical qubit implementations, such as superconducting transmons, the quantum state is far more susceptible to losing its phase information than to losing its energy. This results in **biased noise**, where one type of error (typically Z phase-flips) is significantly more probable than the others.

Bit-flip noise

A special type of biased noise is one where only X -type errors can occur. This is specially relevant in our case as bit-flip only, code capacity models are the standard in the literature to compare across different decoders. The bit-flip noise model is specially adequate when only X -type errors can affect the observable (i.e. the final observable is measured in the computational basis).

The XZZX surface code has stabilizers that combine Z and X detectors and is therefore not a CSS code.

When the noise is highly biased, standard CSS codes may not be optimal. Instead, tailored codes (like the XZZX Surface Code) or tailored decoders can exploit this asymmetry by assigning lower weights to Z errors, vastly improving the threshold for that specific hardware.

3.1.3 Correlated noise

Standard simulations usually assume that errors are independent and identically distributed, meaning the probability of an error on one qubit does not depend on the state of its neighbors. However, physical systems suffer from **correlated noise**, such as cosmic ray impacts or crosstalk between neighboring microwave lines, which can cause simultaneous errors on multiple adjacent qubits. While more difficult to model compu-

tationally, analyzing correlated noise is critical because multi-qubit errors can easily form logical strings that bypass the topological protection of the code.

3.2 Error Models

While the *type* of noise defines the probability distribution of the Pauli operators, the *error model* defines the abstraction level of the circuit where these errors are injected. We classify error models into three hierarchical categories: code-capacity, phenomenological, and circuit-level noise.

3.2.1 Code-capacity noise

The **code-capacity model** is the simplest and most abstract error model. It assumes that all quantum gates, measurements, and state preparations are perfectly flawless. Errors are only injected into the data qubits while they are “idle”.

Under this model, the syndrome extracted by the ancilla qubits is always 100% reliable. Because measurements never fail, multiple rounds of the simulation process (i.e. analyzing the error caused through multiple idle cycles of the quantum computer) are equivalent to one single round at a higher physical noise level. Therefore, under this model the decoder only needs to solve a 2D spatial problem (i.e. matching the errors on the lattice, with no time dimension).

While physically unrealistic, the code-capacity model is an essential theoretical tool. It isolates the raw error-correcting capability of the code’s geometry and the base performance of the spatial decoder, without the confounding effects of faulty syndrome extraction circuits.

3.2.2 Phenomenological noise

In reality, the classical measurement hardware and the ancilla qubits used to extract the syndrome are also susceptible to noise. The **phenomenological noise model** builds upon the code-capacity model by adding **readout errors**.

In this model, data qubits suffer errors with probability p , and the classical syndrome bits obtained from the measurements are independently flipped with probability q (usually $p = q$). However, the entangling gates used to compute the parity checks are still assumed to be perfect.

Because the syndrome itself can now lie, the decoder can no longer trust a single measurement round. A faulty measurement could trick the decoder into applying an incorrect correction, creating a logical error. To solve this, the syndrome extraction must be repeated over time (typically d rounds for a distance- d code). The decoder must now resolve a 3D spatial-temporal graph, tracking how detection events appear and disappear across time steps.

3.2.3 Circuit-level noise

The **circuit-level noise model** is the most realistic and demanding simulation framework. It removes all assumptions of perfection. In this model, noise is injected into every physical operation:

- **Initialization:** Qubits may be prepared in the wrong state.
- **Idle operations:** Qubits suffer decoherence while waiting for other operations to finish.
- **Single-qubit gates:** A depolarizing channel is applied after every single-qubit rotation.
- **Two-qubit gates:** A two-qubit depolarizing channel is applied after every entangling gate (e.g., CNOT).
- **Measurements:** Measurement outcomes can be flipped, as in the phenomenological model.

Because of error propagation, circuit-level noise can generate correlated errors on the data lattice from a single independent fault. Decoders operating under circuit-level noise must be explicitly aware of the circuit scheduling (the specific order of the CNOT gates) to correctly anticipate and decode these correlated hook errors.

3.3 Relationship between the models

These three error models form a strict hierarchy. Circuit-level noise is the superset that encapsulates all physical reality. If we take a circuit-level simulation and artificially set the error rates of all gates and idle cycles to zero (leaving only measurement errors and a baseline data qubit error), we recover the phenomenological model. If we subsequently set

Circuit-level noise introduces a critical challenge: **error propagation**. A CNOT gate will propagate an X error from its control to its target, and a Z error from its target to its control. A single physical fault occurring in the middle of a syndrome extraction routine can propagate into a multi-qubit error on the data qubits (known as a “hook error”).

We use the $[p_{data} : p_{meas} : p_{gate}]$ notation to define the probability ratios between the three models. As an example, $[1 : 0.5 : 0]$ represents a phenomenological noise mode with data errors happening twice as much as measurement errors.

the measurement error rate to zero, we are left with the code-capacity model.

Throughout this thesis, we will primarily simulate non-correlated bit-flip noise (which is the most common approach in the literature). We will utilize the code-capacity model to analyze the fundamental topological properties of Color Codes. The exception will be the initial implementation in Chapter 5, in which we will use different noise models to illustrate its impact and various properties of the QEC codes.

Because for our simulations, the observable will strictly be in the computing basis (i.e. measured in the Z-axis), Z errors will not have an impact on the observable. We will provide results for both complete depolarizing noise (X, Y and Z errors) and for a simplified noise model where only X-type errors can occur. We will generally use the simplified noise model to compare the results. For an observable measured exclusively in the computation basis, the threshold relationship between complete depolarizing noise and X-type noise is expected to be $p_{th}^{X-type} = \frac{2}{3} * p_{th}^{depolarizing}$, as in an X-Y-Z model, only X and Y errors can affect the observable.

QEC decoders for color codes

Having established the foundational principles of quantum error correction, and having explored both the structure and characteristics of color codes and the physical reality of error models, it is now time to address the final critical component of the QEC cycle: the decoders. A decoder is the classical algorithm responsible for transforming a set of syndrome measurements into a concrete diagnosis. In other words, it formulates a prediction of the error that has affected the logical observable, or equivalently, it infers the correct logical observable given the faulty measured one. The execution speed and accuracy of the decoders are major bottlenecks in realizing large-scale fault-tolerant quantum computation, and there's often a trade-off between them. Speed is relevant because the decoding process must ideally keep pace with the quantum circuit operation, as sometimes errors must be corrected during the execution. However, to increase decoding speed, accuracy may be sacrificed.

To achieve high accuracy, a decoder must do more than just find any arbitrary error string that satisfies the observed syndrome; it must ideally identify the most probable error chain. At this point, we must introduce the concept of the *Decoding Graph* or **Detector Error Model**, DEM. The DEM is a structured representation that captures the specific

The *Decoder Error Model* is often referred to by different names in the literature: *Syndrome Graph*, *Decoding Graph* or *Matching Graph*. Because our focus is in QEC simulation, we will prefer the term **Detector Error Model** or DEM, which is the term used by `stim`. The DEM acts as a bridge between the physical noise phenomena (such as gate infidelities or readout errors) and the abstract graph or hypergraph structures used by most decoding algorithms to find the optimal correction.

probabilities of the different error mechanisms that trigger detection events in the quantum circuit. By incorporating a DEM, decoders can leverage these probability distributions to make informed decisions. This additional statistical context allows the decoder to determine the actual error chain with a significantly higher success rate.

In this chapter, we will not dive into the programming aspects or the specific implementation details of decoders. The practical implementation of a basic decoder will be thoroughly covered in Chapter 5, where it will also be compared against the standard Minimum Weight Perfect Matching (MWPM) algorithm to explicitly demonstrate the clear advantage of incorporating the DEM into the decoding process.

4.1 Minimum Weight Perfect Matching (MWPM)

MWPM relies in two fundamental things: **Perfect Matching**, meaning that it matches pairs of nodes with a set of edges, so no node is left unpaired and no node belongs to multiple matches, and **Minimum Weight**, meaning that the total weight of the selected set of edges is the minimum possible to perfectly match the active nodes.

The Minimum Weight Perfect Matching (MWPM) was the first algorithm to demonstrate that topological codes could be practically decoded. Its theoretical mapping to the surface code was established by Dennis, Kitaev, Landahl, and Preskill in 2002 [23]. MWPM is arguably the most famous and widely used decoder in the field of quantum error correction. Originally conceived as a solution to a classical optimization problem by Jack Edmonds in 1965 [24], MWPM is designed to find a perfect matching in a weighted graph such that the sum of the weights of the edges in the matching is minimized.

In the context of QEC, and specifically for the Surface Code, the decoding problem maps naturally to a standard graph problem. When a single Pauli error occurs on a data qubit in a surface code, it typically flips the measurement outcome of exactly two adjacent stabilizers. These two activated stabilizers are the “defects” or “detection events” that form the syndrome. The decoding process consists of constructing a **syndrome graph** where:

- **Nodes** represent the activated stabilizers.
- **Edges** represent the possible physical errors (or chains of errors) that could have caused those stabilizers to flip.

- **Weights** on the edges are assigned based on the error probabilities. Lower probability errors are assigned higher weights.

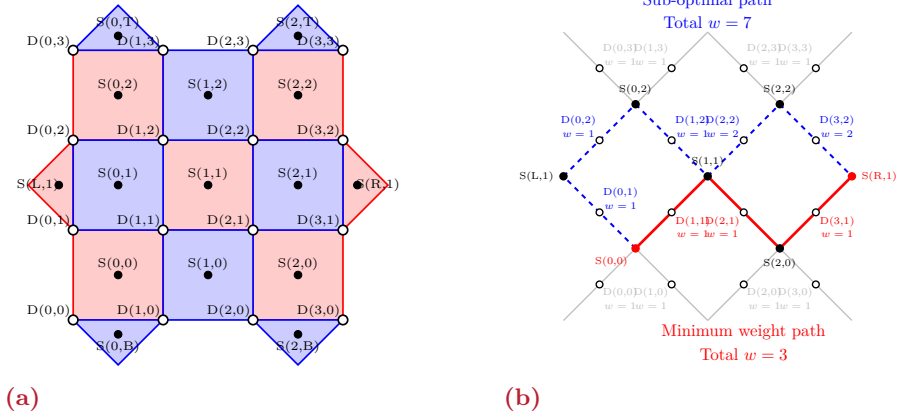


Figure 4.1. A visual representation of the Minimum Weight Perfect Matching problem under a code-capacity noise model. **(a)** The rotated surface code lattice introduced in Chapter 2. Data qubits are labeled as $D(x,y)$ and ancillas (stabilizers) as $S(x,y)$. **(b)** The full syndrome matching graph specifically for X-type errors, consisting of 7 nodes (the Z-stabilizer ancillas, shown as small black circles) and 16 edges (one for each physical data qubit—each possible X-type error in the code-capacity model—, represented by white circles indicating the weight (likeliness) of this error). Two possible error chains are highlighted, one of them being the one with minimum weight.

The MWPM algorithm efficiently searches this graph to find the pairings of defects that minimize the total weight. This is illustrated in Figure 4.1b. Once the optimal pairing is found, the decoder identifies the most probable error chain and applies the corresponding correction. This approach is highly effective and its implementation ([25]) serves as the benchmark against which almost all new decoders are measured.

4.1.1 Edmonds' Blossom algorithm

To understand how MWPM achieves this globally optimal pairing, it is necessary to look at its underlying mathematical engine: Edmonds' Blossom algorithm. A naive greedy approach—simply pairing the closest defects as soon as they are found—can easily get trapped in local minima, leaving isolated defects that force very high-weight connections later. Edmonds' algorithm systematically explores the graph to guarantee the

absolute minimum weight solution.

This exploration is intuitively visualized using expanding regions. The algorithm begins by simultaneously growing a “bubble” outwards from every active defect. These bubbles expand across the adjacent data qubits in the graph, to other non-active defects (or to the boundaries of the code, that are considered a virtual detector that can absorb the error chains that reach the boundary). When the boundaries of two expanding bubbles collide, the two defects are temporarily matched and the bubbles stop growing. The algorithm assumes that the path followed by the bubbles corresponds to the chain of physical errors that connected them.

As the bubbles continue to grow, they often encounter defects that are already matched to someone else, creating topological conflicts. The algorithm efficiently resolves these conflicts by re-evaluating its temporary matchings as new connections are discovered. Conflicting matches are resolved using *alternating paths*—routes that alternate between currently assumed errors (matched edges) and assumed healthy qubits (unmatched edges). If an alternating path is found connecting two free defects, the algorithm “flips” the entire path’s status, simultaneously correcting past suboptimal assumptions and increasing the total number of matched pairs.

Edmonds’ algorithm succeeds where other algorithms fail because of how it handles cycling paths: the temporary connections form a closed loop consisting of an odd number of nodes. Odd-length cycles can’t be paired internally (the flip will not fix the alternating chain). The Blossom algorithm overcomes this by temporarily collapsing the entire odd-cycle into a single “super-node” or *blossom*. This new blossom then resumes expanding as a single entity to find an external pair. Once all defects in the graph are paired, the algorithm unfolds the blossoms back into their original nodes and resolves the internal pairings. This continuous process of growing bubbles, shrinking odd-cycles, and unfolding them guarantees finding the most probable physical error chain.

Let’s see an example. Defects A and B are matched ($A \Rightarrow B$) and then defect C ’s expanding bubble reaches B . The path between them is stored as a non-matched path ($C \rightarrow B$). Defect A is re-activated and its bubble begins growing again. If the bubble reaches an active unmatched defect D , then an alternating path forms ($D \rightarrow A \Rightarrow B \rightarrow C$). This path has two non-matched segments and one matched segment. The algorithm “flips” the entire path’s status ($D \Rightarrow A \rightarrow B \Rightarrow C$), correcting past suboptimal assumptions and increasing the total number of matched pairs.

The Challenge of MWPM in Color Codes

While MWPM is exceptionally well-suited for Surface Codes, it cannot be directly applied to 2D Color Codes. The reason lies in the fundamental topology of the code.

In a surface code, a physical error creates a 1-dimensional string connecting two defects. However, in a color code, a single physical Pauli error activates **three** adjacent stabilizers. Consequently, an error chain in a color code is not a simple path between two points, but a branching, 2-dimensional structure that connects multiple groups of three defects. The result is an hypergraph, where an edge connects more than two nodes.

In the following sections we will study different alternatives to decode color codes, some of which heavily rely on MWPM.

4.2 Union-find

While the Minimum Weight Perfect Matching (MWPM) algorithm sets the standard for decoding accuracy, its worst-case computational complexity ($O(n^3)$ or, at best, $O(n^2 \log n)$) does not scale well with the number of nodes in the syndrome graph. Decoders must operate fast enough to prevent a backlog of uncorrected errors during the circuit execution, which causes a bottleneck as fault-tolerant architectures scale to thousands of physical qubits. To address this speed bottleneck, Delfosse and Nickerson introduced the Union-find decoder in 2018 [26], achieving an almost-linear worst-case complexity while maintaining a decoding threshold highly comparable to MWPM.

The primary innovation of the Union-find decoder is the replacement of the global path optimization of Edmonds' Blossom algorithm with a greedy solution. As mentioned earlier, a naive greedy approach would easily fall into local minima. Union-find circumvents this by proposing a greedy cluster-growth strategy instead. Similar to MWPM, bubbles grow and merge upon collision, but rather than evaluating the optimal pairing when two bubbles collide, the algorithm simply continues growing them as long as they contain an odd number of defects. Once every cluster contains an even number of defects (meaning all defects are safely

The overall worst-case complexity of Union-find is $O(n\alpha(n))$, where n is the number of physical qubits and $\alpha(n)$ is the inverse Ackermann function—a function that grows so slowly that it practically evaluates to less than 3.

encapsulated), a secondary phase acts strictly within each cluster to definitively pair the defects. The algorithm thus operates in two strictly sequential phases, as seen in Figure 4.2:

The **Disjoint-Set** data structure tracks elements partitioned into disjoint subsets, supporting two efficient operations: **Find** identifies the root of a subset (the unique element of the subset that identifies this subset), and **Union** merges two subsets. Checking or merging clusters is executed in practically constant time, bringing the computational cost of the algorithm to near-linear with respect to the number of nodes.

At an intersection, if both (or neither) of the child branches bring an unpaired defect, the error strings are combined and the upwards node is left unmarked. If only one of the two branches brings an unpaired defect, then the upwards node is marked as an unpaired defect so the pruning can continue upwards.

1. **Cluster Growth:** Initially, every individual defect is placed in its own separate cluster. Using the standard *Disjoint-Set* (or Union-find) data structure, the algorithm tracks these disjoint clusters. A cluster is classified as *valid* if it contains an even number of defects, or if it touches a boundary. If a cluster is valid, its growth pauses. Conversely, any *invalid* (odd-parity) cluster grows uniformly by absorbing its neighboring edges. When two growing clusters collide, the algorithm executes a **union** operation to merge them into a single, larger cluster. This iterative growth and merging process continues until every cluster in the graph becomes valid. As opposed to MWPM's blossom contraction, the Disjoint-Set data structure allows an easier handling of a cluster colliding with itself (creating an internal loop) thanks to the *find* operator.
2. **Path Resolution:** Once all clusters are validated, the algorithm must decide which specific edges inside each cluster correspond to the physical errors. For each valid cluster, the Union-Find process has already defined a tree (i.e. a graph without loops). This allows applying a highly efficient *peeling algorithm*: starting from the leaf nodes of the tree and working toward the root, the algorithm greedily assigns error correction strings to pair all internal defects. Each leaf node is evaluated and then pruned. If the node is a defect, the pruned edge is added to an error correction string and the next upwards node is marked as an unpaired defect to continue the pruning from it. When two branches intersect, the pruning on that path pauses until both branches reach the intersection point.

The algorithm described so far does not account for different error probabilities. Because of that, it can lead to lower accuracy than MWPM. *Weighted* variants of Union-find incorporate the DEM edge weights directly into the growth phase, causing clusters to grow faster through edges with higher probabilities (lower weights). This bridges the accuracy gap with MWPM while retaining fast execution speeds.

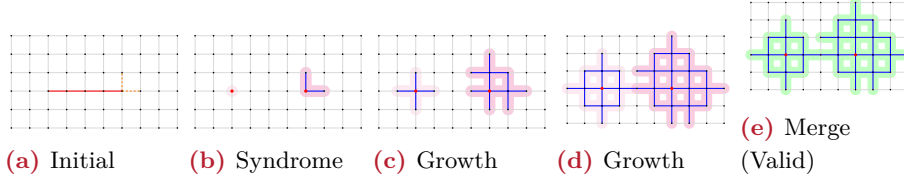


Figure 4.2. Visual representation of the Union-find decoder cluster-growth strategy: edges are data qubits and vertices are the detectors connected to these data qubits. **(a)** An initial error chain (red) occurs on the lattice, alongside two known erased data qubits (dashed orange), meaning there’s prior knowledge of these two data qubits being affected by loss or leakage. **(b)** The decoder identifies the syndrome defects (red dots) and defines the two odd-parity (invalid) clusters (light and dark magenta). The erased qubits are incorporated to the right cluster. **(c)** The algorithm grows the clusters uniformly. Because of the initial erasures, the right cluster expands faster. **(d)** In the second growth step, the boundaries of both clusters collide at a shared node. **(e)** This collision immediately triggers a **union** operation. The newly merged cluster now contains an even number of defects, changing its overall status to *valid* (green) and halting the growth phase.

Application to Color Codes

Like MWPM, the standard Union-find decoder is fundamentally a graph-based algorithm designed to pair defects connected by 1-dimensional strings. Consequently, applying Union-find directly to the branching error chains (string-nets) of 2D Color Codes presents the exact same topological challenge outlined in Section 4.1. To decode color codes efficiently, Union-find is typically integrated as the fast computational engine within reduction frameworks, such as Projection or Restriction decoders, which map the complex hypergraph onto standard surface code sub-graphs.

Note that the Union-Find algorithm has mechanisms to deal with erasure (known loss or leakage of a specific qubit). In the initial state, the edges corresponding to these qubits are accounted for when determining the initial clusters. In the simple code-capacity model that we will follow in this work, the initial erasure will always be empty.

4.3 BP+OSD

While Minimum Weight Perfect Matching and Union-find are highly effective, they are fundamentally graph-based algorithms designed for 1-dimensional errors (strings). As discussed, directly applying them to the branching error chains (string-nets) of 2D Color Codes is not naturally possible. Belief Propagation with Ordered Statistic Decoding (BP+OSD)

In classical and quantum coding theory, a **Tanner graph** is a bipartite graph used to represent the parity-check matrix of a code. It consists of two sets of vertices: *variable nodes* (representing the physical data qubits) and *check nodes* (representing the stabilizers). An edge connects a variable node to a check node if that qubit is involved in that stabilizer measurement.

Unlike MWPM or Union-find, which only use *hard information* (binary 0 or 1 from the syndrome), Belief Propagation utilizes *soft information*, continuously updating a continuous probability for each qubit. Mathematically, this relies on the Sum-Product Algorithm: a check node deduces the error probability of a target qubit by using Gallager’s formula [27] to compute the likelihood that all its *other* connected qubits possess a collective parity that matches the measured syndrome. Note that to calculate the error probability of a certain variable qubit, the algorithm only uses extrinsic information (the information from all the other variable qubits and the measured syndrome).

represents a complete paradigm shift: rather than trying to solve a geometrical routing problem on a lattice, BP+OSD treats decoding as a purely algebraic problem based on the parity-check matrix of the code. This makes it natively compatible with any quantum code, including color codes, without requiring any complex geometrical mappings or projections.

The BP+OSD decoder consists of two algorithms working sequentially. First, Belief Propagation leverages the Tanner graph, iteratively evaluating the syndrome to estimate the error probabilities of the individual qubits. Then, if this process fails to converge to a valid configuration, Ordered Statistic Decoding serves as a post-processing step, using the previously calculated probabilities to enforce a valid mathematical solution.

Belief Propagation (BP)

Belief Propagation is a classical message-passing algorithm proposed by Pearl in 1988 [28]. It operates on the *Tanner graph* of the code. BP works by having nodes “talk” to each other iteratively.

Initially, the *DEM* information is used to assign error probabilities to the variable nodes (data qubits), which then send this probability to the check nodes.

The check nodes process these newly received probabilities to enforce the code’s parity rules. To update a particular variable node, a check node looks at its own syndrome outcome and the estimated error probabilities of all the other variable nodes attached to it. From this collective information, it deduces the likelihood that the target variable node has an error, and sends this newly refined probability back to it.

The variable nodes gather these messages from all the stabilizers they belong to, update their own internal probability of holding an error, and send a message back to the check nodes, informing them of their evaluation of their own error probability.

This continuous back-and-forth exchange, represented in Figure 4.3, iteratively refines the global distribution of errors. This cycle continues until the variable nodes’ probabilities converge to a clear, definitive error

pattern that perfectly satisfies the observed syndrome.

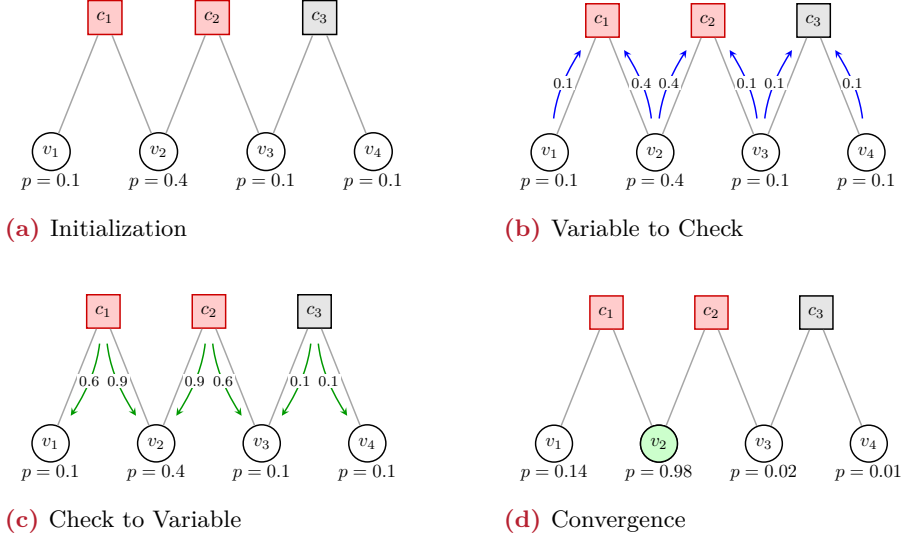


Figure 4.3. Visual representation of the Belief Propagation (BP) algorithm numerical updates on a simple Tanner graph. **(a)** Variables (circles) are initialized with prior probabilities p based on the *DEM*. Checks (squares) observe the syndrome; active syndromes are marked in red, while inactive ones are gray. **(b)** Variable-to-Check step: Variables broadcast their current probability to connected checks (blue arrows). **(c)** Check-to-Variable step: Checks enforce parity rules. For example, the active check c_1 requires odd parity; knowing v_1 has only a 10% chance of error, it deduces that v_2 must have a 90% chance of being the error, sending this refined extrinsic probability back (green arrows). Conversely, the inactive check c_3 requires even parity; assuming v_4 is healthy (10%), it deduces v_3 must also be healthy (10%). **(d)** Convergence: Variables aggregate the incoming evidence using Bayes' rule. Node v_2 receives strong confirmation from both active checks, spiking its probability to 0.98. Node v_3 is heavily suppressed to 0.02 because the inactive check c_3 vouches for its health. The algorithm flawlessly converges to the true physical error.

If the Tanner graph has a tree-like structure, BP is mathematically guaranteed to find the exact optimal solution. However, in topological codes like the Color Code, adjacent stabilizers share multiple data qubits. This translates into a Tanner graph with short, closed loops. In a graph with many short loops, the messages passed by the nodes start echoing in circles. A node might receive a high probability of error simply because its

own previous message traveled through a loop and came back amplified. This phenomenon creates *trapping sets*, where the probabilities wildly oscillate and the BP algorithm fails to converge to a valid physical error chain, rendering it unusable on its own for surface or color codes.

Ordered Statistic Decoding (OSD)

The algebraic nature of OSD involves Gaussian elimination over the parity-check matrix. In simple terms, the algorithm locks in the assumed state of the qubits that BP is most certain about. By substituting these “reliable” qubits into the system of equations that defines the parities as a function of the qubit errors, it step-by-step isolates the remaining uncertain qubits to logically force a valid error pattern. However, chaining these mathematical deductions across the entire code has a worst-case computational complexity of $O(N^3)$. While this makes BP+OSD remarkably accurate for color codes, it also makes it significantly slower than Union-find for large distances, presenting a classical trade-off between speed and decoding threshold.

To overcome BP’s failure to converge, Roffe introduced Ordered Statistic Decoding (OSD) as a post-processing step [29]. When BP gets trapped and fails to find a valid solution, it doesn’t mean the algorithm was useless. Even if the probabilities oscillated, the final “soft” probabilities calculated by BP still contain incredibly valuable information about which qubits are most likely to be corrupted.

OSD steps in by taking these final, unconverged probabilities from BP and sorting all the data qubits from the most likely to have an error to the least likely. It then selects a subset of the most strongly suspected failing qubits. Using standard linear algebra—specifically, Gaussian elimination over the code’s parity-check matrix—OSD forces a valid mathematical solution that aligns with the syndrome.

By combining the probabilistic insight of Belief Propagation with the brute-force algebraic resolution of Ordered Statistic Decoding, BP+OSD mitigates the hypergraph problem of Color Codes.

Algorithmic Enhancements: OSD-Order and Min-Sum

While the baseline BP+OSD algorithm provides a strong foundation, decoding graphs like those of the 2D Color Codes, with cycles and where multiple physical errors can cause the same syndrome (degenerated) requires further refinements.

Higher-Order OSD Search. As discussed, BP often fails to converge on color codes. When OSD takes over, it ranks the qubits from most to least reliable. A naive approach would simply lock the state of the reliable qubits and algebraically solve for the remaining ones. However, this assumes BP’s ranking is perfect. To maximize accuracy, modern implementations introduce an *OSD order* (k). Instead of solving the system only once, the algorithm isolates the k least reliable qubits in the

information set. It then systematically generates multiple alternative hypotheses by flipping the assumed states of these qubits. For each hypothesis, it performs the Gaussian elimination to find a valid solution. Finally, the decoder compares all the valid solutions generated and selects the one with the lowest weight of physical error (higher chances of being cause of the syndrome).

The Min-Sum Approximation. To address BP’s tendency to get trapped in infinite feedback loops caused by the short cycles, there’s a simplification known as the **Min-Sum Algorithm**. In the exact theoretical version of BP (the Sum-Product Algorithm), probabilities are precisely calculated using complex functions. When these exact probabilities propagate around closed loops, they amplify themselves, causing the decoder to believe in incorrect predictions.

Min-Sum simplifies this by having each check node ignore the precise probabilities of all its physical qubits except the least reliable one, which mostly defines the overall confidence of the prediction. It simply takes this minimum value and passes it along. Implicitly, this is over-estimating the confidence, as it ignores the uncertainty of all qubits except the most uncertain. Because of that, this is combined with a *scaling factor* to artificially lower the certainty.

4.4 Projection Decoder

As detailed in previous sections, the primary obstacle in decoding topological color codes is the branching nature of their error chains (string-nets), which creates a hypergraph incompatible with standard graph algorithms like Minimum Weight Perfect Matching (MWPM) or Union-Find. The Projection decoder, introduced by Nicolas Delfosse in 2014 [30], circumvents this limitation by mathematically reducing the color code into multiple independent surface codes.

The method relies on the 3-colorability of the color code lattice. Because adjacent faces always have different colors (e.g., red, green, and blue), the removal of all faces of a single color produces a bipartite graph composed of the remaining two colors. This resulting graph is equivalent to a standard surface code: each edge connects exactly two nodes. The

By default, *Order-0* OSD simply performs Gaussian elimination and accepts the single resulting mathematical solution. This is highly naive, as it entirely trusts the reliability ordering provided by BP. If BP misjudged even a single critical qubit, Order-0 will confidently output an incorrect high-weight error chain, significantly lowering the decoding threshold. The alternative higher-OSD search improves OSD at the cost of increased complexity up to $O(N^o)$ for an order o .

For simplicity, in this section we often refer to MWPM as the decoder for the projected syndrome. However, any surface-code decoder, such as Union-Find, can be used.

By projecting the lattice to only two colors of faces, any branch in the string-net is effectively removed. What remains is a set of 1-dimensional error strings that connect the defects on the bipartite graph, matching the structure of the surface code.

decoder splits the global syndrome into three separate syndromes, each corresponding to one of the possible pairs of colors (Red-Green, Green-Blue, and Blue-Red).

Once the syndrome is projected onto these three surface codes, they are decoded independently using algorithms like MWPM, that compute a correction for each of the three projections. The final step merges the three independent corrections to form a valid error recovery operation for the original color code. By mathematically combining the 1-dimensional string corrections from the three projected graphs, the algorithm reconstructs the original branching structure of the error, mapping the independent surface code paths back to the specific physical qubits of the 2D color code.

This divide-and-conquer strategy provides a computationally efficient path to decode color codes. However, because the three projections are decoded independently, the algorithm loses the global correlation information of the noise, which increases the logical error rate compared to global decoders.

Given that the intermediate MWPM logic has already been described in Section 4.1, we will now detail the projection and merging processes.

4.4.1 The Projection Step

The core mechanism of the projection decoder relies on transforming the 2D color code lattice into distinct surface code lattices. To create a projected surface code, the algorithm ignores one of the three colors. For instance, to create the Red-Green (RG) projection, all Blue faces are removed from the lattice. The remaining Red and Green faces result in a bipartite syndrome graph. In this mapped graph:

- The original Red and Green faces become the vertices (the parity-checks) of the new surface code.
- The physical data qubits, originally connecting three vertices, become the edges connecting a Red vertex to a Green vertex. The graph nodes can therefore be split into two separate groups (one for each color) and the edges will always connect one element of each group.

When dealing with realistic noise (beyond code-capacity), assigning weights (failure probabilities) to the edges of the projected surface codes becomes relevant. Under circuit-level noise, multiple distinct physical fault mechanisms (e.g., a single data-qubit error vs a failing gate propagating errors to several qubits) might produce the same projected syndrome. A simple option is to assign the weight based on the probability of the independent single-qubit error, effectively blinding the MWPM decoder to the correlated noise. A more robust approach consist in calculating the edge weight by summing the probabilities of all the possible errors that could trigger that specific projected edge.

This transformation is applied to create three projected lattices (e.g., RG , GB , and BR), which is illustrated in Figure 4.4. When an error occurs on a physical qubit in the original color code, it typically triggers three stabilizers (one of each color). However, in the RG projection, the Blue stabilizer is ignored. The error now only appears to trigger the Red and Green stabilizers, acting exactly as a standard 1-dimensional string with two endpoints. This matches the syndrome of a surface code, and therefore can be directly processed by the MWPM decoder.

Because each pair of adjacent detectors is separated by exactly two physical qubits in the original lattice, any edge in the projected bipartite graph represents two parallel qubits. Before running the MWPM decoder over this projection, these two parallel qubits are abstracted into a single logical edge. Consequently, the output of the MWPM algorithm is a set of 1-dimensional edges, leaving an inherent ambiguity regarding which specific physical qubits were originally affected.

4.4.2 The Merge Step

Once the MWPM instances process their respective syndromes, they output independent corrections: C_{RG} , C_{GB} , and C_{BR} . These corrections do not propose specific physical qubits, but rather sets of edges on their respective bipartite graphs. As detailed during the projection step, each of these edges maps to two parallel edges (physical qubits) in the original graph. This leaves a 1-to-2 ambiguity: the decoder knows an error occurred along a specific edge, but a single projection alone cannot determine which of its two corresponding physical qubits is the faulty one.

To resolve this ambiguity and merge the three corrections, the Projection decoder utilizes the topological properties of the dual graph of the lattice (a transformed graph where physical qubits are represented as faces, and detectors as vertices), as illustrated in Figure 4.5.

The merging process, illustrated in Figure 4.6, overlays the three independent sets of selected edges onto the global dual graph. Since each projected graph is a subgraph of the full dual graph, the edges selected by C_{RG} , C_{GB} , and C_{BR} can be directly placed on their corresponding geometric locations in the global graph, effectively computing the geo-

In his 2014 paper, Delfosse formulates this proof using the dual graph (G^*) of the color code lattice. In the dual graph, detectors are vertices and physical qubits are triangular faces. The MWPM corrections correspond to the edges of these triangles. Mathematically, the union of the edges is treated as a homological chain sum (modulo 2), $C = C_{RG} \oplus C_{GB} \oplus C_{BR}$. This sum forms a set of closed loops (homological cycles) in the dual graph, and the “lifting” step finds the set of triangular faces (qubits) filling the interior of these cycles.

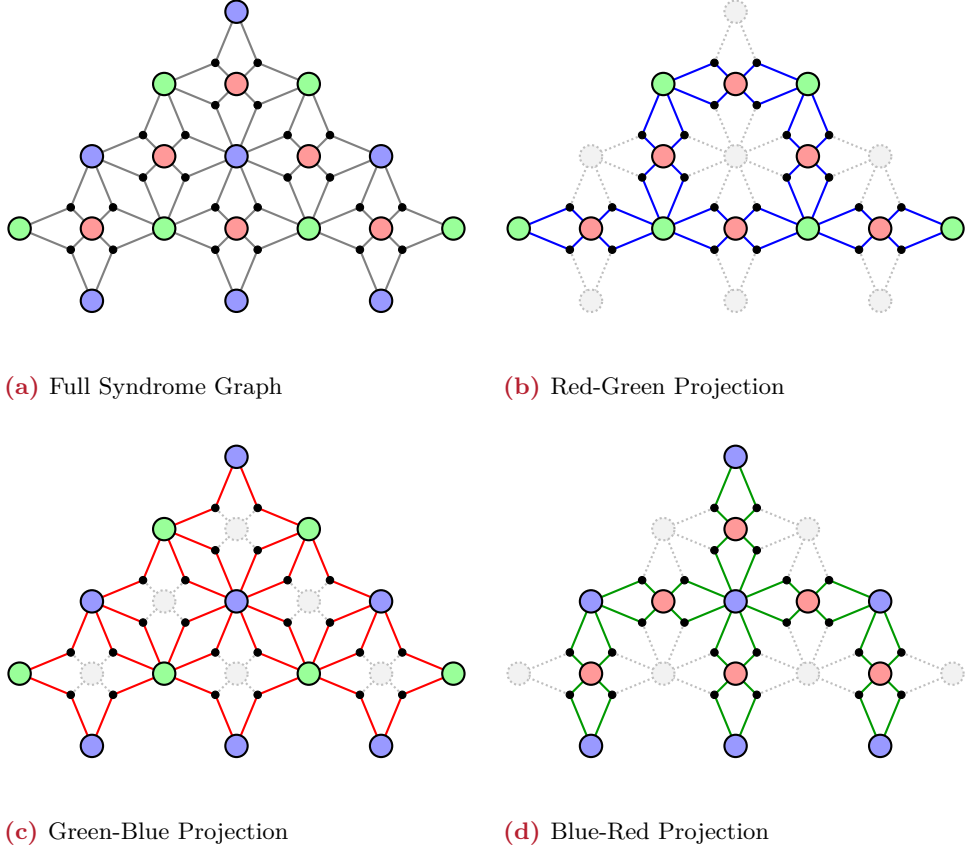


Figure 4.4. The syndrome hypergraph for a local patch of a color code (a) and its decomposition into three independent bipartite surface code graphs (b, c, d) using the projection method. Black circles correspond to possible errors in data qubits, and color circles correspond to the detectors for these errors. The deleted nodes and hyperedges in each projection are shown in faint dotted gray to illustrate the mapping. Before running MWPM on this graph, the parallel edges between nodes are combined into a single, logical edge.

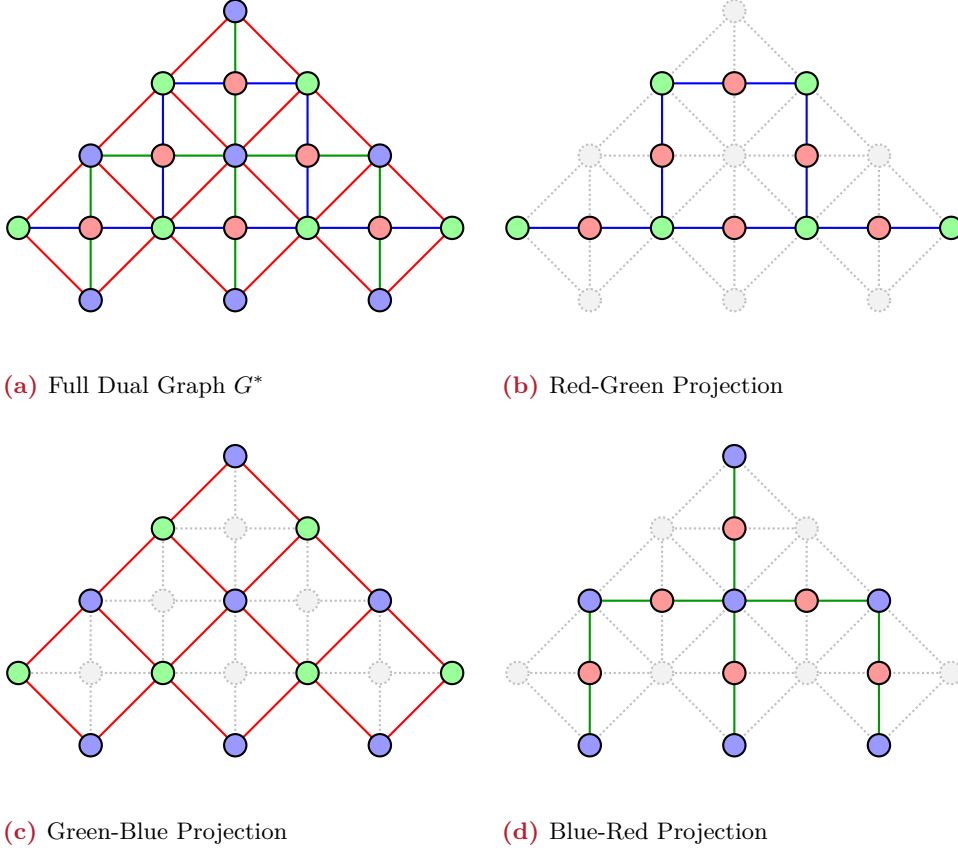


Figure 4.5. The dual graph G^* for the local color code patch displayed in Figure 4.4 (a) and its three projected bipartite graphs (b, c, d). In this representation, the detectors are vertices and physical qubits are triangular faces. Removing one color of vertices leaves exactly one type of edge, creating independent surface codes. Note how the two parallel edges in the syndrome graph become a single edge in this representation.

metric union of the graphs to combine them. Delfosse proved that this combined set of 1-dimensional edges forms a continuous, closed boundary (a set of polygons) drawn across the dual graph.

With these polygons defined, the algorithm performs a final *lifting* step. Because the physical qubits are represented as triangular faces in this geometry, the lifting operation identifies the set of triangles that geometrically fill the interior of polygons. This mapping guarantees that the final selected set of qubits will satisfy the parity checks of all three colors, resolving the 1-to-2 ambiguity.

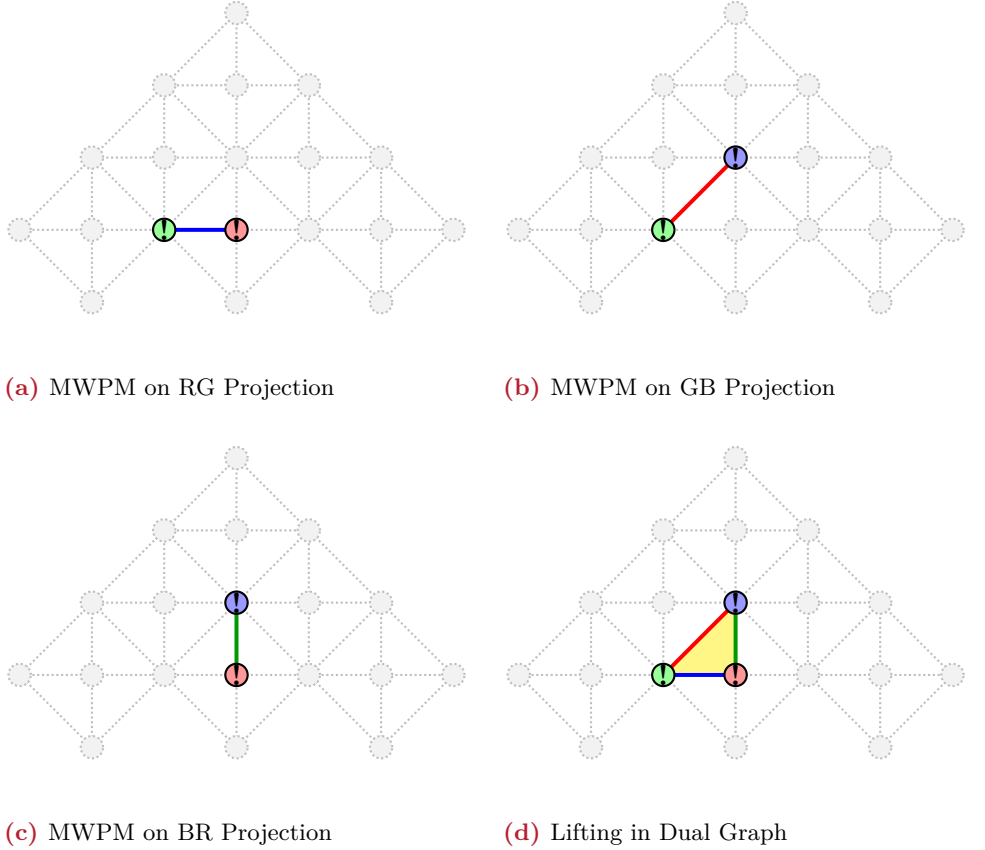


Figure 4.6. The merging process on the dual graph displayed in Figure 4.5: (a, b, c) Each MWPM instance returns a 1-dimensional edge corresponding to its projection. (d) The union of these edges forms a continuous closed boundary (a triangle) in G^* . The Lifting operation finds the physical qubit (yellow face) that exactly fills this boundary.

4.5 The Restriction Decoder

As previously explored, the Projection Decoder resolves the hypergraph challenge of the color code by transforming the Color Code into three separate surface codes. While this divide-and-conquer strategy succeeds in enabling MWPM decoding, the structural ambiguity when the results are combined and analyzed at the global level, limits its accuracy. To address this, Kubica and Delfosse introduced the Restriction Decoder in 2023 [31]. This algorithm improves upon the Projection Decoder in two fundamental ways: it reduces the problem to only **two** restricted lattices instead of three (which had already been presented by Kubica in his PhD thesis [32] as a simplified Projection Decoder), and it replaces the global geometric lifting (finding closed cycles in the dual graph) with an approach based on logical restrictions (constraints) and satisfaction requirements (finding a solution that meets these constraints). This allows the algorithm to preserve the topological structure of the code during the restriction process, improving the decoding threshold.

4.5.1 The Restriction Step

Similar to the Projection Decoder, the algorithm begins by dropping one color of faces from the lattice to form a bipartite syndrome graph. However, the crucial difference lies in the number of required projections.

When creating a restricted lattice, for instance the Red-Green ($\mathcal{L}_{RG}$) restricted lattice, all Blue faces are removed. In this new graph, the original Red and Green faces become the vertices, and the edges between two faces in the lattice become the edges connecting a Red vertex to a Green vertex (which are equivalent to the two merged qubits in the projection graph).

This transformation is applied to create just **two** restricted lattices (e.g., $\mathcal{L}_{RG}$ and $\mathcal{L}_{RB}$), whereas the Projection Decoder required all three. At the decoding phase, the full syndrome is mapped onto these two restricted graphs. The syndromes s_{RG} and s_{RB} are then independently decoded using MWPM to find matchings p_{RG} and p_{RB} .

While the Projection Decoder requires three distinct bipartite graphs, the Restriction Decoder achieves full decoding with only two. This is because fixing the boundaries on two of the three colors is mathematically sufficient to uniquely constrain the third color, given the properties of the color code lattice.

While the formal description of the restriction graph differs from the one of the projection graph (in the projection, vertices are physical qubits of the lattice and in the restriction vertices are edges in the lattice), the fact that the two physical qubits are merged in the projection graph so MWPM can process them, makes the final graph equivalent: two parallel physical qubits between two faces are equivalent to the edge dividing the two faces.

4.5.2 The Local Lifting Step

While p_{RG} and p_{RB} neutralize the syndromes on their respective restricted lattices, they are independent solutions that must be reconciled. A valid physical correction must simultaneously satisfy the syndrome conditions on all three colors, meaning the two solutions must perfectly agree on the selected physical qubits.

Unlike the Projection Decoder, which resolves conflicts globally using a geometric dual spanning tree (i.e. examines the complete graph to find closed loops to fill), the Restriction Decoder guarantees a physically consistent global correction by performing a strictly **local** analysis around the active detectors.

For every single Red vertex v in the lattice that belongs to an active edge, the algorithm examines the local neighborhood:

- It looks at the specific edges from p_{RG} and p_{RB} that are incident to v .
- It selects a local subset of physical qubits, τ_v , strictly from the ones touching v , such that their boundaries perfectly match the active edges touching this red vertex.

Because the maximum number of physical qubits touching a single vertex is small (e.g., 4 or 8 on the 4.8.8 lattice), this local constraint-satisfaction can be trivially solved independently for each Red vertex.

To achieve this local reconciliation, the algorithm, illustrated in Figure 4.7, internally constructs a purely algebraic **Lifting Graph** (see 6). This bipartite graph operates in a conceptually inverted way compared to standard syndrome graphs:

- **Nodes:** The nodes of the Lifting Graph are the specific edges activated by the restricted lattices $e_{RG} \in \mathcal{L}_{RG}$ and $e_{RB} \in \mathcal{L}_{RB}$.
- **Edges:** Each true physical qubit q connects the node representing its corresponding edge in $\mathcal{L}_{RG}$ to the node representing its corresponding edge in $\mathcal{L}_{RB}$.

The active syndrome (the “defects”) fed into this Lifting Graph is the combined set of edges selected by the previous restricted matchings:

$$s_{lift} = p_{RG} \oplus p_{RB}.$$

To select the physical qubits around a red vertex, two conditions must be met: they must touch the red vertex and their restricted boundaries must match the active edges. In this context, restricted boundaries means the boundary edges for the set of qubits that are within the set of all active edges.

As we will see when we implement this decoder, this structure allows solving the lifting by solving a final MWPM instance over this bipartite Lifting Graph. This process identifies the minimal set of physical qubits that connect the active nodes. Geometrically, this is perfectly equivalent to examining every single vertex and finding a set of physical qubits whose boundaries exactly match the active edges predicted around that specific vertex.

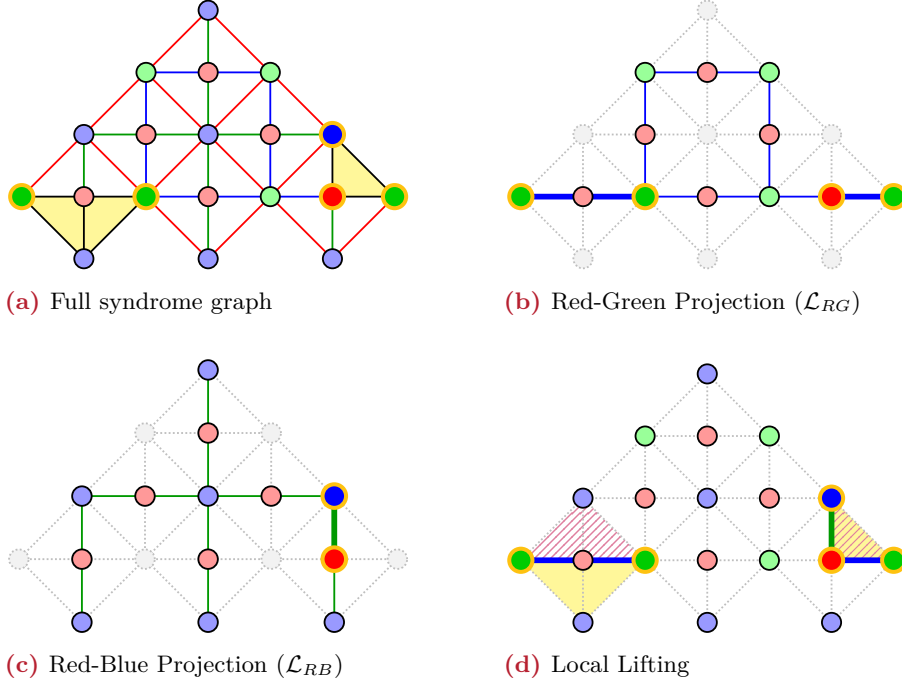


Figure 4.7. The Restriction Decoder on the dual graph G^* . (a) Three physical errors occur (yellow triangular faces). On the right, an isolated error activates an R , G , and B detector. On the left, a pair of adjacent errors activates only two Green detectors (orange outlines). (b) The syndrome is projected onto $\mathcal{L}_{RG}$, retaining only R-G edges (blue lines). MWPM finds paths between active detectors (blue thick lines). (c) Projected onto $\mathcal{L}_{RB}$ (R-B edges), the left error cluster has no active detectors, while the isolated error yields a path (red thick line). (d) The local lifting step superimposes the edges from both restricted lattices to identify the physical qubits (hatched patterns). For the isolated error, the boundaries uniquely identify the original faulty qubit. For the left cluster, the algorithm selects an alternative, topologically equivalent pair of qubits that satisfy the local constraints and similarly corrects the error in the observable.

Neutralizing the syndrome means finding a set of qubits that, if flipped, deactivate all the active detectors. It does not necessarily mean finding the original set of errors that activated these detectors.

The final global correction is simply the union of all these subsets of qubits (τ_v). The output correction is mathematically guaranteed to neutralize the full original syndrome. If there are multiple sets of physical qubits that locally resolve the constraints (i.e. that touch the red vertex and contain all and only the active edges incident to it), their common boundary will inevitably form a closed loop (a stabilizer) around the vertex. Therefore, both sets will similarly correct the logical observable.

4.6 Möbius Decoder

The Möbius decoder was proposed by Sahay and Brown in 2022 [33] as a topological approach to decode the triangular color code. This decoder solves the fundamental limitations of projection-based algorithms at the boundaries of the lattice, by unifying the restricted graphs into a single global structure rather than treating them in isolation.

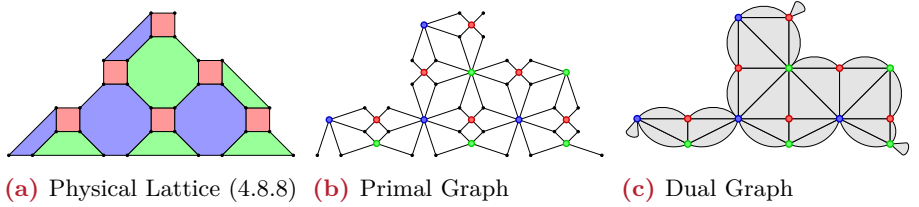


Figure 4.8. Color code representations for a $d = 7$ 4.8.8 lattice. In (a), physical qubits are nodes and detectors are faces (octagons/squares). In (b), detectors become nodes and physical qubits are represented as the hyperedges connecting them. Finally, in (c), the detectors are nodes and the physical qubits become triangular faces. As opposed to simplified representations of local patches (Figure 4.5), when representing the complete code, irregularities show in the border (qubits connected to less than 3 detectors).

As described in previous sections, algorithms like the Projection Decoder divide the color code into three independent restricted graphs ($\mathcal{L}_{RG}$, $\mathcal{L}_{RB}$, and $\mathcal{L}_{GB}$). While this successfully reduces hyperedges to simple edges compatible with Minimum Weight Perfect Matching (MWPM), running it independently on each subgraph causes a loss of information near the boundaries. Since boundaries act as virtual nodes in MWPM to absorb errors that exit the lattice, the algorithm might incorrectly pair the defect to the boundary instead. When the decoder attempts to merge these 1D

strings back into 2D physical qubits, these boundary inconsistencies often lead to logical failures. Figures 4.9 and 4.10 illustrate how Projection and Restriction decoders succeed in errors in the bulk of the Color Code but fail near the borders.

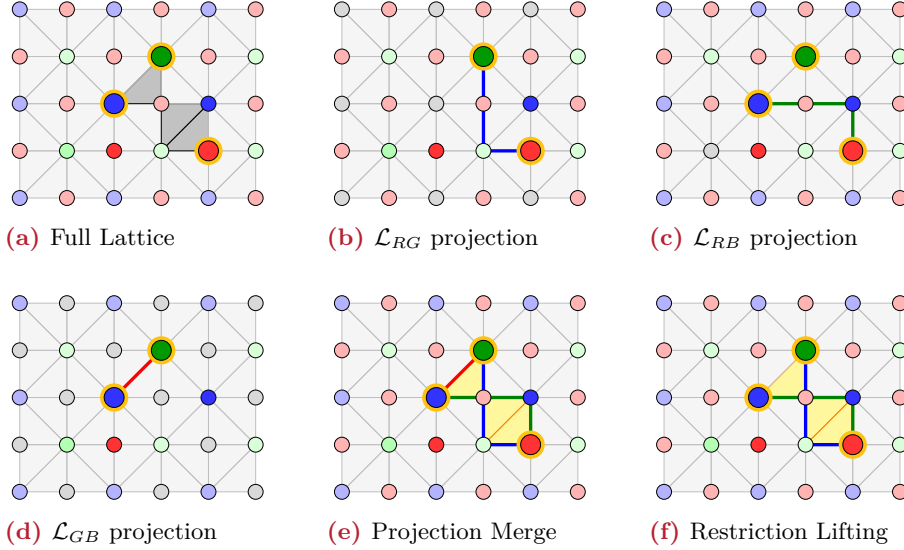


Figure 4.9. Successful decoding of an error pattern within the bulk of the lattice. (a) The disjoint 3-qubit error chain creates exactly three active detectors. (b, c, d) The MWPM algorithm independently pairs the relevant defects in each of the three projected surface codes using the shortest paths. (e) When merging the paths, they form a perfect polygon that exactly traces the exterior edges of the erroneous qubits, unambiguously identifying the failing data qubits. (f) The Restriction decoder achieves the same successful result by matching the combined restriction edges directly onto the correct physical correction.

The Möbius decoder resolves this limitation through a topological gluing process. Instead of executing MWPM on the three restricted graphs separately, the decoder physically joins them along their shared boundaries. This effectively eliminates the virtual boundaries, producing a unified space where the paths found by the MWPM algorithm can flow across different color projections to find a global optimum. This is illustrated in Figure 4.11.

While this unified space prevents boundary failures, the decoder still has to determine the physical corrections on the data qubits. In the projected

The name comes from the resulting topology: when joining these three projected lattices one after the other to form a loop, they are twisted so that at the final junction closing the circle, the top connects to the bottom. This twist produces a mathematical object with only one surface and one continuous boundary, equivalent to a Möbius strip.

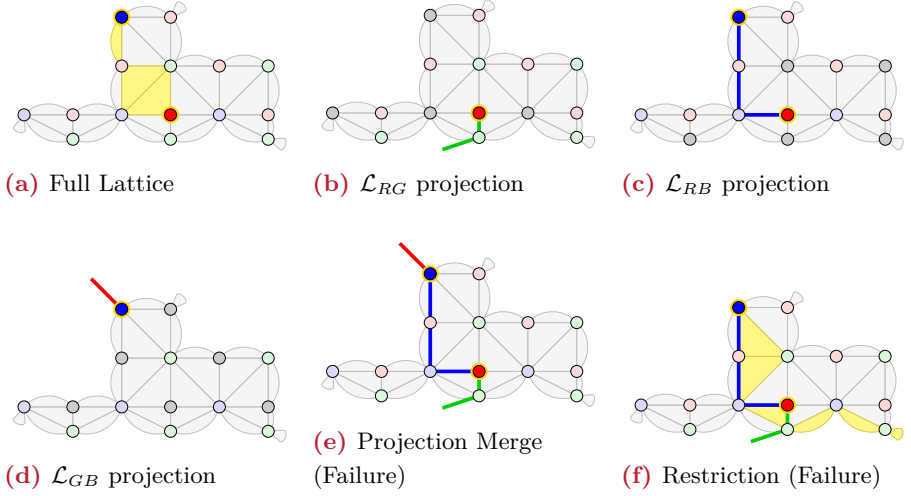


Figure 4.10. Projection and Restriction decoding failures at the boundary of a Color Code. (a) An error chain (yellow) similar to the one in Figure 4.9, but occurring adjacent to the lattice boundary, creates two active detectors. (b, c, d) In each projection, the Minimum Weight Perfect Matching (MWPM) algorithm finds a path (colored lines) that pairs the active defects, routing the path to the virtual boundary when that's the shortest path. (e) When merging the three independent paths, the resulting edges do not form closed polygons around any physical data qubits, leading to an incorrect logical correction. (f) The Restriction decoder executes a local lifting graph matching to resolve the incompatible projected paths. Forced to pair the inconsistent disjoint edges, it creates a long path along the boundary (yellow), resulting in a logical failure.

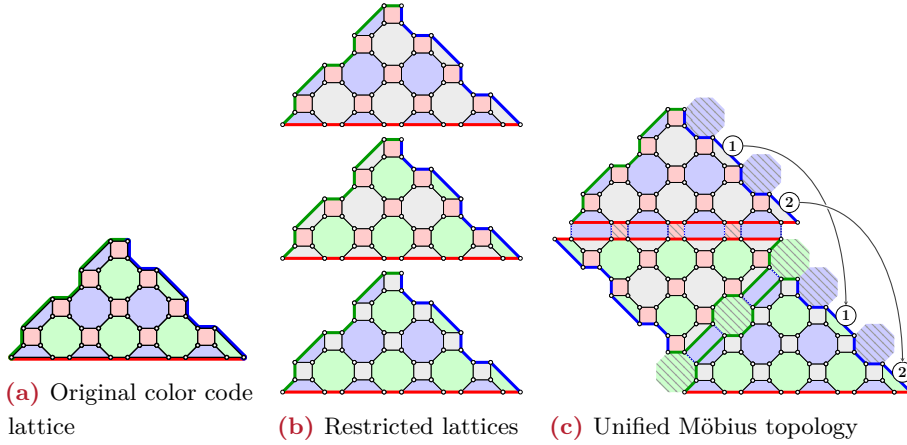


Figure 4.11. Möbius strip construction from 2D color code restrictions. (a) The original planar color code lattice. Outer boundaries are highlighted in their respective colors. (b) Independent restriction decoders generated by eliminating one color at a time (RB, RG, and GB patches). (c) Unified lattice showing the topological connections. The RB and RG patches are sewn along their red boundary, while RG and GB are sewn along their green boundary. Virtual boundary stabilizers (patterned) fill the resulting topological gaps. The twist required to connect the remaining blue boundaries of RB and GB forms a Möbius strip, matching the numbered boundary operators.

graphs that can be used by MWPM, every edge corresponds to a pair of physical qubits. Directly lifting these edges back to physical qubits creates an ambiguity that requires complex graph analysis. To solve this, the Möbius decoder uses a topological approach. The algorithm operates in three sequential steps: first, it calculates a trivial correction on the physical lattice; second, it glues the projections and runs MWPM on the unified lattice; and third, it maps the logical operator (e.g. X_L operator) to the unified lattice to evaluate if the sum of the MWPM and the trivial correction flips the logical qubit. The final correction is the trivial one followed by a possible logical operator to compensate for this flip.

4.6.1 Trivial Correction

The first step of the decoding process is to find any valid physical correction (C_0) that perfectly satisfies the observed syndrome on the original 2D lattice. This correction does not need to be the optimal one, and it is computed without using MWPM.

By scanning the lattice row by row, the algorithm can systematically flip physical qubits beneath any active detector. This action turns off the targeted detector but propagates the defect to the row below. By continuing this sweep across the lattice, all defects are pushed towards the boundaries, where they are absorbed. The total set of physical qubits flipped during this process constitutes C_0 . While this simple correction restores the code to the logical space (i.e. deactivates all the detectors), it likely introduces a logical error.

4.6.2 Unified Lattice and MWPM

To determine if the trivial correction C_0 introduced a logical error, the decoder uses the unified lattice, constructed by physically joining the three restricted graphs ($\mathcal{L}_{RG}$, $\mathcal{L}_{RB}$, and $\mathcal{L}_{GB}$) along their shared boundaries. For example, the red boundary of the $\mathcal{L}_{RG}$ graph is glued directly to the red boundary of the $\mathcal{L}_{RB}$ graph. When connecting the three patches, one of them must be twisted, resulting in a Möbius strip.

In this unified space without boundaries, the MWPM algorithm is executed to find the optimal path (E_{MWPM}) that pairs the active defects. Because MWPM operates on the global unified graph, the resulting 1D

A Homology Decoder (or Comparative Decoder) separates the decoding problem in two parts: finding a valid physical correction that satisfies the syndrome, and determining the impact of that correction to the logical qubit. By comparing a correction with the MWPM result, it deduces if the initial correction causes a flip in the logical state.

edges are mathematically consistent and avoid the failures that occur when projections are treated independently.

4.6.3 Homology Evaluation

The final step avoids the complex process of translating the abstract MWPM edges back into physical qubits. Instead, it evaluates the logical parity directly on the unified lattice.

To do this, the logical operator is mapped onto the unified graph. By combining the mapped trivial correction (C_0) and the optimal path (E_{MWPM}) using modulo 2 addition, the algorithm forms a set of closed loops on the unified lattice. Counting how many times these closed loops cross the mapped logical operator determines the parity. If the loops cross an odd number of times, it means the trivial correction C_0 applies the logical operation (i.e. causes a logical flip).

In a transversal code such as the one we are implementing, the logical observable Z_L is defined on all the data qubits. Because all physical qubits belong to the logical operator, every single abstract segment in the unified lattice is a part of the logical observable. Any segment selected by MWPM alters the logical operator exactly once and the abstract logical parity of the MWPM path is the total number of edges in E_{MWPM} modulo 2.

This allows the algorithm to compute the parity of C_0 (by counting the flipped qubits modulo 2) and the abstract parity of E_{MWPM} (by counting its edges modulo 2). If the parities differ, a logical flip is caused by the trivial correction. The final correction applied to the quantum computer is simply the trivial correction C_0 , followed by a global logical operator if the parities were different. This elegantly corrects the state without ever needing to map the abstract MWPM paths back to specific physical qubits.

The Dual Graph, G^* presented in prior sections is useful to understand the unified lattice. Since it's made of glued projections, its edges are not individual qubits, but pairs of parallel qubits. Therefore, an active edge implies a flip in one of the two qubits that map to it.

4.7 Concatenated MWPM

As we have seen, the core difficulty in decoding color codes is that a single physical error activates three stabilizers, creating a hypergraph problem. While the Projection Decoder tackles this by resolving three separated

projections and geometrically combining the solutions to disambiguate, the Concatenated MWPM decoder [34] takes a more graph-theoretic approach.

Unlike decoders that rely on geometric lifting (projection) or local algebraic analysis (restriction) to resolve ambiguities, this approach keeps the problem strictly within the domain of graph theory by chaining two bipartite matchings.

Instead of performing a single matching followed by a lifting, this decoder circumvents the hypergraph problem entirely by performing two sequential MWPM executions per color. The decoding process is divided into three parallel sub-routines (one for each color), which operate independently to find a valid correction. Once the three sub-routines finish, the decoder compares their results to choose the most optimal one.

At a high level, the algorithm follows three main steps:

1. **First-round MWPM:** A matching is performed on a restricted graph to identify the edges of the original lattice where the error likely occurred. These edges correspond to pairs of qubits.
2. **Second-round MWPM:** A second matching is performed on a new graph to resolve the ambiguity of the edges and identify the exact physical qubits.
3. **Final Selection:** The sub-routine is repeated for all colors, and the most probable physical correction is chosen based on its total weight.

Let us examine these steps in detail by looking at the Red sub-routine as an example.

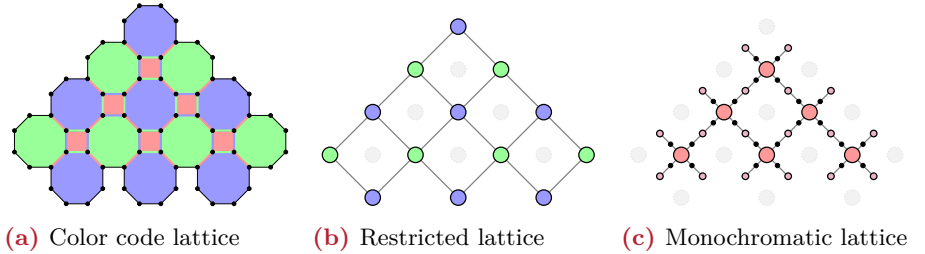


Figure 4.12. Graph structures used in the Concatenated MWPM decoder (Red sub-routine). (a) The a local patch of the primal color code lattice. (b) The Red-restricted lattice $\mathcal{L}_{\text{rest}}^{(r)}$ representing a bipartite dual graph where nodes are the Green and Blue faces, and edges are the primal Red edges. (c) The Red-only (monochromatic) lattice $\mathcal{L}_{\text{mono}}^{(r)}$ where nodes are the Red faces and the primal Red edges, and edges represent the physical data qubits.

4.7.1 Step 1: First-round MWPM on the Restricted Lattice

The Red sub-routine is designed to find a valid correction by ignoring the Red stabilizers of the syndrome. To do this, the decoder first constructs a bipartite graph called the *Red-restricted lattice*.

As illustrated in Figure 4.12, In this graph, the nodes are exclusively the Green and Blue stabilizers. The edges connecting these nodes correspond strictly to the *Red edges* of the primal color code lattice (which are equivalent to the merged two qubits shared by these faces).

The algorithm inputs the activated Green and Blue stabilizers from the syndrome as defects and runs a standard MWPM. The output is a set of primal Red edges that are suspected to contain the error (see Figure 4.13). However, since a Red edge effectively connects two physical qubits, the algorithm still does not know which of the two qubits failed.

The restricted lattice here is equivalent to the projected or restricted lattices on the Projection and Restriction decoders.

4.7.2 Step 2: Second-round MWPM on the Monochromatic Lattice

To resolve the physical qubit ambiguity left by the first step, the decoder builds a completely new graph called the *Red-only lattice* (or monochromatic lattice).

As illustrated in Figure 4.12, in this new graph, twice as much nodes are added: the Red stabilizers (which were completely ignored in the first step) and the Red edges of the primal lattice. The edges of this new graph are the actual physical qubits. Geometrically, this perfectly matches the color code structure, as every physical qubit connects one Red stabilizer and one Red edge.

The algorithm marks two types of defects in this graph: the activated Red stabilizers from the original syndrome, and the suspected Red edges returned by the first MWPM. Running a second MWPM on this graph directly yields a set of edges. Since the edges in this graph are the physical qubits themselves, the output is a concrete, unambiguous prediction of the physical errors (see Figure 4.13). By concatenating these two matchings, the Red sub-routine returns a valid set of physical errors that perfectly explains the syndrome.

4.7.3 Step 3: Repetition and Final Selection

The Red sub-routine alone guarantees a mathematically valid set of physical errors. However, to maximize the accuracy of the decoder, the algorithm repeats the exact same two-step process for the Green and Blue colors.

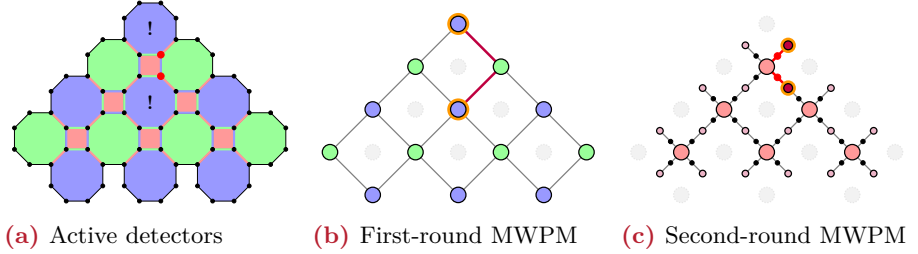


Figure 4.13. Decoding flow of the Concatenated MWPM Red sub-routine. **(a)** An error string of two physical qubits (red dots) triggers two Blue detectors (highlighted in glowing orange). **(b)** First-round MWPM: The active Blue detectors are perfectly matched in the Red-restricted lattice, successfully identifying the faulty Red edges (thick red lines). **(c)** Second-round MWPM: The previously identified Red edges are mapped as an active defect (red node with orange glow) in the Red-only lattice, along with any active Red detectors (none in this example). The final matching correctly identifies the exact physical qubits (thick red lines with a dot) that caused the syndrome.

As illustrated in Figure 4.13, the decoder ultimately produces three valid physical correction candidates (one from each color). To make the final decision, the algorithm evaluates the total weight of each predicted error chain. In a basic code-capacity noise model where all errors are equally likely, this is equivalent to simply counting the number of physical qubits in each prediction. However, for noise models where errors have different probabilities, the total weight is calculated based on these specific probabilities.

The decoder compares the three candidates and selects the one with the lowest total weight. This ensures that the algorithm always outputs the most probable physical error configuration to successfully correct the quantum state.

4.8 Correlated matching decoder

As detailed in previous sections, many of the established color code decoders—such as the Projection (Section 4.4) and Restriction (Section 4.5) decoders—rely on dividing the complex decoding problem into independent sub-lattices. While this strategy provides an efficient framework for executing standard graph matching algorithms, this strict separation discards the underlying physical correlations between the decomposed sub-graphs.

When a physical error occurs, it generally triggers syndrome defects across multiple colors simultaneously. Because projection-based decoders process each projected graph in isolation, they are vulnerable to error degeneracy. Specifically, on the 4.8.8 lattice, the algorithm struggles to distinguish between degenerate error configurations of weight $O(d/2)$, such as distinguishing between two failing qubits forming a diagonal in a square, red detector, and two adjacent qubits (connected by a horizontal or vertical edge) in the same detector. Both error configurations look the same in the corresponding projection. An example of the projection failure is illustrated in Figure 4.14.

Due to this ambiguity, the independent matchings often select diverging paths, which leads the decoder to overestimate the total weight of the true physical error, resulting in an incorrect correction and a logical failure [35].

To address this widespread limitation, the Correlated matching decoder [36] introduces a sequential re-weighting mechanism. Instead of decoding the sub-lattices entirely independently and in parallel, the algorithm establishes a strict dependency between the matching instances. The core idea is to use the output of the first matching to dynamically update the edge weights of the second matching.

To understand the mechanism, as illustrated in Figure 4.14, consider the decoding of a 4.8.8 lattice using two restricted graphs: $\mathcal{R}_b$ (where all blue checks are removed) and $\mathcal{R}_g$ (where all green checks are removed). The decoding proceeds as follows:

1. **First Matching and Node Marking:** The algorithm first executes a standard Minimum Weight Perfect Matching on the $\mathcal{R}_b$

The concept of decoding over a global manifold to avoid this loss of information was first introduced by the Möbius decoder (Section 4.6), which stitches the sub-lattices of a triangular patch into a Möbius strip. The Unified decoder [35], upon which the Correlated decoder is founded, is a subsequent development that adapts this exact solution to square-boundary patches. While highly accurate, the resulting combined matching graph in both approaches is larger and denser, significantly penalizing the decoding speed, as MWPM does not scale linearly as the number of qubits grows (although with some optimizations, it can scale near-linearly).

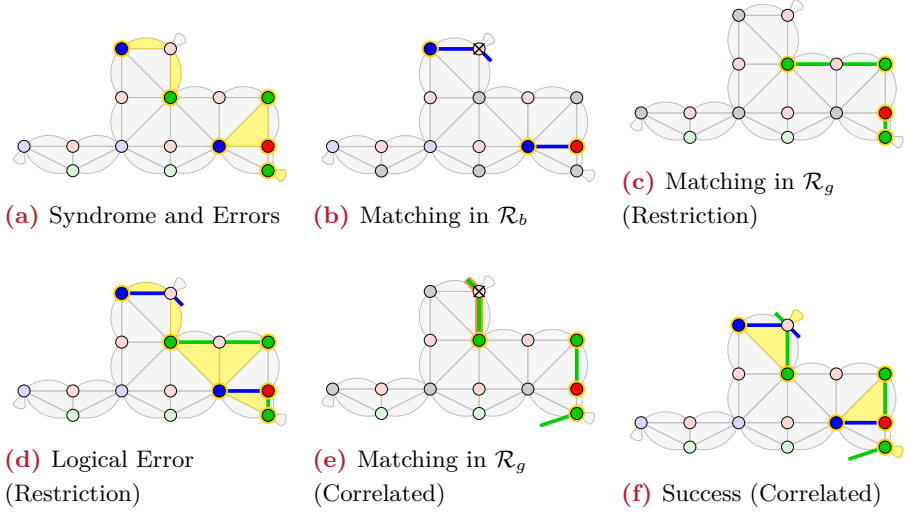


Figure 4.14. Detailed step-by-step comparison showing how the Correlated Decoder corrects a physical error configuration that causes the Restriction Decoder to fail. **(a)** The true physical errors (yellow faces) generate a syndrome in the lattice. **(b)** Both decoders first compute a matching in the $\mathcal{R}_b$ projection. For the correlated decoder, any red detector fully traversed by this path is flagged (black cross). **(c)** The Restriction Decoder computes the $\mathcal{R}_g$ matching independently. **(d)** The boolean sum of both independent matchings results in a logical error (predicted operations shown in yellow). **(e)** The Correlated Decoder incorporates the flagged detectors into $\mathcal{R}_g$ by assigning zero-weight to all connected edges (orange glow), which created a lower weight path and thus a new matching. **(f)** The boolean sum of the correlated matchings successfully mitigates the physical error.

lattice. The decoder inspects the resulting error path and identifies the shared nodes (the red checks) that have been completely traversed logically. A completely traversed red check implies the presence of a weight-2 error that did not trigger a defect on the check itself. These specific red checks are internally marked.

2. **Graph Re-weighting and Second Matching:** Before executing the matching on the second restricted graph ($\mathcal{R}_g$), the algorithm modifies its structure. Any edges in the $\mathcal{R}_g$ graph that are connected to the marked red checks have their weights set to zero. The MWPM algorithm is then executed on this modified $\mathcal{R}_g$ lattice. By making these paths weight-free, the second matching instance is strongly

guided to route its errors through the exact same red checks.

While the decoding algorithm is presented as an evolution over the restriction decoder, the authors rely on a different method to resolve the final physical correction. Rather than performing a lifting to map edges back to physical qubits, the authors propose relying on the mathematical equivalence between the color code under bit-flip noise and the surface code under depolarizing noise. In this mapping, the diagonal errors on the red squares of the color code translate to Y errors (i.e. combined X and Z errors) on single physical qubits of the surface code, while the adjacent errors on the same squares translate to only one error type, enabling the decoding process to distinguish them. Furthermore, because the equivalent surface code lattice does not contain parallel edges between stabilizers, the output of the matchings translates directly to specific physical qubits. The decoder simply uses a translation dictionary to convert the surface code correction back into the exact two-qubit errors on the color code, bypassing the need for a local lifting step.

Through this sequential correlation and surface code mapping, the algorithm successfully corrects the low-weight errors that the standard Projection and Restriction decoders miss.

Implementation and simulation of QEC codes: Framework setup and baseline models

We have so far conducted a profound theoretical study of the different QEC codes, decoders, and error models. Moving from theory to practice requires specialized computational tools capable of simulating quantum systems efficiently.

Simulating arbitrary quantum circuits is exponentially hard for classical computers; however, the Gottesman-Knill theorem [37] guarantees that circuits restricted to Clifford group operations (which include stabilizer measurements, CNOTs, and Pauli gates) can be simulated efficiently in polynomial time.

In this chapter, we establish our experimental framework using `stim` [8], a high-performance Clifford simulator that relies on the referred Gottesman-Knill theorem and its subsequent improvements by Aaronson and Gottesman[38], and incorporates some additional improvements. We will build our simulations progressively: starting with a basic repetition code to understand the mechanics of syndrome extraction and decoding,

The Gottesman-Knill theorem is referred to in the original Gottesman paper simply as Knill's theorem, as it was initially only privately communicated. It is now commonly known as the Gottesman-Knill theorem to recognize both Knill's original insight and the subsequent formal proof by Gottesman using his Stabilizer formalism.

moving through different noise models, and finally implementing the 9-qubit Shor code to empirically demonstrate the Threshold Theorem.

5.1 Simulating QEC codes and decoders with `stim`

Stim is a QEC specialized simulator. The primary advantage of `stim` is its ability to sample detection events –syndromes– and logical observables –flips in the logical state of our code– extremely fast. Instead of tracking the full state vector, it tracks how Pauli errors propagate through the circuit. To utilize this tool, a QEC simulation requires three main components: a circuit defining the code structure, a noise model that injects errors, and a classical decoder that interprets the syndromes.

5.1.1 Simulating the repetition code under code-capacity noise

To introduce the simulation pipeline, we begin with the simplest QEC protocol: the distance- d repetition code. As discussed in Chapter 1, this code protects against a single type of error (e.g., bit-flips) by encoding one logical qubit into d physical data qubits, requiring $d - 1$ additional ancilla qubits to perform parity measurements.

For our baseline simulation, we use a **code-capacity noise model** (see Chapter 3) using a depolarizing channel.

Stim makes it exceptionally easy to implement a basic repetition code. The building blocks of this simulation are:

- **The QEC code circuit.** `stim`'s library already provides a method to easily generate a basic repetition code. This generates not only the circuit for the encoding, but the necessary ancilla qubits and measurements for syndrome extraction.
- **Injecting the error.** As part of building the circuit, `stim` allows inputting the physical error rates for different types of noise. In this case, to simulate code-capacity noise, we use the parameter `before_round_data_depolarization`.

- **Decoding the error.** The `pymatching` library [25] will allow us to decode the error using the MWPM algorithm that we discussed in Chapter 4. For such decoding to happen, `stim` requires the circuit to have specific tags appended (`DETECTORS`) to specific measures. In this example, the generated circuit already contains them.

The code in Listing 5.1 illustrates how this simulation can be performed. It is important to understand how `stim` works. The `sample` method returns two things:

1. A binary array of detection events (in the code, `syndromes`). It contains one value for each measured detector. The value is 0 if the detector observed the expected parity, and 1 if the detector was activated, indicating the presence of an error.
2. A binary array of logical outcomes (in the code, `actual_observables`). Rather than representing logical states, these values act as boolean flags. A value of 1 indicates that the accumulated physical errors during that shot successfully flipped the corresponding logical observable.

During the simulation, errors can occur with a certain probability p . When they happen, they are tracked and cascade down to the detectors and observables, allowing `stim` to determine both if a detector is triggered and if an observable flips. If the code and the decoding succeed, both should match. Because flips, rather than states, are tracked, the circuit does not have to be prepared on a certain state, we only evaluate its capacity to preserve its state.

Figure 5.1 shows the logical error rate as a function of the physical error rate for this code with two different decoders. Note that, as a basic repetition code, this code only corrects for bit-flip errors.

The preparation of the initial state of a circuit is not generally required. However, noise models can be asymmetric (e.g. a qubit may be more likely to flip from $|1\rangle$ to $|0\rangle$ than from $|0\rangle$ to $|1\rangle$) and so can be QEC codes (the 9-qubit Shor code has better protection against bit-flip errors than against phase-flip errors). In these cases, preparing the initial state may be required.

Source: `src/chapter_5/s1_stim_rep_code.py`

```

7 def sample_repetition_code_mwpm(distance: int = 3, shots: int = 10000)
  -> None:
8     """
9     Simulates a repetition code over a range of physical error rates,
10    using the pymatching MWPM decoder.
11    """
12    physical_error_rates = np.linspace(0.01, 0.15, 10)

```

```
13     logical_error_rates = []
14
15     for p in physical_error_rates:
16         # Define the circuit
17         circuit = stim.Circuit.generated(
18             "repetition_code:memory",
19             distance=distance,
20             rounds=1,
21             before_round_data_depolarization=p
22         )
23
24         # Sample the circuit to get syndromes and actual observables
25         sampler = circuit.compile_detector_sampler()
26         syndromes, actual_observables = sampler.sample(shots,
27             separate_observables=True)
28
29         # Decode the syndromes using PyMatching
30         matcher = pymatching.Matching.from_stim_circuit(circuit)
31         predicted_observables = matcher.decode_batch(syndromes)
32
33         # Calculate the logical error rate as the cases where the
34         # actual observables differ from the predicted ones
35         num_errors = np.sum(predicted_observables !=
36             actual_observables)
37         logical_error_rates.append(num_errors / shots)
38
39     return physical_error_rates, logical_error_rates
```

Listing 5.1. Simulation 1: Basic repetition code simulation

5.1.2 Decoders: From Minimum Weight Perfect Matching to Custom Logic

Once the circuit is executed and noise is applied, `stim` outputs an array of detection events. The role of the decoder is to map these events to a predicted logical error.

As seen in Chapter 4, the standard approach for 1D repetition codes and 2D surface codes is the **Minimum Weight Perfect Matching (MWPM)** algorithm. In our framework, we implement this using the `PyMatching` library.

To better understand the internal workings of a decoder, we implement

a custom **Majority Vote Decoder**. Unlike the graph-based approach of MWPM, this decoder resolves errors by directly evaluating the local parity checks and “voting” on the most likely state of the data qubits.

The code Listing 5.2 shows a very simple implementation of such decoder, which can be used with the same repetition circuit created in Listing 5.1.

Note that we show here the specific part of the code that decodes a single shot of the sampling. When the circuit is simulated, multiple shots are run to get statistically significant information

Source: `src/chapter_5/s2_majority_vote_decoder.py`

```

10 # We iterate over each observable, which corresponds to a block of
    detectors
11 # This would allow codes encoding multiple logical qubits
12 for i in range(num_observables):
13     # The number of detectors in our code, for a given observable
14     # is a function of the code distance.
15     # We may receive additional detectors (frontier detectors) that we
        can ignore.
16     # For the 3-qubit repetition code, we have 2 detectors per
        observable.
17     # We need to iterate over the blocks of detectors for each
        observable
18     start_index = i * detectors_per_observable
19     end_index = start_index + (distance - 1)
20     local_syndrome = shot_syndrome[start_index:end_index]
21
22     # For a repetition code, decoders are based on pairs of data
        qubits
23     # that are chained, e.g. (q0, q1) and (q1, q2).
24     # We anchor to the first qubit, assume that it has not flipped (set
        to 0).
25     qubits_state = [0]
26     current_state = 0
27
28     # Let's iterate over the detectors
29     for detector_value in local_syndrome:
30         # If a given detector is triggered, it indicates a flip
31         # of this qubit relative to the previous one.
32         if detector_value == 1:
33             # We flip with respect to the prior qubit state
34             current_state = 1 - current_state
35             # Store the state of this qubit
36             qubits_state.append(current_state)
37
38     # If, as we assumed, q0 has not flipped, then our qubit_state
        tells us

```

```
39      # which qubits have flipped.
40      # But if q1 has flipped, then our qubit_state is the opposite
41      # of the actual flip-state of each qubit.
42
43      # If we have more 1s than 0s, that is a very unlikely scenario
44      # (2 bits flipped)so we can infer that q1 had actually flipped
45      # (and our qubits_state is the opposite of the actual flip-state).
46      num_ones = sum(qubits_state)
47      num_zeros = len(qubits_state) - num_ones
48
49      # We now need to make our flip-prediction for the observable.
50      # Stim's circuit will contain the observable measurement in the
51      last qubit
52      # of the block, because it uses the last measurement.
53      if num_ones > num_zeros:
54          # If we have more 1s than 0s, we assume that our flip-states
55          are inverted,
56          # so we take the opposite of the last qubit's flip-state
57          # as our prediction for the observable.
58          local_prediction = 1 - qubits_state[-1]
59      else:
60          # If we have more 0s than 1s, we assume our flip-states are
61          correct.
62          # So the flip-state of the last qubit is the right flip-state
63          # for the observable.
64          local_prediction = qubits_state[-1]
65
66      shot_predictions.append(local_prediction)
```

Listing 5.2. Simulation 2: Implementation of a simple Majority Vote decoder for a repetition code

As seen in Figure 5.1, for simple repetition codes under basic noise models, both decoders perform similarly, effectively reducing physical errors. In the following section we will stress-test our decoders to see how their efficiency decays with different noise models.

5.1.3 Introducing circuit-level noise

While the code-capacity model is useful for validating decoders, it does not reflect physical reality. In a real quantum computer, the gates used to entangle the qubits and the measurements used to extract the syndrome are themselves noisy.

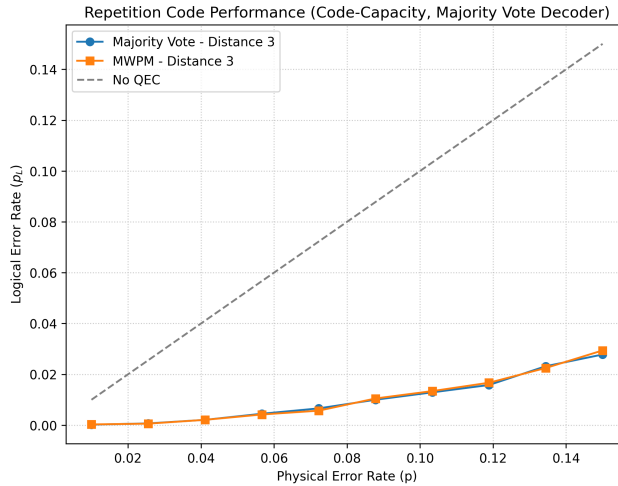


Figure 5.1. Simulation 3: Comparison of the results of simulations 1 and 2. Displays the logical error for a basic repetition code using Majority Vote and MWPM decoders. The $x=y$ dotted line helps comparing the QEC protocol to not implementing any QEC. If the logical error rate is above the physical error rate, the QEC protocol is adding errors

To illustrate this, we expand our simulation to include **circuit-level noise**. As seen in Chapter 3, in this model, we inject depolarizing noise into single-qubit and two-qubit gates, and we add a probability of measurement readout errors. For the purpose of this test, we will use a very simple noise model, where the three types of noise added to the circuit have the same probability (ratio 1:1:1). This is easily implemented with `stim` in the circuit creation, as seen in Listing 5.3.

Source: `src/chapter_5/s4_majority_vote_noise.py`

```

16 circuit = stim.Circuit.generated(
17     "repetition_code:memory",
18     distance=distance,
19     rounds=1,
20     before_round_data_depolarization=p,
21     after_clifford_depolarization=p,
22     before_measure_flip_probability=p
23 )

```

Listing 5.3. Simulation 4: Adding circuit-level noise.

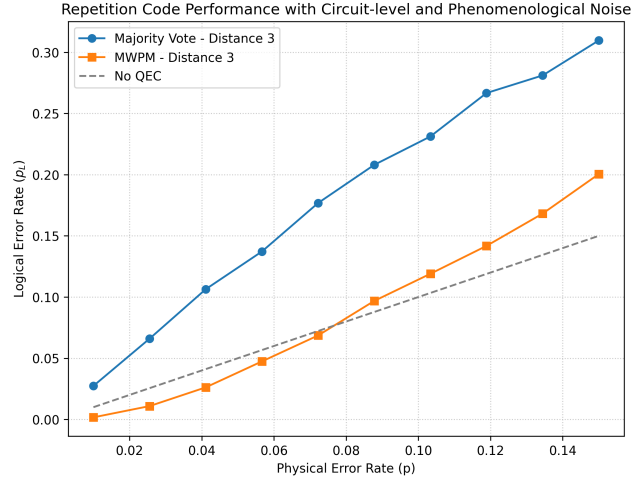


Figure 5.2. Simulation 4: Impact of circuit-level and phenomenological noise on the repetition code performance. Introducing more complex noise significantly degrades the logical error suppression compared to the idealized code-capacity baseline. With this additional noise, MWPM shows its advantage over the Majority Vote because of its capacity to consider the probabilities of each error to select the most probable set of errors that could have caused the observed syndrome.

As shown in Figure 5.3, realistic noise drastically reduces the effectiveness of the code, but it also shows the importance of the decoder. Handling circuit-level noise requires complex spatial-temporal decoding graphs, which makes MWPM a better solution than a basic Majority Vote decoder. The figure also shows how increasing the distance improves the results of the MWPM decoder, but it degrades the result of the Majority Vote decoder. This means that, for this specific noise model and the MWPM decoder, the system is operating under the threshold, so the error correcting capacity of the protocol can be increased by increasing the distance. However, with our Majority Vote decoder, the system operates above the threshold.

5.1.4 Implementing the 9-qubit Shor code

In our previous simulations, we relied on `stim`'s built-in circuit generation library. To better illustrate how a quantum error correction circuit is explicitly constructed from scratch, we now implement the 9-qubit Shor

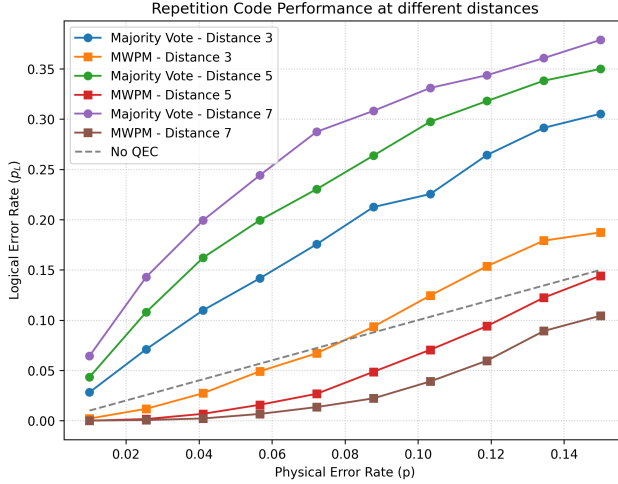


Figure 5.3. Simulation 5: Under a complex noise model (code-capacity, circuit-level and phenomenological) increasing the distance of the code has the opposite effect for each of the decoders.

code ($[[9, 1, 3]]$). Building a custom circuit in `stim` follows a strict pipeline.

First, the circuit is declared and initialized, followed by the application of the necessary gates to encode the logical state and extract the stabilizer operators. A crucial step in custom `stim` circuits is explicitly tracking the outcomes: stabilizer measurements must be labeled as `DETECTORS` (Listing 5.7), while the final measurements of the data qubits are targeted by `OBSERVABLE_INCLUDE` instructions to track the logical state parity (Listing 5.8). This strict labeling allows the simulator to decouple the syndrome extraction—which provides the inputs for the decoder—from the logical observable tracking, which is ultimately compared against the expected baseline to determine if a logical failure occurred.

Finally, to evaluate the code under realistic conditions, noise must be integrated into the circuit. Unlike the automated generator, building a circuit from scratch requires manually injecting specific noise channels—such as gate depolarization, measurement flips, or idle decoherence—at the exact coordinates where they occur (Listings 5.4, 5.5, and 5.6).

To illustrate the impact of different noise models, our initial analysis compares the performance of the Shor code under code-capacity, phenomenological, and circuit-level noise. The results demonstrate how it

Because the final measurement of the data qubits can also be noisy, we must add one last set of detectors at the end of the circuit. These detectors, known as boundary detectors, compare the parity of the final data measurements against the last syndrome extracted via ancillas, effectively catching readout errors in the final layer.

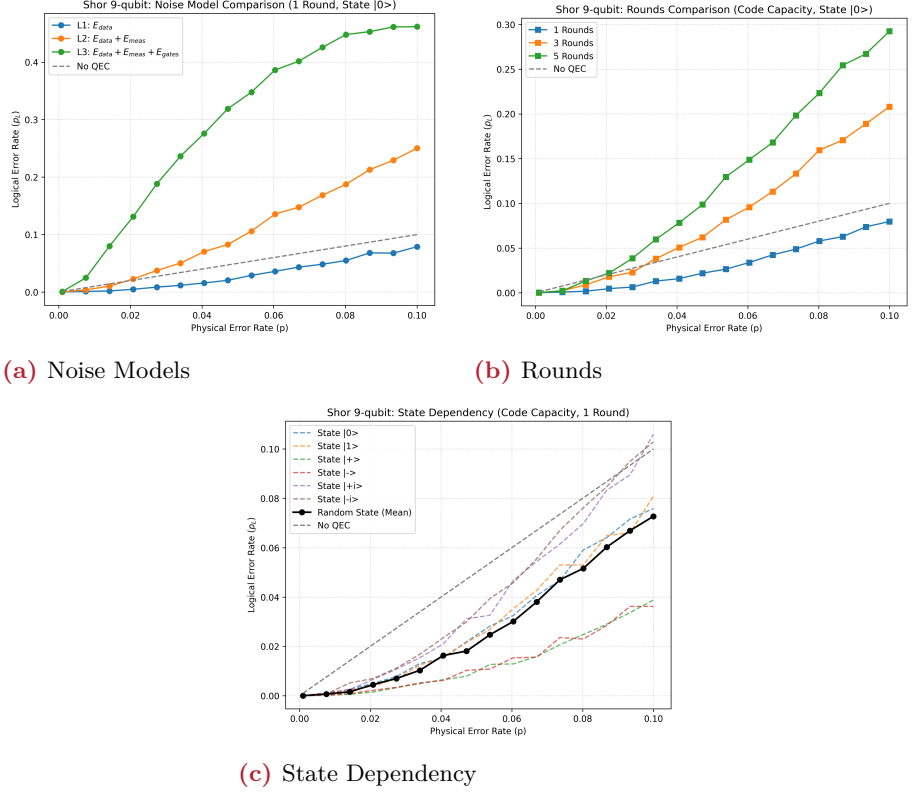


Figure 5.4. Simulation 6: Performance analysis of the Shor 9-qubit code using the MWPM decoder. **(a) Noise models:** Comparison between Code-capacity, Phenomenological, and Circuit-level noise. **(b) Rounds:** Accumulation of errors over syndrome measurement rounds. **(c) State Dependency:** Performance for different initial states. When measuring in the Z -basis ($|0\rangle$ or $|1\rangle$), the logical outcome is insensitive to logical phase-flips (physical X errors) but vulnerable to logical bit-flips (physical Z errors). Conversely, measurements in the X -basis ($|+\rangle$ or $|-\rangle$) are vulnerable to logical phase-flips but insensitive to logical bit-flips. Due to the code’s concatenated structure, there are fewer fatal combinations for physical X errors than for physical Z errors, resulting in higher logical protection for the $|\pm\rangle$ states. Finally, measurements in the Y -basis ($|\pm i\rangle$) are vulnerable to fatal combinations of both physical X and physical Z errors, effectively accumulating the vulnerabilities of both previous cases and resulting in the highest logical failure rate.

is highly vulnerable to error from noisy gates and faulty measurements, degrading its error-correcting capabilities.

We also analyze the accumulation of errors across multiple syndrome extraction rounds. Our results show that the logical error rate scales rapidly with each round, making the code unsuitable for long computations. While the standard solution to this would be increasing the distance, the Shor code does not scale well: increasing the distance increases the number of necessary qubits exponentially and also increases the number of qubits that must be measured by a single detector. Furthermore, the code is not robust against noisy gates or measurements. Topological codes, which we simulate in the following sections, offer solutions to these problems through improved scaling and the capacity to handle time-dependent noise.

Finally, by preparing the system in different cardinal states before measurement, we expose the logical observable to different types of uncorrected physical errors. This methodological choice empirically reveals how the code performs asymmetrically due to the inherent characteristics of its concatenated structure.

To evaluate how the code performs against different error types, we could have explicitly biased the physical noise channels. However, by injecting uniform depolarizing noise and altering the initial states instead, we preserve a standardized noise model while probing specific vulnerabilities.

Source: `src/chapter_5/s6_shor_9_code.py`

```

6 def append_noisy_gate(circuit: stim.Circuit, gate: str, targets: list,
   noise: float) -> None:
7     """Appends a gate and adds depolarizing noise if noise > 0."""
8     circuit.append(gate, targets)
9     if noise > 0:
10         if gate in ["CX", "CY", "CZ", "SWAP"]:
11             circuit.append("DEPOLARIZE2", targets, noise)
12         else:
13             circuit.append("DEPOLARIZE1", targets, noise)

```

Listing 5.4. Simulation 6: Injecting circuit-level noise by appending error channels to quantum gates.

Source: `src/chapter_5/s6_shor_9_code.py`

```

17 def append_noisy_measurement(circuit: stim.Circuit, targets: list,
   noise: float) -> None:
18     """Appends a measurement with pre-measurement bit-flip noise."""
19     if noise > 0:
20         circuit.append("X_ERROR", targets, noise)
21     circuit.append("M", targets)

```

Listing 5.5. Simulation 6: Simulating readout errors by assigning flip probabilities to measurement operations.

Source: src/chapter_5/s6_shor_9_code.py

```
62 # Syndrome measurement rounds
63 for r in range(rounds):
64     if before_round_data_depolarization > 0:
65         circuit.append("DEPOLARIZE1", range(9),
                        before_round_data_depolarization)
```

Listing 5.6. Simulation 6: Applying data qubit decoherence during idle cycles. Each round represents a time step where the logical state must be preserved, followed by detector evaluations to track newly occurring errors.

Source: src/chapter_5/s6_shor_9_code.py

```
72 append_noisy_measurement(circuit, range(9, 15),
                          before_measure_flip_probability)
73 for i in range(6):
74     if r == 0: circuit.append("DETECTOR", [stim.target_rec(-6 + i)])
75     else: circuit.append("DETECTOR", [stim.target_rec(-6 + i), stim.
                          target_rec(-6 + i - 8)])
```

Listing 5.7. Simulation 6: Declaring circuit detectors. Following a stabilizer measurement, a DETECTOR is defined by comparing the current outcome with the corresponding record from the previous round to identify parity changes.

Source: src/chapter_5/s6_shor_9_code.py

```
100 append_noisy_measurement(circuit, range(9),
                           before_measure_flip_probability)
101
102 # Boundary detectors and observables
103 if basis == "X":
104     for i, targets in enumerate([(0,1), (1,2), (3,4), (4,5), (6,7),
105                                (7,8)]):
106         circuit.append("DETECTOR", [stim.target_rec(-17 + i), stim.
107                                     target_rec(-9 + targets[0]), stim.target_rec(-9 + targets[1])])
108         # Logical X Distinguishes + and - (Measure Z0*Z3*Z6)
109         circuit.append("OBSERVABLE_INCLUDE", [stim.target_rec(-9), stim.
110                                     target_rec(-6), stim.target_rec(-3)], 0)
111 elif basis == "Z":
```

```

109     circuit.append("DETECTOR", [stim.target_rec(-11)] + [stim.
        target_rec(-i) for i in range(4, 10)])
110     circuit.append("DETECTOR", [stim.target_rec(-10)] + [stim.
        target_rec(-i) for i in range(1, 7)])
111     # Logical Z Distinguishes 0 and 1 (Measure transversal X parity)
112     circuit.append("OBSERVABLE_INCLUDE", [stim.target_rec(-i) for i in
        range(1, 10)], 0)
113 else:
114     # For basis Y, we just measure transversal parity
115     circuit.append("OBSERVABLE_INCLUDE", [stim.target_rec(-i) for i in
        range(1, 10)], 0)

```

Listing 5.8. Simulation 6: Final destructive measurement of the data qubits, including the explicit declaration of boundary detectors and the logical observable target.

5.1.5 Visualizing the Threshold Theorem

To conclude our exploration of baseline models, we return to the repetition code to visually demonstrate one of the most fundamental concepts in quantum error correction: the Threshold Theorem. As discussed theoretically in Chapter 1, this theorem guarantees that if the physical error rate is below a certain threshold p_{th} , we can arbitrarily reduce the logical error rate by increasing the distance of the code.

To observe this transition clearly in a simulation, two critical adjustments are required. First, we must adopt an appropriate noise model. Under pure code-capacity noise, the 1D repetition code lacks a standard visible threshold because the decoder can adapt to extreme error rates with perfect syndromes. We opt for a noise model that is fundamentally code-capacity, but we inject small rates of circuit-level noise (specifically, small probabilities of measurement and gate errors, at a 1:0.05:0.05 ratio). This forces the decoder to handle imperfect syndromes, allowing us to see exactly how the code behaves when its limits are tested.

Second, we must ensure that the number of syndrome extraction rounds grows proportionally with the code distance (setting `rounds = d`). Fault tolerance must be achieved in both space (number of qubits) and time (measurement rounds). If we were to increase the number of qubits but only measure the syndrome once, the code would be heavily protected spatially but extremely fragile temporally. By increasing the rounds

The noise model in this simulation has been specifically selected to cause a visible threshold while having a logical error rate that is lower than the physical error rate. Circuit-level noise is required for the code to have a visible threshold. Such noise needs to be at a very low rate because a simple repetition code does not perform well and quickly saturates at the maximum logical noise.

along with the distance, we force the larger codes to survive the noise for a longer period. While this might intuitively seem unfair—giving larger codes more opportunities to fail—the power of the Threshold Theorem is that, below the threshold, the error suppression is so strong that the larger code still achieves a lower total error rate despite enduring more rounds of noise. Figure 5.5 demonstrates this effect perfectly.

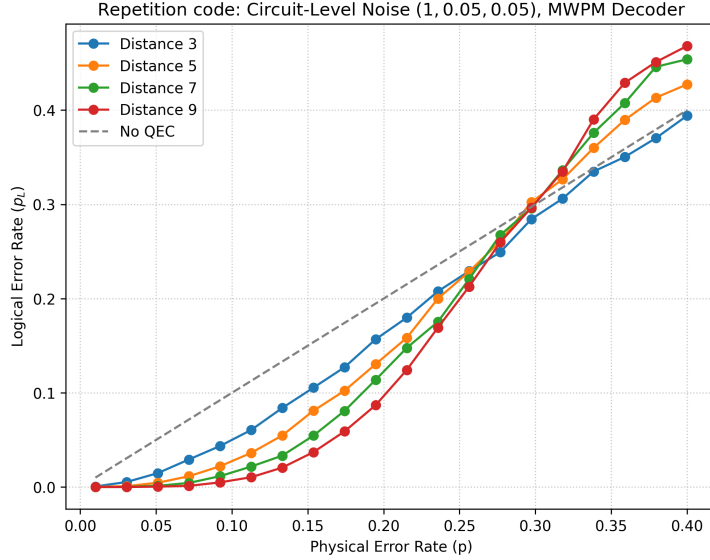


Figure 5.5. Simulation 7: Visualizing the Threshold Theorem using a repetition code under a predominantly code-capacity noise model with small rates of circuit-level noise (ratio 1:0.05:0.05). For physical error rates below the threshold (the point where the curves intersect), increasing the distance of the code strictly improves its performance, driving the logical error rate down. Conversely, for physical error rates above the threshold, the noise overwhelms the system, and adding more qubits only introduces more errors, degrading the overall performance.

5.2 Beyond stim: Experimental setup for implementation and simulation

As demonstrated in the previous sections, `stim` is an exceptionally powerful tool for defining and sampling quantum circuits. However, when transitioning from baseline models to complex, scalable topological codes

like the Surface Code or the Color Code, relying solely on `stim` becomes insufficient.

To efficiently perform large-scale research, we must introduce two additional abstraction layers on top of `stim`: one for orchestrating massive parallel simulations, and another for procedurally generating highly complex, geometrically constrained circuits.

5.2.1 Sampling technologies: `sinter`

When evaluating QEC protocols below their threshold, the logical error rate decays exponentially. To observe statistically significant failure rates at extremely low physical error probabilities (e.g., 10^{-6}), billions of circuit shots must be sampled. Executing these operations sequentially in standard Python loops is computationally unfeasible.

To solve this, we use `sinter`, a specialized tool designed to orchestrate and parallelize `stim` simulations. `Sinter`'s primary advantages are:

- **Automatic parallelization:** It seamlessly distributes the sampling tasks across all available CPU cores.
- **Dynamic stopping criteria:** Instead of executing a fixed number of shots, `sinter` allows defining a maximum number of logical errors (e.g., `max_errors=500`). The simulation will automatically halt for a given data point once sufficient statistical confidence is reached, saving immense computational time.
- **Data management:** It automatically aggregates the results into structured CSV files, mitigating the risk of data loss during long-running simulations.

Listing 5.9 illustrates how to configure and execute a massively parallel simulation using `sinter`.

Source: `src/chapter_5/s8_beyond_stim.py`

```

84 # Define the tasks we want Sinter to simulate
85 tasks = []
86 if distances is None:
87     distances = [3, 5, 7]
88 if physical_errors is None:
89     physical_errors = np.linspace(0.01, 0.4, 20)

```

```
90
91 for d in distances:
92     for p in physical_errors:
93         circuit = build_rep_code_cirq(distance=d, p=p)
94         tasks.append(
95             sinter.Task(
96                 circuit=circuit,
97                 json_metadata={'d': d, 'p': p}
98             )
99         )
100
101 # Collect statistics using Sinter in parallel
102 print("Starting Sinter parallel sampling...")
103 stats = sinter.collect(
104     num_workers=num_workers,
105     tasks=tasks,
106     decoders=['pymatching'],
107     max_shots=max_shots,
108     max_errors=max_errors,
109     print_progress=True
110 )
```

Listing 5.9. Simulation 8: Executing a large-scale parallel simulation using `sinter`. The tool samples up to a million shots, but halts early if 500 errors are found. `Sinter` automatically parallelizes using multiple workers.

5.2.2 High level circuit design for quantum error correction: `cirq` and `stimcirq`

While `stim` includes built-in generators for Surface Codes, it lacks native support to generate Color Codes. Constructing a generic topological code directly in `stim` requires manually managing numerical qubit indices and their intricate spatial interactions, a process that is highly error-prone.

To overcome this limitation, we utilize `cirq` [39], Google’s high-level framework for quantum circuits. `cirq` provides critical advantages for QEC design:

- **Geometric layout:** Qubits can be defined using planar coordinates (e.g., `cirq.GridQubit(x, y)`). This will allow us to map the hexagonal or triangular faces of a Color Code directly into the circuit’s architecture.

- **High-level abstractions:** Because of the mentioned geometric layout, operations can be designed geometrically, making syndrome extraction routines more intuitive.

However, mapping a high-level `cirq` circuit into a fault-tolerant simulation requires tracking metadata across the temporal and space dimensions. As shown in Listing 5.10, we must use the `stimcirq` integration library to manually annotate the circuit. This involves defining unique string keys for every measurement, strictly placing annotations in the right circuit tick (`cirq.Moment`), and explicitly declaring `stimcirq.DetAnnotation` to compare parity checks across rounds.

Once the circuit is constructed and annotated, it compiles into an ultra-fast `stim` circuit, granting us both the expressive power of `cirq` and the raw performance of `stim`.

Source: `src/chapter_5/s8_beyond_stim.py`

```

18 qubits = cirq.LineQubit.range(2 * distance - 1)
19 circuit = cirq.Circuit()
20 rounds = distance
21
22 for r in range(rounds):
23     # Add noise to data qubits. the NEW_THEN_INLINE prevents
24     # the noise annotation to be moved to the beginning.
25     circuit.append([cirq.depolarize(p).on(q) for q in qubits[0:2]],
26                    strategy=cirq.InsertStrategy.NEW_THEN_INLINE)
27
28     # CNOTs to compute parity -- First qubit is a data qubit. Then we
29     # alternate ancillas and qubits. Each ancilla
30     # is connected to two data qubits, one on each side.
31     # Odd qubits are ancillas, even qubits are data qubits.
32     circuit.append([cirq.CNOT(qubits[i-1], qubits[i]) for i in range
33                     (1, 2*distance-1, 2)], strategy=cirq.InsertStrategy.
34                     NEW_THEN_INLINE)
35     circuit.append([cirq.CNOT(qubits[i+1], qubits[i]) for i in range
36                     (1, 2*distance-1, 2)], strategy=cirq.InsertStrategy.
37                     NEW_THEN_INLINE)
38
39     # Measure ancillas (with explicit string keys for stimcirq
40     # tracking)
41     circuit.append([cirq.measure(qubits[i], key=f'm_{r}_{i}') for i in
42                     range(1, 2*distance-1, 2)], strategy=cirq.InsertStrategy.

```

```
NEW_THEN_INLINE)

35
36 # Define Detectors using stimcirq.DetAnnotation
37 det_ops = []
38 for i in range(1, 2*distance-1, 2):
39     key = f'm_{r}_{i}'
40     if r == 0:
41         det_ops.append(stimcirq.DetAnnotation(parity_keys=[key],
42         coordinate_metadata=(i, r)))
43     else:
44         prev_key = f'm_{r-1}_{i}'
45         det_ops.append(stimcirq.DetAnnotation(parity_keys=[key,
46         prev_key], coordinate_metadata=(i, r)))
47 circuit.append(det_ops, strategy=cirq.InsertStrategy.
48 NEW_THEN_INLINE)
49
50 # Reset ancillas for the next round
51 circuit.append([cirq.reset(qubits[i]) for i in range(1, 2*distance
52 -1, 2)], strategy=cirq.InsertStrategy.NEW_THEN_INLINE)
53
54 # Final Data measurement (of data qubits)
55 circuit.append([cirq.measure(qubits[i], key=f'd_{i}') for i in range
56 (0, 2*distance-1, 2)], strategy=cirq.InsertStrategy.
57 NEW_THEN_INLINE)
58
59 # Final Detectors (comparing data parity to last ancilla measurement)
60 # For each ancilla we add the ancilla measurement from prior round and
61 the measurement of the two data qubits that it is connected to.
62 final_det_ops = []
63 for i in range(1, 2*distance-1, 2):
64     ancilla_key = f'm_{rounds-1}_{i}'
65     data_left = f'd_{i-1}'
66     data_right = f'd_{i+1}'
67     final_det_ops.append(stimcirq.DetAnnotation(
68         parity_keys=[ancilla_key, data_left, data_right],
69         coordinate_metadata=(i, rounds)
70     ))
71 circuit.append(final_det_ops, strategy=cirq.InsertStrategy.
72 NEW_THEN_INLINE)
73
74 # Define Logical Observable
75 # The observable is simply the first data qubit
76 circuit.append(stimcirq.CumulativeObservableAnnotation(
77     parity_keys=[f'd_0'],
78     coordinate_metadata=(0, 0),
79     strategy=cirq.InsertStrategy.NEW_THEN_INLINE))
```

```

70     observable_index=0
71 ), strategy=cirq.InsertStrategy.NEW_THEN_INLINE)

```

Listing 5.10. Simulation 8: Using `cirq` to procedurally generate a fully functional Repetition Code. The circuit is manually annotated with noise models, `stimcirq.DetAnnotation`, and `stimcirq.CumulativeObservableAnnotation` to define the spatial-temporal parity checks required by decoders, ensuring the translation to `stim` is exact.

5.3 Automated noise injection: Extracting error models from Qiskit Fake Providers

In the preceding sections, we simulated circuit-level noise by manually injecting depolarizing channels and measurement flips at specific coordinates in the `stim` circuit. While this illustrates the mechanics of noise, it does not scale when we try to construct complex QEC codes.

Fortunately, `cirq` provides a robust mechanism to automate this process: the `cirq.NoiseModel` allows us to apply a set of noise rules to every operation in a circuit. To demonstrate its power, we will extract physical error rates from a real quantum processor using IBM’s Qiskit FakeProviders, and automatically inject them into our simulations.

5.3.1 Extracting physical parameters

Using Qiskit’s backend properties, we can extract the specific error rates of a physical chip. For our simulation, we compute the average single-qubit gate error (p_{1q}), two-qubit gate error (p_{2q}), and measurement readout error (p_{meas}).

Listing 5.11 demonstrates how these average parameters are programmatically extracted from a Qiskit backend.

Source: `src/chapter_5/s9_qiskit_noise_model.py`

```

14 def get_average_error_rates(backend):
15     """Extracts average error rates from a Qiskit backend."""
16     target_1q = []
17     target_2q = []
18     target_meas = []
19

```

In a strictly exhaustive circuit-level noise model, one should also include *idle noise*. Because two-qubit gate errors (p_{2q}) are typically orders of magnitude higher than idle decoherence during the gate’s duration, we approximate the system by only applying noise directly to active gates and measurements.

```
20     for q in range(backend.num_qubits):
21         meas_props = backend.target.get("measure", None)
22         if meas_props is not None and (q,) in meas_props:
23             target_meas.append(meas_props[(q,)].error)
24
25         sx_props = backend.target.get("sx", None)
26         if sx_props is not None and (q,) in sx_props:
27             target_1q.append(sx_props[(q,)].error)
28
29         cx_props = backend.target.get("cx", None)
30         if cx_props is not None:
31             for edge in cx_props:
32                 target_2q.append(cx_props[edge].error)
33
34         p_1q = np.mean(target_1q) if target_1q else 0.001
35         p_2q = np.mean(target_2q) if target_2q else 0.01
36         p_meas = np.mean(target_meas) if target_meas else 0.02
37
38     return p_1q, p_2q, p_meas
```

Listing 5.11. Simulation 9: Extracting average physical error rates from a Qiskit FakeProvider.

5.3.2 Automating noise injection with `cirq.NoiseModel`

Once the physical parameters are extracted, we must map them into our `cirq` workflow. By inheriting from `cirq.NoiseModel`, we can override the `noisy_operation` method to define specific rules for each gate type.

As shown in Listing 5.12, we apply bit-flips to measurements and depolarizing channels to gates.

Source: `src/chapter_5/s9_qiskit_noise_model.py`

```
42 class IBMNoiseModel(cirq.NoiseModel):
43     """Custom Cirq noise model applying extracted error rates."""
44     def __init__(self, p_1q, p_2q, p_meas, scale=1.0):
45         self.p_1q = min(p_1q * scale, 1.0)
46         self.p_2q = min(p_2q * scale, 1.0)
47         self.p_meas = min(p_meas * scale, 1.0)
48
49     def noisy_operation(self, operation: cirq.Operation):
50         if isinstance(operation, (stimcirq.DetAnnotation, stimcirq.
            CumulativeObservableAnnotation)):
```

```

51         yield operation
52         return
53
54         if circ.is_measurement(operation):
55             yield circ.bit_flip(p=self.p_meas).on_each(*operation.
qubits)
56             yield operation
57             elif len(operation.qubits) == 1:
58                 yield operation
59                 yield circ.depolarize(p=self.p_1q).on_each(*operation.
qubits)
60             elif len(operation.qubits) == 2:
61                 yield operation
62                 yield circ.depolarize(p=self.p_2q).on_each(*operation.
qubits)
63             else:
64                 yield operation

```

Listing 5.12. Simulation 9: Implementing a custom `cirq.NoiseModel` that applies the extracted error rates. The scaled error rates are bounded to avoid exceeding 1.0.

To run our simulations, we often want to plot physical noise in the X axis. Since our extracted Qiskit noise provides a single fixed point, we introduce a `scale` factor. We multiply the base error rates by this factor to artificially worsen or improve the hardware’s performance, allowing us to trace the error curve while maintaining the realistic relative ratios between single-qubit, two-qubit, and measurement errors.

When plotting these results, we must select a single metric for the X-axis to represent the physical error rate. We use the two-qubit gate error rate (p_{2q}) as the reference.

Listing 5.13 shows how we build a perfectly clean circuit, use the `with_noise` method to automatically inject noise in the circuit based on our IBM model.

Source: `src/chapter_5/s9_qiskit_noise_model.py`

```

131 noise_model = IBMNoiseModel(base_1q, base_2q, base_meas, scale=scale)
132 noisy_circuit = clean_circuit.with_noise(noise_model)
133 stim_circuit = stimcirq.cirq_circuit_to_stim_circuit(noisy_circuit)

```

Listing 5.13. Simulation 9: Applying the noise model to a clean circuit and converting it to stim for large-scale sampling.

Simulating 2D triangular color codes with a 4.8.8 lattice geometry and a code-capacity noise model

6.1 Syndrome Graph Construction and Decoding

6.1.1 The Detector Error Model (DEM)

As seen in Chapter 4, many modern decoders rely on mapping the decoding problem to a classical graph-theory problem. A key element of this framework is the **Detector Error Model**, which represents the error vulnerabilities with a specific quantum circuit. The DEM is constructed by `stim` before the simulation begins and passed to the decoder. It is constructed as an hypergraph:

- **Nodes (Detectors):** Each node represents a deterministic parity check (a *detector*), which evaluates to zero under noiseless conditions

Unlike standard graphs, in an hypergraph a hyper-edge can connect any number of nodes. That allows the edges in the DEM to connect one single physical error to any number of detectors (e.g., 1, 2, or 3).

(e.g., the comparison of a stabilizer measurement between cycle t and $t - 1$).

- **Hyperedges (Errors):** A hyperedge connects the specific set of detectors flipped by a single, independent physical error.
- **Weights:** Each hyperedge is assigned a weight calculated as $w = \ln((1 - p)/p)$, where p is the physical probability of that specific fault occurring.
- **Observable Payload:** Each hyperedge carries a boolean payload indicating whether the physical error causes a flip in the logical state (which means this specific physical error operation anti-commutes with the logical observable).

`stim` automates the generation of the DEM through fast symbolic simulation:

1. **Independent physical errors (nodes):** `stim` identifies every possible error in the circuit, which is defined by the error channels that have been injected in the circuit definition. This includes code-capacity, gate and measurement errors.
2. **Error simulation:** For each independent error identified, `stim` evaluates all the detectors and logical observables in the circuit. Instead of running a computationally expensive full simulation of the circuit, `stim` mathematically propagates each error forward through the circuit. Because code-correction circuits contain exclusively Clifford gates, determining how the error propagates is efficient.
3. **Logical error characterization (hyperedges):** `stim` observes exactly which detectors and logical observables are inverted by the propagated error. It then adds a hyperedge to the DEM connecting all the flipped detectors, alongside its probability and observable payload.

6.1.2 Decoding the DEM

In the simulation process, for each experiment shot, `stim` generates errors with the probability established by the error model and produces the syndrome (the list of detectors that have been activated) and the observable that is measured at the end of the execution.

The DEM serves as the static blueprint for the decoder. Given a certain syndrome, the decoder must determine the physical error (or combination of multiple physical errors, which we call the error chain) that caused such syndrome. Because different error chains can cause a given syndrome, the problem consist on determining the one that has the highest probability of occurring.

Once the most probable error chain is identified, the decoder computes the collective parity of their observable payloads (i.e. which of the physical errors in the chain cause a flip in the observable). If the final parity is 1, it predicts a logical flip. This prediction is compared against the actual logical observable measured by Stim to determine the success or failure of the decoding cycle.

6.2 Implementing a 2D color code

To systematically implement the 2D Color Code and allow for future scalability, we have adopted a modular software architecture. The problem has been divided into two strictly independent layers: the topological construction of the grid (*Lattice Construction*) and the quantum logic representation (*Circuit Generation*). This separation of concerns ensures that the circuit generator is completely agnostic to the underlying geometry; it simply receives a set of abstract shapes and coordinates, meaning new lattices (such as the 6.6.6 hexagonal grid) can be added in the future without altering the core quantum logic.

Furthermore, this approach simplifies visualizing the lattice geometry and confirming correctness before having to dive into the circuit implementation.

6.2.1 Lattice Construction

The specific implementation for this work is the `Lattice488` class, which procedurally generates the 4.8.8 semi-regular lattice (consisting of squares and octagons). The algorithm builds the lattice level by level:

1. It first places the `SquareShape` instances in a pyramid-like structure.
2. It then fills the gaps between these squares with `OctagonShape`

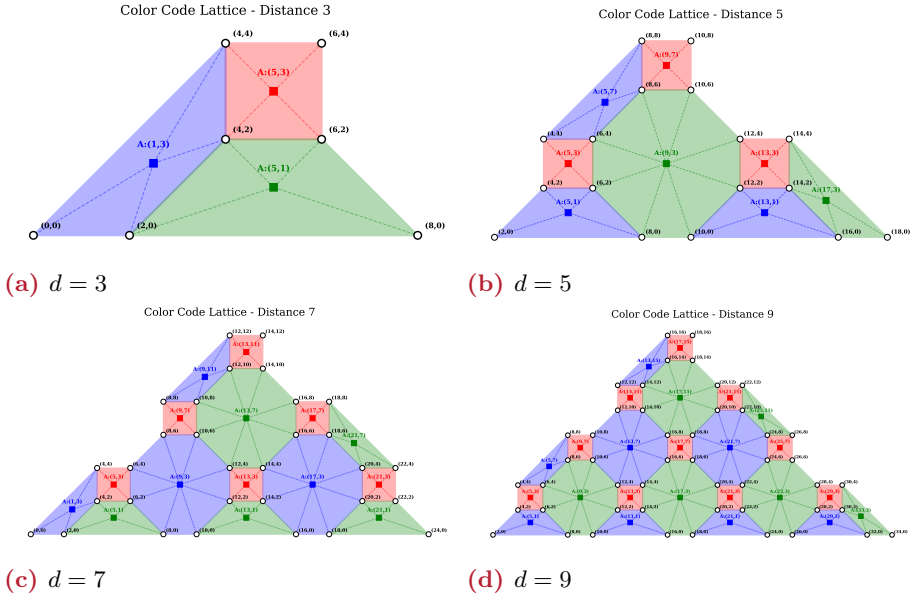
The abstract class `Lattice` serves as the foundation. It defines the generic behavior of a 2D topological grid, treating it simply as a collection of geometric shapes. Each shape inherits from a base `Shape` class and defines its own data qubits, its ancilla (detector) qubit, and its color.

instances.

3. Finally, to ensure all boundary data qubits are protected by valid stabilizers, it caps the left, right, and bottom boundaries with TrapezoidShape instances.

By dynamically calculating coordinates and alternating colors based on the distance d , the lattice scales automatically to any arbitrary size. Figure 6.1 shows the automated geometric evolution of the 4.8.8 lattice for distances 3, 5, 7 and 9.

To facilitate converting the lattice in a circuit, and inspired by the mapping of a 4.8.8. geometry to a square grid proposed by Fowler [19] and discussed in Chapter 2, we assign grid coordinates to each data and ancilla qubit in the grid. The geometry and the specific grid mapping results in the detectors having odd x and y coordinates, and data qubits having even x and y coordinates.



6.2.2 Circuit Generation

With the geometry resolved, the `ColorCode` class constructs the quantum circuit. It receives a `Lattice` object and extracts all unique coordinates, translating them into `cirq.GridQubit` objects.

Unlike a manual layout, this class dynamically maps the geometric shapes to stabilizer measurement routines. For each syndrome extraction round, it prepares the ancillas, applies Hadamard gates where necessary, and executes CNOT gates between each ancilla and its surrounding data qubits. Finally, it performs a measurement of the data qubits to extract the logical observable.

Detectors are defined after each syndrome extraction. To reduce the number of qubits, ancilla qubits are reused for the X and Z stabilizer measurement, which are sequential. It's worth noting that in the first round X detectors are not added. This is because, with the initial state of the qubits set as $|0\rangle$, measurements in the X axis are not deterministic, so the measured syndrome would be random. After the first round of syndrome measurements, qubit state collapses to a defined value, which can then be used in subsequent rounds to detect phase-flip errors. To allow Z errors to be corrected, we add one initial noiseless round that forces the syndrome to collapse so it can be used later to correct errors.

Similarly, the observable is defined at the end of the circuit. While in the 2D color codes observables are generally defined as a string-nets connecting the boundaries of the lattice (and with minimal weight), an observable defined by all the data qubits is topologically equivalent. This allows us to implement a very simple definition of the observable.

Figure 6.2 illustrates the resulting quantum circuit for a distance 3 code with a single measurement round.

6.2.3 X and depolarizing noise types

Our circuit generation is parametrized, allowing for two noise types: depolarizing and X . Because we are defining an observable in the computing basis, Z -type errors do not affect the observable. Adding Z and Y noise influences the results. While we want our decoders to be able to handle this type of noise in the DEM, to compare decoders fairly (both between

Under a code-capacity error model, the order in which CNOT gates are performed is not relevant. However, under more realistic error models, physical errors can occur during syndrome extraction and propagate through subsequent CNOT gates, causing correlated multi-qubit failures known as *hook errors*. This phenomenon requires a carefully planned CNOT schedule.

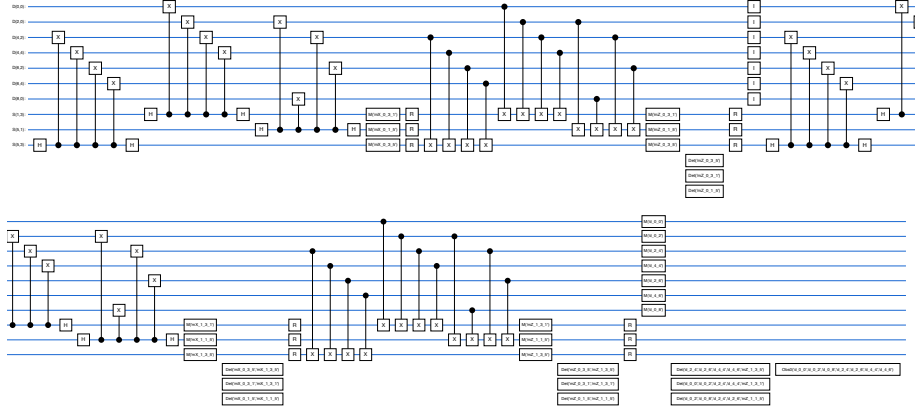


Figure 6.2. Complete quantum circuit for the 4.8.8 Color Code ($d = 3$, 1 round). The data qubits have been labeled as $D(x, y)$ and the ancilla qubits for the stabilizers as $S(x, y)$, so they are grouped to ease the visualization. Due to its significant width, the image has been split into two halves for readability. High resolution versions of these images are available in the github repository (see Page ix)

them and with the published results by other researchers), a model with only X-type errors is the most common reference in the published results, and is specially appropriate when the observable is immune to Z errors. In this model, all the Z detectors are removed and the error changes from depolarizing to bit-flip.

6.2.4 The color and detector type dimension

Inspired by how the Möbius decoder uses the `stim` annotation's metadata to make detector's face color and type available to the decoder, we incorporated the same solution in our circuit generation, as seen in Listing 6.1.

Source: `src/chapter_6/s1_color_code.py`

```

390 final_dets = []
391 for face in self.faces:
392     ax, ay = self.plot_coords[face['ancilla']]
393     keys = [f"mZ_{rounds}_{face['ancilla'].row}_{face['ancilla'].col}"
394             ]
395     keys.extend([f'd_{dq.row}_{dq.col}' for dq in face['data']])
396     c_idx = {'R': 3, 'G': 4, 'B': 5}[face['color']]

```

```

396     metadata = (ax, ay, rounds + 1, c_idx)
397     final_dets.append(stimcirq.DetAnnotation(
398         parity_keys=keys,
399         coordinate_metadata=metadata
400     ))
401     circuit.append(cirq.Moment(final_dets))

```

Listing 6.1. Implementation of the color dimensionality tag for the Möbius Decoder. The values $[0 - 2]$ represent X detectors of each color, and the values $[3 - 5]$ represent Z detectors of each color

This allows decoders to rely on the type (X or Z) and color of a detector to implement the detection algorithm. This is specifically relevant when performing projections (which is required by many decoders that utilize MWPM internally, as MWPM can't process hypergraphs) and when decomposing the Y-type errors into its X and Z components, which is again a frequent requirement for decoders to successfully process depolarizing noise.

6.3 Simulation Architecture and Evaluation Metrics

To systematically evaluate and compare the performance of different decoders on the 2D Color Code, it is essential to establish a robust, automated, and scalable simulation pipeline. Rather than writing isolated scripts for each decoding algorithm, we have designed a unified modular architecture that leverages the **sinter** library described in Chapter 5.

The architecture is composed of several main intercommunicating modules:

1. **The simulation manager (`simulator.py`):** This is the main entry point for a given simulation. It receives a configuration matrix detailing the code distances (d) and the physical error rates (p) to simulate. For each configuration, it invokes the **ColorCode** generator (discussed in Section 6.2) to dynamically create the Stim circuits and the corresponding Detector Error Models (DEM). Because generating these symbolic models can be computationally expensive, the simulator implements a caching system. It saves the `.stim` and

.dem files to disk, automatically detecting when parameters change to invalidate and clean the cache, speeding up subsequent runs.

2. **The decoder interface (decoder_interfaces.py):** To ensure the simulation pipeline works similarly with any decoding algorithm being tested, all decoders are wrapped into `sinter.Decoder`, which is an adapter class. This module acts as a plug-and-play interface that standardizes the input (syndrome data) and the output (logical predictions), allowing us to swap the decoder with a single parameter change.
3. **Data storage and visualization (plotter.py):** As `sinter` processes the shots, the raw statistical results (total shots, logical errors observed, and execution time) are systematically appended to a CSV file, so the data can be used and compared later, which will be critical in Chapter 7. Once the simulation concludes, the CSV data is plotted and stored. At this stage, we also perform an interpolation of the logical error rate curves to determine their crossing points, and average all the crossing points to estimate the threshold p_{th} . It is worth noting that for each of our simulations, once we estimate the threshold, we perform a second run of the simulation zooming in in the region where the estimated threshold is. This secondary simulation runs a significantly larger number of shots to perform a more accurate estimation of the threshold.
4. **General orchestration of simulation runs (run_simulations.py):** While each simulation can be run independently, this orchestrator allows running simulations and calculation processes for all the implemented decoders with a single command.
5. **Centralized configuration of simulation parameters (config.py):** This interface provides a centralized place to determine the simulation parameters (distances, physical error rates, number of shots, etc.) that are used by all the simulations. The file also supports overriding parameters for a specific decoder.

Additionally, for each decoder, the architecture is composed of the following:

1. **The simulation entry point (e.g. s4_restriction_simulation.py):**

this is the main python script that runs the simulations for a given decoder. They generally support three operating modes: **simulate** to run the main simulation and generate the corresponding data, **plot** to calculate the threshold and generate the figures and **zoom** to run a new, more detailed simulation around the threshold and generate the corresponding data and figures.

2. **The decoder implementation (e.g. `sinter_projection_decoder.py`):**

For those decoders that are not implemented by a third party library, we have implemented their algorithm wrapped on the `sinter.Decoder` interface.

3. **The data:** When each simulation is run, csv files with the results are stored in the data directory. Generally, one file stores the **main simulation results** and another one stores the **zoomed-in simulation results**. Two pairs of files are generated for each decoder and noise model (depolarizing or X-type only). The **threshold estimation** is stored in a separate, centralized file (`threshold_summary.csv`).

4. **The plots:** For each simulation (decoder, noise model), several plots are generated and stored in the `figures` directory: logical noise p_L as a function of physical noise p with a threshold estimation, a zoom-in around the threshold and the decoder execution time as a function of the code distance.

By structuring the project this way, we can guarantee that all algorithms are subjected to the exact same conditions. The captured data allows measuring the four key metrics that will drive the comparative analysis:

- **Determination of error thresholds (p_{th}):** Identifying the crossing point in the logical error curves where increasing the code distance begins to suppress errors rather than amplifying them.
- **Logical failure rate (p_L) scaling as a function of physical error rate (p) and code distance (d):** Analyzing the curves to evaluate the decoder's raw accuracy.
- **Sub-threshold scaling analysis and theoretical suppression limits:** Evaluating how steeply the logical error drops when operating below the threshold.

- **Computational complexity and empirical decoding time (Accuracy vs. Speed trade-off):** Analyzing the empirical decoding time to determine if the decoder’s complexity is practically scalable.

With this systematic architecture in place, we are now ready to implement and simulate the specific decoding algorithms.

We will simulate the different decoders at distances ranging from $d = 3$ to $d = 21$, with physical noise levels ranging between 2% and 15%. To guarantee robust statistics, especially at low physical error rates, we will execute simulations up to 40×10^6 shots per configuration, but stopping when 400 logical errors have occurred. While the simulations will be run both with depolarizing and bit-flip noise types (results are available in our repository), in this thesis we will primarily present the results for bit-flip noise, as it seems to be the most common reference in the literature.

6.4 BP+OSD

As discussed in Chapter 4, the Belief Propagation with Ordered Statistics Decoding (BP+OSD) algorithm provides a method to decode 2D Color Codes without projections or other adaptations. However, this flexibility comes at the cost of higher complexity. The BP+OSD algorithm allows several parameters to fine-tune its performance and complexity.

To carry out the numerical simulations for this decoder, we will use Roffe’s `ldpc` Python package [40], based on the algorithms published by the same author [29]. Originally developed to decode classical Low-Density Parity-Check (LDPC) codes, this library provides a robust C++ implementation wrapped in a Python interface, making it very efficient. The `ldpc` package includes native integration with the `sinter` framework via the `SinterBpOsdDecoder` class. This built-in adapter allows the decoder to ingest the parity-check matrices and Detector Error Models (DEM) generated by `stim`. It supports multiple BP update rules and implements the advanced OSD post-processing strategies introduced in 4.3, which become input parameters for the class.

In this section, we first perform a systematic optimization of these

parameters to identify the optimal configuration for our specific 4.8.8 lattice geometry. Subsequently, we use these parameters to execute a full threshold evaluation.

6.4.1 Parameter Optimization

To isolate the effect of the different BP+OSD parameters, we fixed the lattice distance to $d = 7$ and evaluated the decoder's performance across a range of physical depolarizing error rates. Our optimization focuses on three key components: the BP update rule, the OSD method, and the OSD search order.

BP Update Rule Comparison

The standard Belief Propagation algorithm utilizes the Product-Sum (PS) update rule, which computes exact probabilities. A widely used alternative is the Min-Sum (MS) approximation, which simplifies the check-node updates. This minimizes computational overhead and can mitigate the effect of the short cycles present in the color code Tanner graph.

Figure 6.3 compares both approaches. The empirical results show that the Min-Sum and Product-Sum rules yield virtually identical logical error rates. However, the Min-Sum approximation consistently reduces the average execution time. Based on these results, we select the Min-Sum (MS) update rule to accelerate all subsequent simulations.

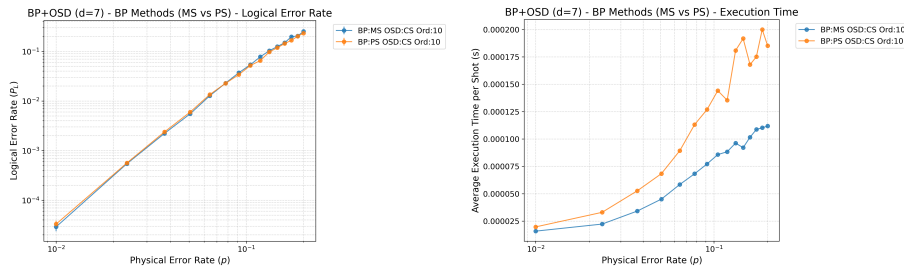


Figure 6.3. Performance comparison of the Min-Sum and Product-Sum BP update rules for $d = 7$. The logical error rate (left) remains largely unaffected, but the Min-Sum approximation provides a clear advantage in execution time (right).

OSD Method Comparison

When BP fails to converge, the OSD post-processing phase forces a valid solution based on the soft probabilities. We evaluated three OSD strategies, maintaining a constant search order of $k = 5$ when applicable:

- **Order-0 (`osd_0`):** Directly applies Gaussian elimination without further search.
- **Exhaustive (`osd_e`):** Performs an exhaustive search over the k least reliable qubits.
- **Combination Sweep (`osd_cs`):** Explores a wider non exhaustive search over the least reliable qubits, with k controlling the search depth.

While `osd_e` is an exhaustive search, it is limited to the k lowest-confidence qubits, which prevents it from evaluating highly probable error combinations that include other qubits. `osd_cs` overcomes this by sampling combinations across a wider subset of uncertain qubits.

As depicted in Figure 6.4, `osd_0` underperforms significantly because it entirely trusts the unconverged BP probabilities. The Exhaustive method (`osd_e`) improves the threshold, but the Combination Sweep method (`osd_cs`) heavily outperforms both by efficiently exploring combinations of likely error vectors. Consequently, `osd_cs` is selected as our baseline OSD method.

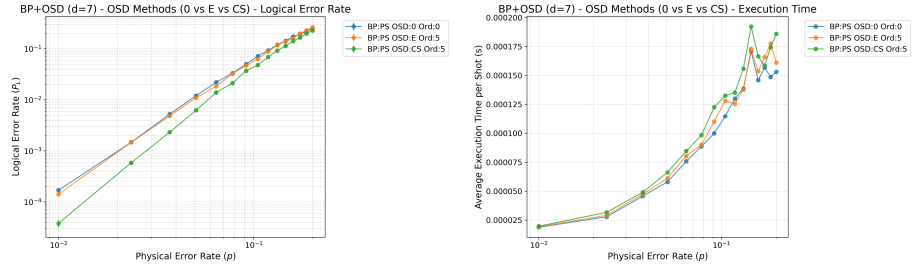


Figure 6.4. Comparison of different OSD post-processing strategies (`osd_0`, `osd_e`, `osd_cs`) at a fixed search order $k = 5$. The Combination Sweep method yields the lowest logical error rates.

OSD Search Order Analysis

Having selected `osd_cs`, we must define its search order k . A higher order allows the decoder to explore more alternative hypotheses, improving accuracy but heavily penalizing execution time due to the combinatorial explosion.

Figure 6.5 illustrates this trade-off. Increasing k from 5 to 10 yields some improvement in the logical error rate without significant impact to the execution speed. However, pushing k to 20 provides similar small improvements while severely impacting the execution speed. To maintain computational feasibility without significantly sacrificing the decoding threshold, we fix the OSD order at $k = 10$.

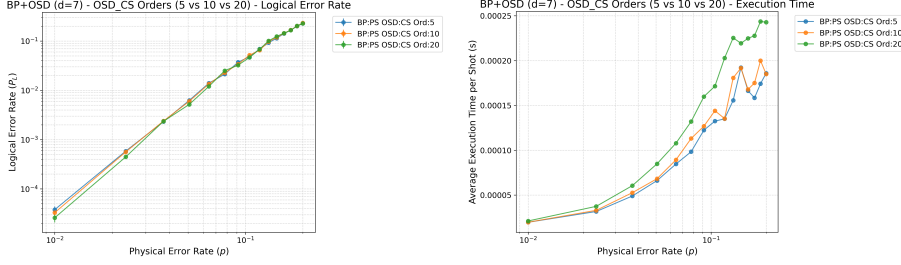


Figure 6.5. Effect of the OSD search order ($k \in \{5, 10, 20\}$) on the logical error rate and execution time. An order of $k = 10$ provides an optimal balance between accuracy and computational cost.

6.4.2 Threshold Evaluation

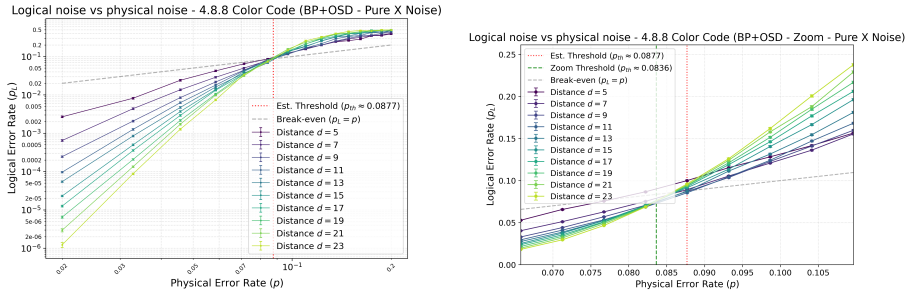


Figure 6.6. Threshold evaluation for the configured BP+OSD decoder on the 4.8.8 Color Code lattice under a pure bit-flip noise model. The standard logical error plot (left) and the high-resolution zoom (right) show the crossing point at $p_{th} \approx 8.36\%$.

With the optimal parameters established (**BP Method:** MS, **OSD Method:** `osd_cs`, **Order:** 10), we conducted a full Monte Carlo simulation to evaluate the algorithm’s threshold on the 4.8.8 color code lattice. We evaluated distances from $d = 5$ to $d = 23$.

The results of this simulation are presented in Figure 6.6. A high-

The threshold is calculated by interpolating the logical error curves for all pairs of distances and finding their crossing point, in a logarithmic representation. If the curves cross multiple times (which can happen because of the error margins of the Monte Carlo simulations), the minimum is selected (as, at higher noise levels, when logical error approaches the maximum theoretical limit (50%), the curves are packed together and more likely to cross due to limited precision. Finally, we determine the continuous mode of all the crossing points.

resolution zoom plot is also provided to further illustrate the threshold calculation. The logical error rate curves cross at an empirical threshold of $p_{th} \approx 8.36\%$ under a pure bit-flip noise model. Below this crossing point, we observe a clear exponential suppression of the logical failure rate as the code distance increases, empirically proving the capability of the BP+OSD algorithm to effectively decode 2D Color Codes under a code-capacity noise model.

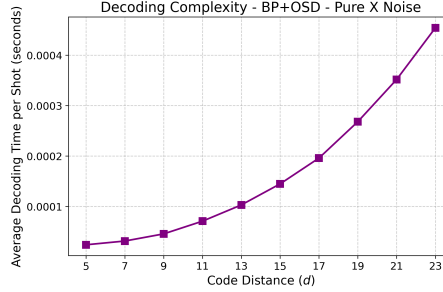


Figure 6.7. Average execution times per shot for the BP+OSD decoder under a pure X-noise model, confirming the polynomial scaling of the algorithm.

Published references and preliminary evaluation

While we haven’t found any direct reference of BP+OSD used against a 4.6.6 color code on code-capacity and bit-flip errors, Kung, Kuo and Lai achieved a $p_{th} = 15.43\%$ threshold applying BP+OSD to 6.6.6 color codes under a depolarizing noise model ([41]), which is higher than our results under depolarizing noise ($p_{th} = 13.51\%$). We can similarly estimate the equivalent threshold of Kung’s simulation for bit-flip noise as $p_{th}^{bit-flip} = \frac{2}{3} * p_{th}^{depolarizing}$, resulting in an estimated bit-flip noise threshold for their implementation on a 6.6.6. lattice of $p_{th} = 10.28\%$, above our observed threshold.

6.5 Projection decoder

In order to evaluate the topological Projection Decoder within our simulation pipeline, we implemented a custom decoder interface compatible with the `sinter` framework. We need to also implement the decoder itself, for which we have not found any library or existing implementation.

Delfosse [30] details the lifting process in an Appendix. Stephens [42] proposes an improvement to the algorithm. While we have not implemented such improvement, the results in their article served as a reference to validate our implementation.

Our implementation is divided into three stages: the Projection step (where the lattice is decomposed), the Minimum Weight Perfect Matching (MWPM) step, and the Lifting step. To improve performance, the projections and graph transformations are pre-compiled and cached for each specific code distance.

6.5.1 The Projection Step and DEM Generation

The first step of the algorithm consists of transforming the original 2D Color Code Detector Error Model (DEM) into three independent Surface Code DEMs (representing the RG , GB , and BR projections).

We iterate through all the independent error mechanisms defined in the original DEM. For a specific projection (e.g., RG), any error that triggers a removed detector (Blue in the example) is modified to ignore that detector. If an error mechanism triggers exactly two detectors of the remaining colors (one Red and one Green), it forms a valid 1-dimensional edge in the projected graph.

Because multiple physical qubits in the Color Code can collapse into the same parallel edge in the projected graph, we mathematically merge these parallel edges into a single virtual edge, keeping a map to identify the original physical qubits. In a full circuit-level noise simulation, the probability of this new merged edge would be calculated as a weighted sum of the independent error probabilities of its physical qubits. However, since our evaluation focuses on the code-capacity noise model, we implemented a simplified unweighted approach. Every edge is assigned the same error probability (e.g., $p = 0.01$).

Finally, these filtered and merged edges are compiled into three new, independent `stim.DetectorErrorModel` objects.

In this chapter, we often refer to physical qubits in the graph. This is a simplification, as the DEM and the corresponding graph actually contains possible errors causing the syndrome. Therefore, rather than physical qubits, the graph contains possible errors (e.g. X, Y and Z errors) affecting the physical qubit. For each physical qubit, in a depolarizing noise model, there are three possible errors

6.5.2 Running MWPM on the projected surface codes

With the three projected DEMs generated, the algorithm must decode them. We utilize `PyMatching` as the core MWPM engine for this task.

Typically, `PyMatching` takes a syndrome as input and returns a boolean array predicting whether the logical observables of the code have been flipped. However, the Projection Decoder requires knowing *exactly which physical edges* were selected by the MWPM algorithm to form the correction paths.

To force `PyMatching` to output this specific internal pathing information, we modify the DEM generation: we assign a unique, virtual logical observable (L_i) to every single edge in the projected graph. Consequently, when the MWPM algorithm is executed, the returned “observable flip” tell us all the edges that were identified as flipping in the projection. A value of 1 at index i indicates that the edge i was flipped (included in the correction path), while a 0 indicates it was not.

MWPM under depolarizing noise

MWPM’s struggle to run the matching on hypergraphs (i.e. when a certain error mechanisms can trigger more than two detectors) has already been exposed. A consequence of that struggle relates to depolarizing noise. Y-type errors cause the activation of both X and Z detectors. Even on a projected lattice, we may still have 4 detectors activated by a Y-type error.

In order to solve this, we must decompose the Y-type error mechanisms into separate X and Z errors. We achieve this with the `decompose_errors=True` option of `stim`’s DEM generation. This adds additional marks in the DEM for error mechanisms that trigger different detector types, effectively separating the two subsets of detectors. An additional step is required for MWPM to process this: each DEM line with both X and Z detectors must be split into two separate DEM lines. With this modification we fully decompose Y-type errors, avoiding hypergraphs that MWPM can’t properly handle. However, this prevents the decoding process from leveraging correlations between X, Y and Z errors.

6.5.3 The Lifting Step

The final stage of the decoder is the “Lifting” process. The three MWPM instances returned a set of edges. When combined, these edges form continuous, closed homological cycles (loops) in the dual graph (G^*) of the Color Code. The objective of the lifting step is to find the exact set of physical qubits (which correspond to the triangular faces in G^*) that are enclosed within these boundaries.

To solve this efficiently, we build a Spanning Tree of the dual graph using a Breadth-First Search (BFS) algorithm. We add a virtual node called the *OUTSIDE*, which represents the external boundary of the lattice and is defined to always have a state of 0 (no error). The spanning tree determines the paths (as in edges that must be crossed) from a common root node (which for simplification we can assume it’s the *OUTSIDE* node) to any other node in the graph.

Because the *OUTSIDE* node is exterior, if the path to a node crosses active edges an even number of times, it must be outside a boundary. If it crosses them an even number of times, it must be inside the boundary.

With this logic, we can identify every physical qubit that is contained inside the loops in $O(N)$ time.

Finally, the algorithm checks the logical observables of the Color Code (from its original DEM) against this recovered set of physical errors. If the physical errors trigger a logical flip, the decoder registers a logical failure.

6.5.4 Threshold Evaluation

We have conducted a full Monte Carlo simulation to evaluate the threshold of the Projection Decoder, with the selected lattice and noise model, at distances from $d = 5$ up to $d = 23$ and the same ranges and number of shots used in prior simulations. Once again, while we’ve run simulations for both depolarizing and X-type noise, we present here the results of the latter. The results of this evaluation are presented in Figure 6.8. A high-resolution zoom plot is provided to further illustrate the threshold calculation. We observe an empirical threshold of $p_{th} \approx 7.50\%$ under a pure bit-flip noise model.

Rather than assuming that the root of the Spanning Tree is the *OUTSIDE* node, we can first determine if the outside node is, in the spanning tree, in the same region as the root. If it is, then a path from the root crossing an even number of times remains in the same region. If the root is in a region different to the *OUTSIDE* node, we have to apply a global flip

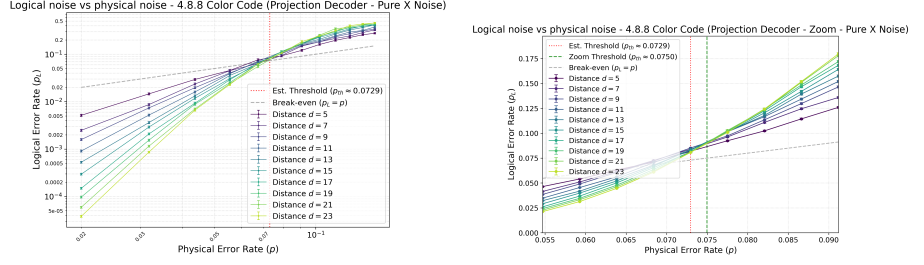


Figure 6.8. Threshold evaluation for the topological Projection Decoder under a pure bit-flip noise model. The standard logical error plot (left) and the high-resolution zoom (right) show the crossing point at $p_{th} \approx 7.50\%$.

Below this crossover point, the logical error curves exhibit a robust exponential suppression across all distances.

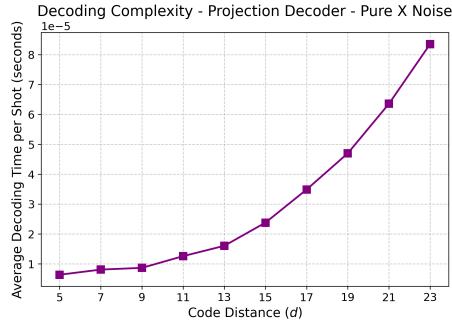


Figure 6.9. Decoding execution time as a function of the distance under a pure X-only noise model.

Published references and preliminary evaluation

Delfosse ([30]) reports a $p_{th} = 8.7\%$ threshold for bit-flip errors on a 6.6.6 lattice, which is better than our results. Our implementation of the projection decoder performs below the BP+OSD decoder presented before (i.e. lower threshold).

6.6 Restriction Decoder

Following the same architectural principles used for the Projection Decoder, we have implemented the Restriction Decoder as a custom `sinter` plugin. The Restriction Decoder introduces a localized approach to

resolving the decoding constraints, relying on only two restricted lattices instead of three projections, and performing a local lifting rather than a global search.

Our implementation is similarly divided into three stages: the Restriction Step, the execution of MWPM on the restricted lattices, and the Local Lifting Step. As before, all graph transformations are compiled and cached per lattice distance to accelerate the Monte Carlo simulations.

6.6.1 The Restriction Step and DEM Generation

The first step translates the 2D Color Code Detector Error Model (DEM) into two independent Surface Code DEMs, corresponding to the restricted lattices $\mathcal{L}_{RG}$ and $\mathcal{L}_{RB}$.

The Restriction Decoder operates by simply deleting vertices of a specific color. To form $\mathcal{L}_{RG}$, we ignore all Blue detectors. Any physical error that previously triggered a Blue detector is modified to only register its Red and Green syndrome. If it triggers exactly one Red and one Green detector, it forms a valid 1D edge in the $\mathcal{L}_{RG}$ lattice. We apply the same logic to form $\mathcal{L}_{RB}$ by dropping the Green detectors.

Since our implementation merges parallel physical qubits into a single virtual edge in the restricted DEM, we keep an internal mapping that associates each physical qubit to its corresponding virtual edge in both $\mathcal{L}_{RG}$ and $\mathcal{L}_{RB}$.

6.6.2 Running MWPM on the restricted lattices

Once the two restricted DEMs are generated, we decode them independently using `PyMatching`.

As with the Projection Decoder, we need the MWPM algorithm to return the exact path of selected edges rather than a single logical observable prediction. We employ the same technique: assigning a unique virtual observable (L_i) to every edge in the restricted graphs. After decoding, the returned boolean array indicates exactly which virtual edges in $\mathcal{L}_{RG}$ and $\mathcal{L}_{RB}$ were selected to form the matching paths connecting the active detectors.

Similarly, for MWPM to run on this restricted lattice, we need to decompose Y-type errors into X and Z errors, as we already did in the Projection Decoder implementation.

6.6.3 The Local Lifting Step

The Local Lifting step reconciles the 1D paths found in $\mathcal{L}_{RG}$ and $\mathcal{L}_{RB}$ to identify the final 2D correction of physical qubits. Theoretically, this is a strictly local operation: for every Red vertex, the algorithm examines the selected incident edges and finds a subset of neighboring physical qubits whose boundaries match those selected edges.

While iterating vertex by vertex is possible, it can be slow in a Python-based simulation environment. To optimize this, we leverage **PyMatching**, natively implemented in C++, to solve the local constraints simultaneously across the entire lattice.

To achieve this, we construct a secondary, artificial DEM (the Lifting Graph) with the following structure:

- **Nodes:** The nodes (DEM detectors) of the Lifting Graph are the virtual edges defined in $\mathcal{L}_{RG}$ and $\mathcal{L}_{RB}$.
- **Edges:** The edges (DEM physical errors) of the Lifting Graph represent the actual physical qubits (the triangles in the Dual Graph).

By running a final MWPM instance on this Lifting Graph, the decoder automatically selects the minimal set of physical qubits that connect all the active restricted edges. This is illustrated in Figure 6.10. While this is a global process, because of how we define it, it's equivalent to the local solution defined by the theoretical algorithm. If multiple sets of physical qubits can be selected, MWPM selects the minimum-weight combination.

6.6.4 Threshold Evaluation

We evaluated the performance of our Restriction Decoder implementation using the same full Monte Carlo methodology, examining lattice distances from $d = 5$ to $d = 23$ under an X-only code-capacity noise model (depolarizing code-capacity noise results have also been produced). The

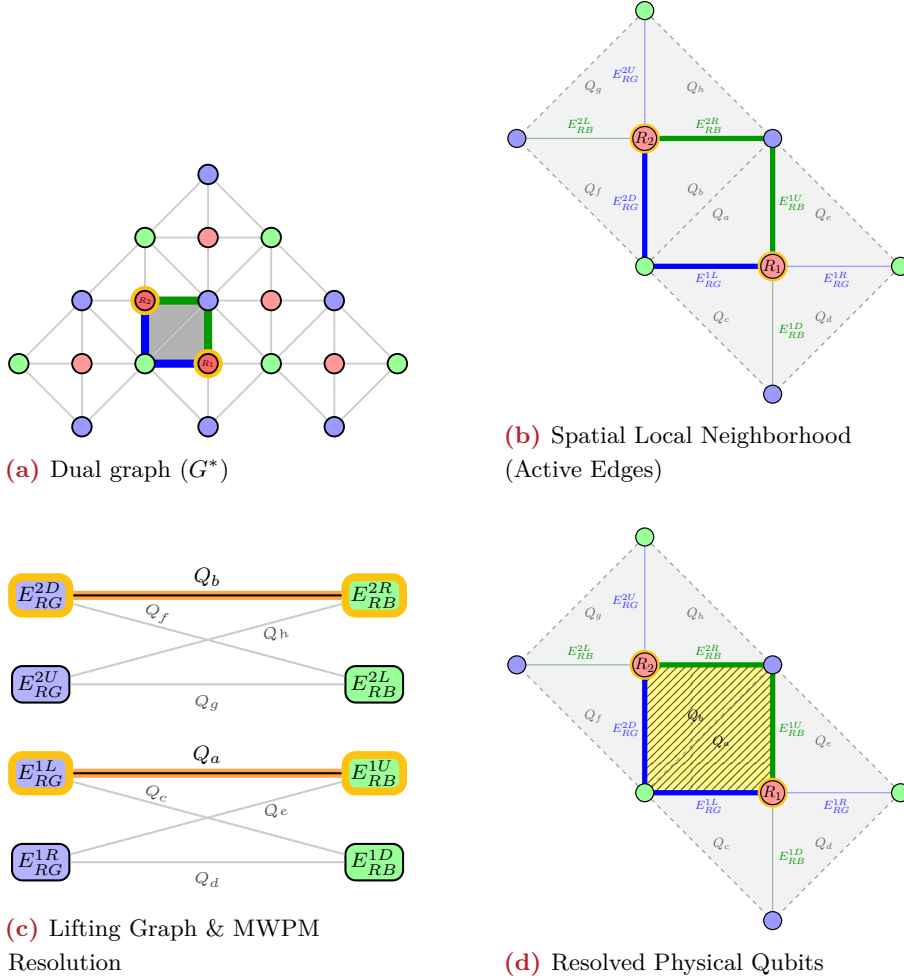


Figure 6.10. Lifting as a MWPM problem: Visual representation of the lifting procedure for a 2 qubit error. (a) In the dual graph, two failing physical errors Q_a and Q_b cause the activation of two red detectors (R_1 and R_2). The MWPM over the two restricted lattices finds the two blue and green paths. (b) Zooming into the local neighborhood containing the active Red vertices. (c) The dual graph is abstracted into the bipartite Lifting Graph. Because physical qubits only connect restricted edges incident to the same Red vertex, the Lifting Graph naturally decomposes into disjoint local subgraphs. MWPM simultaneously solves the matching path for active edges. (d) The selected edges map directly back to the spatial lattice, fully correcting the syndrome.

results of this simulation are shown in Figure 6.11, alongside a high-resolution zoom plot. The Restriction Decoder achieves a threshold of $p_{th} \approx 8.96\%$ for X-type noise.

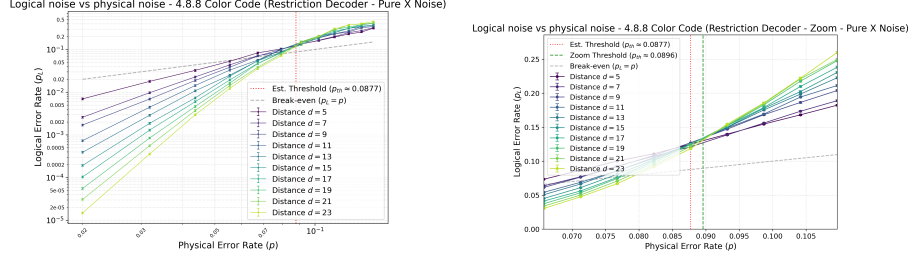


Figure 6.11. Threshold evaluation for the topological Restriction Decoder under a pure bit-flip noise model. The standard logical error plot (left) and the high-resolution zoom (right) show the crossing point at $p_{th} \approx 8.96\%$.

Published references and preliminary evaluation

Kubica and Delfosse [31] report a $p_{th} = 10.2\%$ threshold for bit-flip errors on a 4.8.8 lattice. Our implementation of the restriction decoder has a lower threshold. However, our simulation of the restriction decoder yields better threshold estimations than the BP+OSD and the projection decoders.

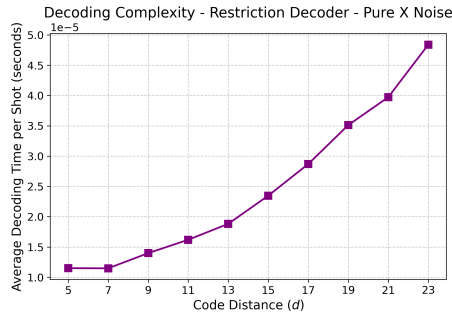


Figure 6.12. Decoding execution time as a function of the distance under a pure X-only noise model.

6.7 Möbius Decoder

The Möbius Decoder provides an efficient approach to decoding 2D color codes by gluing the three projections of the lattice so they can directly be processed by MWPM. In our architecture, we employ the **Chromobius** library [43], an optimized implementation of the Möbius decoder developed by Craig Gidney.

6.7.1 Implementation and Configuration

To integrate **Chromobius** with our simulation pipeline, prepare our **ColorCode** instances to follow the format expected by the decoder. The only requirement to adapt our generated STIM circuits for this decoder is the addition of a fourth coordinate to the metadata of the detectors. Specifically, each detector in the circuit must include its spatial coordinates and a numeric identifier indicating the color and detector type of the face it corresponds to (i.e., 1, 2, 3 for Red, Green and Blue X-type detectors and 3, 4, 5 for the corresponding Z-type ones). This mapping is required for the decoder to interpret the boundaries and connections of the color code lattice.

We incorporated this into our **ColorCode** circuit generation, and actually reused it for the implementation of all the decoders, as shown in Listing 6.1.

Since **Chromobius** is designed to act directly as a Sinter decoder, its execution within our framework requires no additional post-processing layers.

6.7.2 Threshold Evaluation

We evaluated the Möbius Decoder with distances from $d = 5$ to $d = 23$ under an X-only code-capacity noise model (depolarizing code-capacity noise results have also been produced and are available in the repository). The simulation parameters follow the general methodology described in Section 6.3. Figure 6.13 presents the logical error curves and a high-resolution zoom plot, exhibiting a threshold of $p_{th} \approx 9.50\%$ for X-type noise.

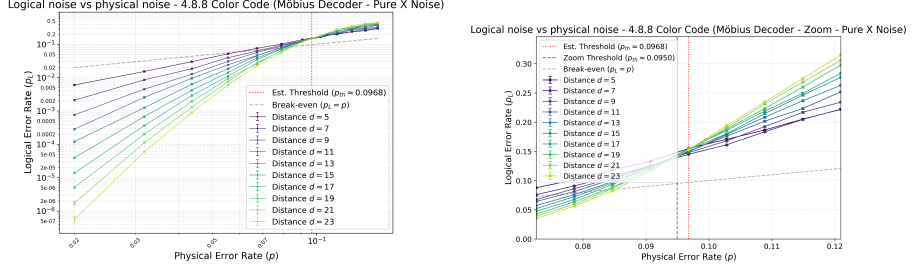


Figure 6.13. Threshold evaluation for the Möbius Decoder under a pure bit-flip noise model. The standard logical error plot (left) and the high-resolution zoom (right) show the crossing point at $p_{th} \approx 9.50\%$.

Published references and preliminary evaluation

Sahay and Brown [33] report a $p_{th} = 9\%$ threshold for bit-flip errors on a 6.6.6 lattice, which is in line with our results for the 4.8.8 lattice.

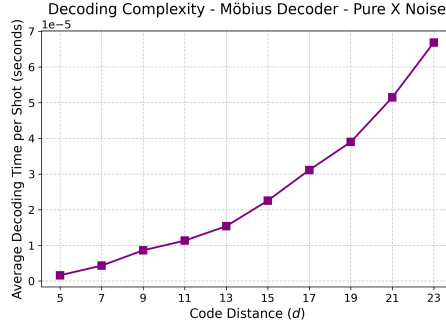


Figure 6.14. Decoding execution time as a function of the distance under a pure X-only noise model.

6.8 Concatenated MWPM Decoder

The Concatenated Minimum Weight Perfect Matching (MWPM) decoder is a heuristic approach designed to decode color codes by projecting the syndrome onto multiple restricted lattices and resolving them using standard MWPM algorithms. For this thesis, we utilized the `concatmatching` library [44], an existing Python implementation of this decoder.

6.8.1 Implementation and Configuration

Unlike other decoders such as **Chromobius** or **PyMatching**, which can directly interface with Stim’s Detector Error Model (DEM) and the Sinter framework, the **concatmatching** library requires the code’s parity check matrix (H) and the individual error probabilities for each fault mechanism (priors). To integrate this external library into our simulation pipeline, we developed a dedicated Sinter wrapper (**sinter_concatenated_decoder.py**).

The main responsibility of this wrapper is to translate the instructions from the Stim DEM into the sparse check matrices expected by the decoder. There are built-in methods in **stim**’s DEM to generate the check matrices, but they presented a significant technical challenge regarding Y errors that trigger 6 three X detectors and three Z detectors.

While **stim** provides a **decompose_errors=True** flag to automatically split independent fault mechanisms, this algorithm was not sufficient, so we had to manually split the DEM error mechanisms corresponding to Y errors into two independent X and Z error mechanisms.

The authors of the **concatmatching** library also published a **stim** wrapper [34] [45]. While this library is designed for their own implementation of the **stim** circuit for the color code, it served as a reference to integrate **concatmatching** in our framework.

6.8.2 Threshold Evaluation

We evaluated the Concatenated MWPM Decoder with lattice distances ranging from $d = 5$ to $d = 23$ under an X -only code-capacity noise model (depolarizing code-capacity noise results have also been produced and are available in the repository). The simulation parameters follow our general Sinter methodology, collecting statistics up to 40 million shots per data point.

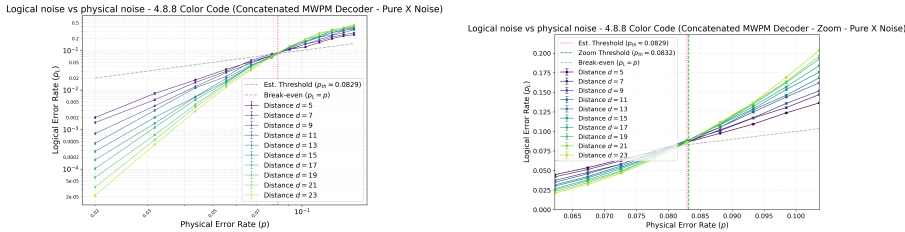


Figure 6.15. Threshold evaluation for the Concatenated MWPM Decoder under a pure bit-flip noise model. The standard logical error plot (left) and the high-resolution zoom (right) show the crossing point at $p_{th} \approx 8.32\%$.

The results of this simulation are shown in Figure 6.15, alongside a high-resolution zoom plot. The Concatenated MWPM decoder achieves an estimated threshold of $p_{th} \approx 8.32\%$ for X-type noise.

Published references and preliminary evaluation

Reported threshold for the concatenated MWPM decoder is $p_{th} \approx 8.2\%$ for a 6.6.6 lattice, in line with our estimated threshold of 8.32% for the 4.8.8 lattice. This threshold is below the threshold of other simulated algorithms.

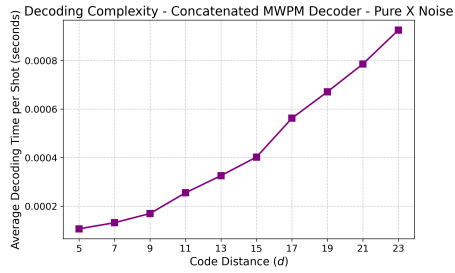


Figure 6.16. Decoding execution time as a function of the code distance under a pure X-only noise model.

6.9 Correlated matching decoder

The Correlated Matching Decoder builds upon the foundation of the Restriction Decoder by introducing correlations between the decoding graphs to improve the overall accuracy.

6.9.1 Implementation and Configuration

In their original proposal [36], the authors describe a process of mapping the color code syndrome to a surface code subject to correlated X and Y-type errors. However, for our implementation within the `sinter` simulation pipeline, we have opted for an alternative approach.

Rather than explicitly mapping to a surface code with correlated errors, we execute the exact same lifting procedure that we implemented for the Restriction Decoder (detailed in Section 6.6). The key difference is that we apply the correlation logic to the algorithmic resolution before the

lifting. We consider this approach to be entirely valid and theoretically consistent with the original proposal because the authors themselves use the Restriction Decoder as their structural baseline, focusing their core contribution on the performance improvement achieved by correlating the two projections. By reusing our optimized local lifting procedure and injecting the required correlations, we accurately capture the algorithmic advantage of the decoder.

6.9.2 Threshold Evaluation

We evaluated the Correlated Matching Decoder with lattice distances ranging from $d = 5$ to $d = 23$ under an X-only code-capacity noise model (depolarizing code-capacity noise results have also been produced and are available in the repository). The simulation parameters follow our general Sinter methodology.

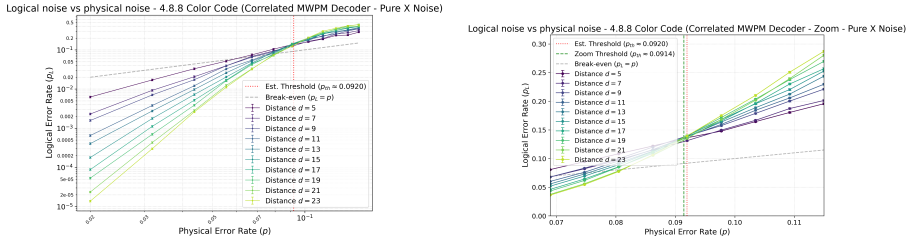


Figure 6.17. Threshold evaluation for the Correlated Matching Decoder under a pure bit-flip noise model. The standard logical error plot (left) and the high-resolution zoom (right) show the crossing point at $p_{th} \approx 9.14\%$.

The results of this simulation are shown in Figure 6.17, alongside a high-resolution zoom plot. The Correlated Matching decoder achieves an empirical threshold of $p_{th} \approx 9.14\%$ for X-type noise.

Published references and preliminary evaluation

The original publication of the Correlated Matching decoder [36] evaluates its performance for a square-boundary 4.8.8 code under independent bit-flip noise, obtaining a threshold of $p_{th} \approx 10.38\%$. Unfortunately, the square-boundary code can't be directly compared with our triangular one. Nevertheless, by comparing our empirical threshold for X-type noise (9.14%) against our results for the standard Restriction Decoder (8.96%),

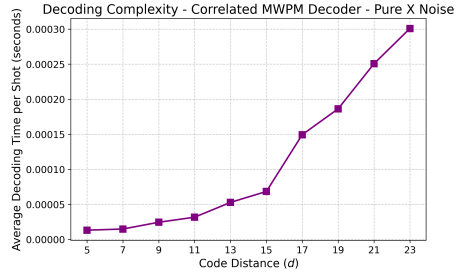


Figure 6.18. Decoding execution time as a function of the code distance under a pure X-only noise model.

we observe the theoretical improvement provided by the correlation logic, effectively increasing the error correction capability of the algorithm. This improvement is in-line with the improvement observed by the authors when comparing to the Restricted MWPM (10.2% vs 10.38%).

Comparative analysis and discussion

After detailing the theoretical foundations of the 2D Color Code and implementing a comprehensive simulation pipeline, we now present the results of our empirical evaluation. In Chapter 6 we have already anticipated some comparisons between the different decoders, exclusively in terms of threshold. This has allowed us to validate that the implementation was in line with the theoretical expectations and the existing published references. Now, we will conduct a deeper and wider comparison.

We have simulated six different decoding algorithms under identical conditions: a 4.8.8 lattice geometry subjected to a code-capacity bit-flip noise model. The captured data allows measuring the four key metrics that will drive the comparative analysis:

- **Determination of error thresholds (p_{th}):** Identifying the crossing point in the logical error curves where increasing the code distance begins to suppress errors rather than amplifying them.
- **Logical failure rate (p_L) scaling as a function of physical**

error rate (p) and code distance (d): Analyzing the curves to evaluate the decoder’s raw accuracy.

- **Sub-threshold scaling analysis and theoretical suppression limits:** Evaluating how steeply the logical error drops when operating below the threshold.
- **Computational complexity and empirical decoding time (Accuracy vs. Speed trade-off):** Analyzing the empirical decoding time to determine if the decoder’s complexity is practically scalable.

7.1 Determination of error thresholds (p_{th}) for 4.8.8 color codes

The error threshold represents the physical error rate below which the quantum error correction code becomes useful. To determine the threshold for each decoder, we evaluate the logical failure rate across multiple code distances ($d = 5$ to $d = 23$) and plot the results as a function of the physical error rate.

As discussed in Chapter 6, the empirical threshold is estimated by finding the crossing point of the logical error curves for different distances. We compute the intersection of all pairs of curves and calculate the median of these crossing points.

Figure 7.1 and Table 7.1 show the calculated thresholds for all the evaluated decoders. While there are differences in their performing, the results suggest that the threshold will not be the main parameter to evaluate the capacity of a decoder, as they are all within a similar range.

Decoder	Bit-flip Threshold (p_{th}^X)	Depolarizing Threshold (p_{th}^{dep})
BP+OSD	0.0836	0.1351
Projection	0.0750	0.1123
Restriction	0.0896	0.1340
Möbius	0.0950	0.1423
Concat	0.0832	0.1250
Correlated	0.0914	0.1368

Table 7.1. Code capacity thresholds for X-type and depolarizing noise.

Seeing this data, we conclude that the capacity of the code to mitigate noise below the threshold (and how that suppression scales with the distance), as well as the feasibility of running the decodification process

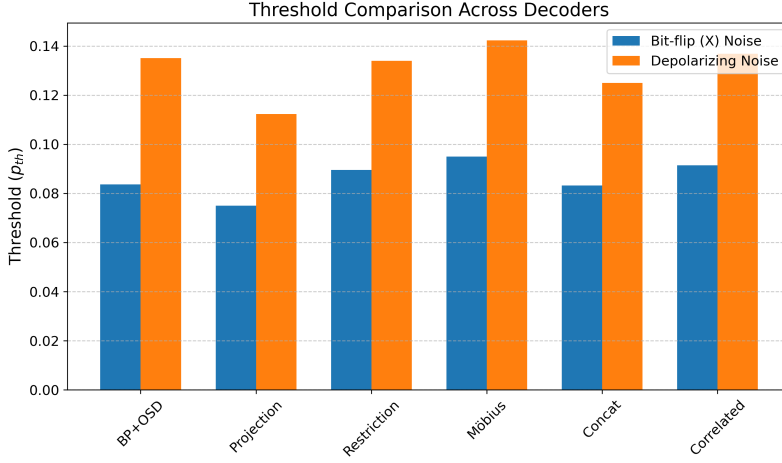


Figure 7.1. Empirical error thresholds (p_{th}) for the six evaluated decoders under a pure bit-flip code-capacity noise model on a 4.8.8 color code.

within one cycle of the quantum computer, will be the determining factors to evaluate decoders.

7.2 Logical failure rate (p_L) scaling as a function of physical error rate (p) and code distance (d)

While the threshold determines the absolute limit of the decoder, quantum computers will be useful when they can operate well below this threshold. It is crucial to evaluate how effectively a decoder suppresses logical errors in this operational regime.

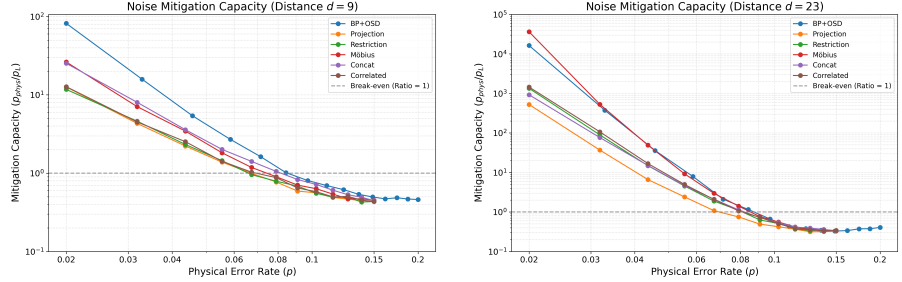
To visualize this, we introduce the concept of the mitigation ratio, calculated as p_{phys}/p_L . A value greater than 1 means the decoder is actively reducing the error compared to a single physical unencoded qubit. A higher ratio indicates a better performance.

Figure 7.2 displays the mitigation ratio across various physical error rates for two fixed distances, $d = 9$ and $d = 23$. Additionally, Table 7.2 summarizes these mitigation ratios for all decoders at a fixed physical error rate of $p = 0.02$ (2%), which is below the threshold of all evaluated

algorithms and matches the lowest physical error rate we have been able to simulate.

Decoder	Distance (d)	p_{phys}	Mitigation Ratio Λ
BP+OSD	9	2.00×10^{-2}	8.18×10^1
BP+OSD	23	2.00×10^{-2}	1.63×10^4
Projection	9	2.00×10^{-2}	1.26×10^1
Projection	23	2.00×10^{-2}	5.25×10^2
Restriction	9	2.00×10^{-2}	1.17×10^1
Restriction	23	2.00×10^{-2}	1.35×10^3
Möbius	9	2.00×10^{-2}	2.63×10^1
Möbius	23	2.00×10^{-2}	3.64×10^4
Concat	9	2.00×10^{-2}	2.53×10^1
Concat	23	2.00×10^{-2}	9.25×10^2
Correlated	9	2.00×10^{-2}	1.27×10^1
Correlated	23	2.00×10^{-2}	1.45×10^3

Table 7.2. Noise mitigation capacity ($\Lambda = p_{phys}/p_L$) at a fixed physical error rate of $p = 0.02$ (2%).



(a) Distance $d = 9$

(b) Distance $d = 23$

Figure 7.2. Mitigation ratio (p_{phys}/p_L) as a function of the physical error rate for code distances $d = 9$ and $d = 23$. The logarithmic Y-axes highlight the exponential improvement as physical error decreases.

The visualization reveals a critical scaling property. At a smaller distance ($d = 9$), the BP+OSD decoder might appear a priori to be the most capable at mitigating noise in the low physical error regime, achieving a mitigation ratio of $\sim 8.2 \times 10^1$ compared to Möbius's $\sim 2.6 \times 10^1$. However, when we increase the topological protection to $d = 23$, the Möbius decoder overtakes it and scales significantly better, achieving

a mitigation ratio of $\sim 3.6 \times 10^4$ compared to BP+OSD's $\sim 1.6 \times 10^4$. Furthermore, the Projection decoder at the same distance achieves a mitigation ratio of just $\sim 5.2 \times 10^2$. This massive difference highlights that when operating below threshold and scaling the code, decoders that utilize the full topological structure without heuristic limitations (like Möbius) suppress errors exponentially faster.

7.3 Sub-threshold scaling analysis and theoretical suppression limits

To predict the resources required to build a fault-tolerant quantum computer, and given that we are unable to run reliable simulations with physical noise levels below $p_{phys} = 2\%$, we must mathematically model how the logical error drops as we increase the size of the code. We modeled the logical error rate using an exponential decay function defined as $\log_{10}(p_L) = A \cdot d + C$.

Using SciPy's `curve_fit` function, we performed a 2D surface regression to fit the empirical data across both variables simultaneously (code distance d and physical error rate p). The full surface model equation is $\log_{10}(p_L) = c_0 + c_1 d + c_2 \log_{10} p + c_3 d \log_{10} p$. The parameters resulting from this surface regression for all decoders, as well as their R^2 coefficients that indicate the goodness of the fit, are documented in Table 7.3.

To visualize this fit, Figure 7.3 shows a one-dimensional “slice” of this 2D mathematical surface. By fixing the physical error rate at $p = 0.02$, the complex surface equation simplifies into a straight 1D line ($\log_{10}(p_L) = A \cdot d + C$), which we plot alongside the empirical data points for that specific noise level.

The regressions confirm a strictly exponential suppression of errors. The slope of these lines represents the suppression efficiency of the algorithm. Möbius displays the steepest slope, meaning it requires adding fewer physical qubits to gain an extra order of magnitude in logical protection.

Using these mathematical fits, we extrapolated the data to answer a practical engineering question: What code distance is required to achieve a target logical error rate? We defined a strict target logical error rate of $p_L = 10^{-12}$ (a standard requirement for large-scale algorithm execution),

In a log-linear plot (where the Y-axis is in logarithmic scale), this exponential decay manifests as a straight line. The parameter A represents the slope, indicating how much the error is suppressed per unit of code distance, while the constant C captures the baseline error overhead independent of the distance, shifting the line vertically to its correct height.

Decoder	c_0	c_1	c_2	c_3	R^2
BP+OSD	0.5711	0.3373	1.4877	0.3004	0.9952
Projection	0.0431	0.2973	1.0532	0.2444	0.9985
Restriction	0.0106	0.2780	0.8954	0.2482	0.9988
Möbius	-0.9666	0.4388	0.0927	0.3867	0.9985
Concat	0.7466	0.2222	1.6693	0.1965	0.9988
Correlated	0.0946	0.2718	0.9540	0.2459	0.9986

Table 7.3. Fitted parameters for the 2D sub-threshold error scaling model. Equation: $\log_{10} p_L = c_0 + c_1 d + c_2 \log_{10} p + c_3 d \log_{10} p$

and evaluated the required distance under two different baseline physical error rates: $p = 10^{-3}$ and $p = 10^{-4}$.

$N = \frac{d^2+2d-1}{2}$ calculates exclusively the number of *data* qubits in the 4.8.8 triangular color code lattice. The total physical hardware footprint will be at least double, as it must also include the ancilla (measurement) qubits interspersed in the lattice to extract the error syndromes.

As shown in Figure 7.4, the choice of decoder drastically affects the hardware requirements. To achieve $p_L = 10^{-12}$ with a base physical error rate of 10^{-3} , the Möbius decoder only requires a code distance of $d = 15$. In stark contrast, the Concatenated MWPM and the Projection decoders require $d = 23$ and $d = 21$, respectively. Given that the number of physical qubits in a 2D color code scales quadratically with the distance ($N = \frac{d^2+2d-1}{2}$), moving from $d = 15$ (127 qubits) to $d = 23$ (287 qubits) more than doubles the size of the required quantum processor, highlighting the immense value of a highly accurate decoder. Furthermore, if the hardware quality improves to a baseline physical error rate of 10^{-4} , the required distances drop drastically across all decoders, with BP+OSD and Möbius requiring only $d = 9$ (49 qubits) and $d = 11$ (71 qubits), respectively, to achieve the exact same 10^{-12} logical target.

7.4 Computational complexity and empirical decoding time

In a log-log plot, a pure polynomial scaling ($A \cdot N^k$) would appear as a straight line. However, we introduced the constant C to capture the fixed software overhead that dominates at small distances, creating the curved shapes seen in the plot.

The final dimension of our comparative analysis focuses on execution time. A decoder can have the necessary accuracy for a given problem, but if it is too slow, errors will accumulate in the quantum processor while the classical computer tries to calculate the correction.

Using, once again, SciPy’s `curve_fit` function, we analyzed the average execution time for each decoder. The empirical data was fitted to a

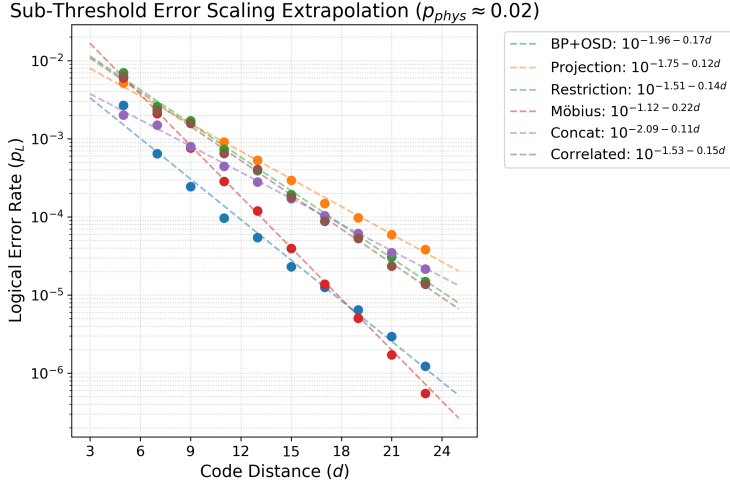


Figure 7.3. Exponential suppression of the logical error as a function of code distance, evaluated at a fixed physical error rate of $p = 0.02$. The solid lines represent the mathematical fit.

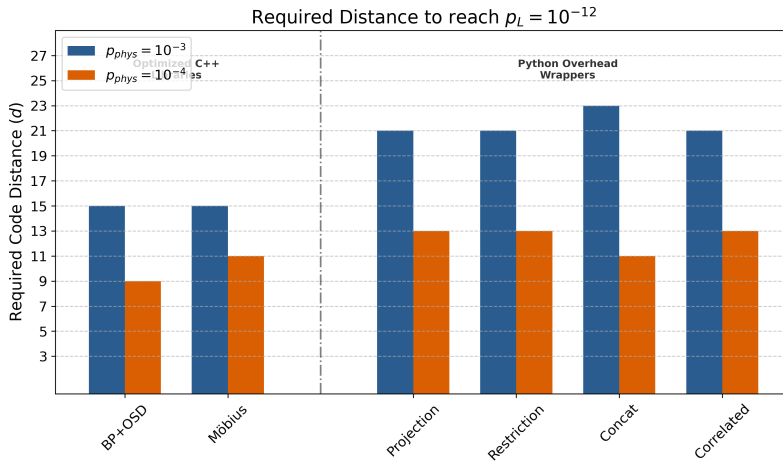


Figure 7.4. Extrapolated code distance required to achieve a target logical error rate of $p_L = 10^{-12}$, evaluated under two different physical error rates ($p = 10^{-3}$ and $p = 10^{-4}$), based on the 2D regression models. The Y-axis shows exclusively valid odd distances.

complexity model $T(N) = C + A \cdot N^k$, where N is the number of qubits and k is the scaling exponent. The resulting parameters are summarized in Table 7.4.

Decoder	C (s)	A	k	R^2
BP+OSD	1.79×10^{-5}	7.43×10^{-8}	1.5343	0.9997
Projection	5.44×10^{-6}	7.24×10^{-9}	1.6290	0.9945
Restriction	1.01×10^{-5}	2.10×10^{-8}	1.3283	0.9966
Möbius	1.00×10^{-9}	5.28×10^{-8}	1.2567	0.9934
Concat	7.03×10^{-5}	1.15×10^{-6}	1.1771	0.9965
Correlated	1.11×10^{-5}	7.14×10^{-9}	1.8960	0.9908

Table 7.4. Fitted parameters for the empirical time complexity. Model: $T(N) = C + A \cdot N^k$

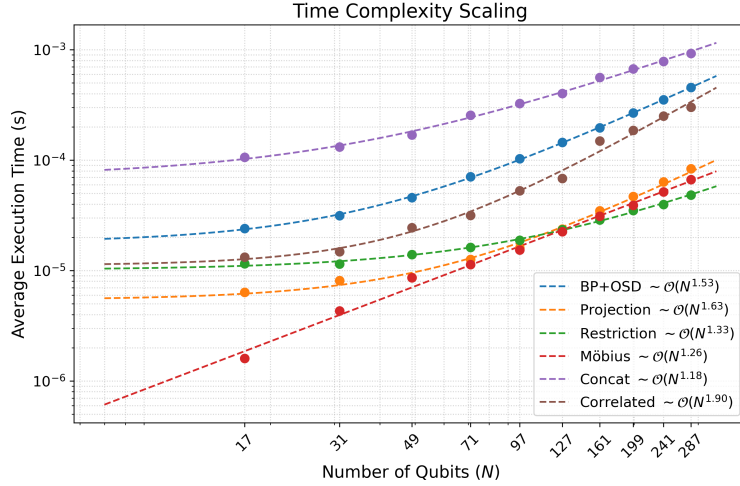


Figure 7.5. Average execution time per shot as a function of the number of qubits, in a logarithmic scale. The lines represent the non-linear complexity fit.

As seen in Figure 7.5, all decoders exhibit a polynomial scaling curve, although their absolute execution times vary drastically.

To provide a realistic hardware context, we tried to extrapolate these times to the required distances calculated in the previous section. Figure 7.6 shows the estimated decoding time required to reach the target logical error rates. We have assumed a 100x hardware acceleration factor to simulate the transition from our Python-based simulation

The 100x speedup assumption is based on the known existence of under micro-second implementations of MWPM[46] and 11.5ns execution times for Union-Find on FPGAs[47].

environment to a dedicated FPGA implementation. While this is possibly a valid assumption for most of the decoders, the Möbius and BP+OSD decoders might require a lower acceleration factor due to their native C++ implementation.

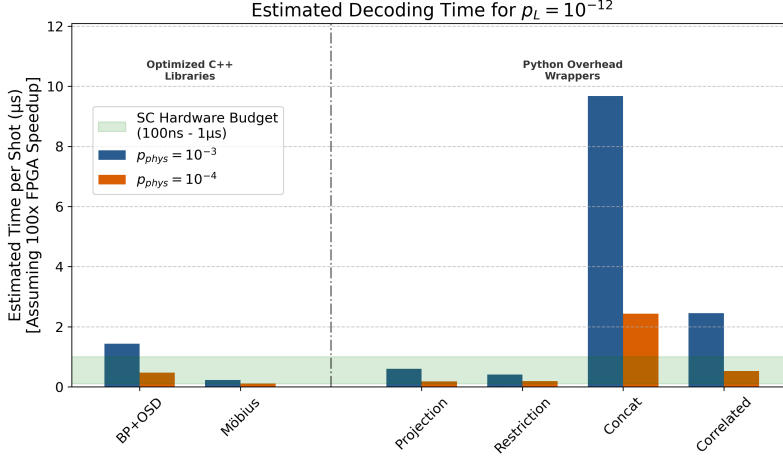


Figure 7.6. Estimated decoding time per shot to reach target logical error rates, assuming a 100x FPGA speedup. The green shaded area represents the target time budget for a superconducting quantum cycle (100 ns to 1 μ s). The decoders are visually separated between natively optimized C++ libraries and Python-wrapped implementations.

The most critical element in Figure 7.6 is the green band at the bottom, which represents the typical time budget for a single cycle in a modern superconducting quantum processor (between 100ns and 1 μ s [48]). If the decoder takes longer than this budget, it will not be able to decode real time, causing error accumulation or forcing the quantum computer to idle, increasing the chances of errors.

We observe a significant gap between the optimized C++ libraries and the custom Python implementations. The Python wrappers introduce a constant computational overhead per shot that heavily penalizes their absolute times. This explains the good performance of BP+OSD when compared with other python implementations like the Correlated or Concatenated MWPM decoders.

It is worth noting the slowness of the Concatenated MWPM decoder.

While this decoder is expected to come with an overhead (as it implements twice as much MWPM matchings as others), the running time per shot is simply at a different scale. We believe that the actual implementation of the chosen library suffers some serious efficiency issue, perhaps executing certain calculations during the decode phase that could be cached if they only depend in the DEM.

Only a few decoders are realistic candidates to perform real-time error correction (note that this estimation is for a simplified code-capacity error model, and in a realistic noise model, significantly larger distances would be required).

This analysis validates a fundamental conclusion: while achieving high accuracy (effective error suppression, resulting in lower required code distances) is critical to minimizing the number of physical qubits required and the complexity of the decoding, computational speed is equally paramount. To build a useful fault-tolerant quantum computer, the industry will require decoders that are both highly accurate and inherently parallelizable or capable of being executed entirely in specialized low-level hardware.

7.5 Final evaluation

While each decoder has unique characteristics, and the “best” decoder may depend on the specific application, a general assesment can be made. All decoders provide thresholds in the 7% – 9% range, making the threshold itself not the most distinguishing feature.

Because of its unique error mitigation and decoding time scaling properties, the Möbius decoder stands out as a very promising candidate for fault-tolerant quantum computing, although it seems to require a significant distance to deliver its full potential.

While Concatenated MWPM and Correlated decoders are meant to produce improvements over the Projection and Restriction decoders, and these are visible as the code scale, the difference is not enough to reduce significantly the required code distance, but the decoding times suffer a penalty.

Therefore, aside the Möbius decoder, only Restriction and Projection decoders would be good candidates to implement real-time error correction.

Potential improvements and future work

We have concluded our systematic review of decoders for 2D color codes and are ready to evaluate potential improvements to the existing decoders and new research directions. We also want to highlight some open questions that remained unanswered during the scope of this thesis and may be interesting for future research.

8.1 Potential improvements

While the literature has focused in the threshold at a time when no physical realization of a quantum computing had proven to be able to run below the threshold, now that this milestone has been reached [48], it's crucial to shift the focus to the decoders performance in the sub-threshold regime and it's ability to be executed efficiently.

The existing solutions rely on a projection mechanism to reduce the dimension of the color code to a surface code equivalent. This has proven to be a valid approach, yet exploring new mechanisms to directly resolve weighted matching in an hypergraph could open the door to a new family

of decoders with better performance.

Similarly, exploring new methods for an algebraic decoding (such as BP+OSD) that can run more efficiently could also bring new opportunities to the field.

We can conclude that the decoding of 2D color codes is still an open problem, and developments in this field would allow a practical implementation of color codes in future quantum computers, a scenario that would be highly desirable due to their unique transversality properties.

8.2 Future work

To ensure this research remained constrained and achievable within the limited timeframe of a Master's Thesis, the scope of the project was significantly narrowed. Consequently, many complex and interesting avenues were intentionally left outside the boundaries of this work. Addressing these uncharted areas presents exciting opportunities for future research:

- **Alternative lattice geometries:** This thesis focused exclusively on the 4.8.8 triangular color code. Future work should analyze the performance of these decoders on other geometric layouts, particularly the 6.6.6 hexagonal lattice and color codes with square boundaries. This would also address one of the challenges we have faced: validating the empirical results presented by other authors. The studied papers use a variety of lattices and, while their results are directional to evaluate our implementation of the different decoders, a direct comparison is not possible.
- **Correlated and circuit-level noise models:** While we implemented a depolarizing code-capacity model, our decoders effectively decompose the errors into independent bit-flip (X) and phase-flip (Z) errors. Future implementations should explore decoders capable of leveraging *a priori* information about correlated noise, or simply about error mechanisms with different probabilities (as the projections we use in most of our decoder implementations overwrite the DEM probabilities and assume uniform probability of all error mechanisms). Furthermore, extending the simulations

to a full circuit-level noise model would allow researchers to analyze the impact of chained errors caused by faulty two-qubit gates during syndrome extraction. This would provide a more realistic benchmark of decoder performance, which would be more directly applicable to the real noise characteristics of quantum hardware.

- **Software optimization for fair benchmarking:** While our study analyzed the execution time scaling, the absolute time differences were heavily influenced by the programming language. Optimizing the custom Python-based decoders by rewriting them as efficient C++ libraries would guarantee optimal performance. This would eliminate the software overhead and allow for a fair execution time comparison against the natively optimized decoders like Möbius and BP+OSD. It would also offer comparison points to determine the practicality of a given decoder with respect to the cycle durations of real quantum hardware.
- **Alternative decoding sub-routines:** The geometric decoders analyzed in this work (Projection, Restriction, Concatenated, and Correlated) rely exclusively on the Minimum Weight Perfect Matching (MWPM) algorithm to pair defects. Future research should evaluate the impact of replacing the underlying MWPM engine with faster, near-linear time algorithms such as Union-Find. Exploring these combinations could significantly reduce the execution time scaling while maintaining highly competitive error mitigation ratios.
- **Alternative mapping mechanisms:** To lift the projections back to physical qubits in the lattice, we have relied on either local or global lifting procedures. The Correlated decoder [36] propose a different approach mapping the Color Code under bit-flip error to a surface code model under depolarizing noise. While they strictly propose this for square-boundary color codes, it would be worth exploring how this could be applied to triangular color codes.

Finally, as this thesis aims to serve as introductory material for new researchers entering the Quantum Error Correction field, a revision of the work may prove useful to structure it specifically for that purpose with the goal of publishing it in a specialized journal that accepts survey and tutorial papers.

Contents

1	Introduction: Classical error correction and foundations of quantum error correction	1
1.1	Project Goals and Scope	3
1.2	Classical error correction	4
1.3	Shor code	6
1.4	The Stabilizer Formalism	12
	Stabilizer Operators	13
	The Stabilizer Group and Generators	14
	Logical Operators and Code Construction	15
	Standard $[[n, k, d]]$ Notation	16
1.5	The Threshold Theorem	17
2	State of the art of QEC codes: Surface codes, color codes and QLDPC codes	19
2.1	Introduction	19
	2.1.1 About terminology: Topological vs. LDPC Codes	20
2.2	Transversality and the Eastin-Knill Theorem	20
2.3	Surface Codes	22
	2.3.1 From 1D Repetition to 2D Topologies	22
	2.3.2 Origins: The Toric Code	23
	2.3.3 Observables, Capacity, and Code Distance	23
	2.3.4 Planar Implementation and Rotated Surface Codes	24
	2.3.5 Advantages and Disadvantages	25

2.4	Color Codes	26
2.4.1	Origins and Topologies	26
2.4.2	2D Color Code Configurations	27
2.4.3	Topological Observables: Colored Strings and Nets	28
2.4.4	Advantages: Transversality and 3D Topologies	29
2.4.5	Disadvantages and Processor Mapping	30
2.5	Non-topological QLDPC Codes	31
2.5.1	Bivariate Bicycle Codes (BBC)	32
3	Modeling quantum error	35
3.1	Types of quantum noise	35
3.1.1	Depolarizing noise	36
3.1.2	Biased noise	36
	Bit-flip noise	36
3.1.3	Correlated noise	36
3.2	Error Models	37
3.2.1	Code-capacity noise	37
3.2.2	Phenomenological noise	37
3.2.3	Circuit-level noise	38
3.3	Relationship between the models	38
4	QEC decoders for color codes	41
4.1	Minimum Weight Perfect Matching (MWPM)	42
4.1.1	Edmonds' Blossom algorithm	43
4.2	Union-find	45
4.3	BP+OSD	47
4.4	Projection Decoder	51
4.4.1	The Projection Step	52
4.4.2	The Merge Step	53
4.5	The Restriction Decoder	57
4.5.1	The Restriction Step	57
4.5.2	The Local Lifting Step	58
4.6	Möbius Decoder	60
4.6.1	Trivial Correction	64
4.6.2	Unified Lattice and MWPM	64
4.6.3	Homology Evaluation	65
4.7	Concatenated MWPM	65

4.7.1	Step 1: First-round MWPM on the Restricted Lattice	67
4.7.2	Step 2: Second-round MWPM on the Monochromatic Lattice	67
4.7.3	Step 3: Repetition and Final Selection	68
4.8	Correlated matching decoder	69
5	Implementation and simulation of QEC codes: Framework setup and baseline models	73
5.1	Simulating QEC codes and decoders with <code>stim</code>	74
5.1.1	Simulating the repetition code under code-capacity noise	74
5.1.2	Decoders: From Minimum Weight Perfect Matching to Custom Logic	76
5.1.3	Introducing circuit-level noise	78
5.1.4	Implementing the 9-qubit Shor code	80
5.1.5	Visualizing the Threshold Theorem	85
5.2	Beyond <code>stim</code> : Experimental setup for implementation and simulation	86
5.2.1	Sampling technologies: <code>sinter</code>	87
5.2.2	High level circuit design for quantum error correction: <code>cirq</code> and <code>stimcirq</code>	88
5.3	Automated noise injection: Extracting error models from Qiskit Fake Providers	91
5.3.1	Extracting physical parameters	91
5.3.2	Automating noise injection with <code>cirq.NoiseModel</code>	92
6	Simulating 2D triangular color codes with a 4.8.8 lattice geometry and a code-capacity noise model	95
6.1	Syndrome Graph Construction and Decoding	95
6.1.1	The Detector Error Model (DEM)	95
6.1.2	Decoding the DEM	96
6.2	Implementing a 2D color code	97
6.2.1	Lattice Construction	97
6.2.2	Circuit Generation	99
6.2.3	X and depolarizing noise types	99
6.2.4	The color and detector type dimension	100

6.3	Simulation Architecture and Evaluation Metrics	101
6.4	BP+OSD	104
6.4.1	Parameter Optimization	105
	BP Update Rule Comparison	105
	OSD Method Comparison	106
	OSD Search Order Analysis	106
6.4.2	Threshold Evaluation	107
	Published references and preliminary evaluation	108
6.5	Projection decoder	108
6.5.1	The Projection Step and DEM Generation	109
6.5.2	Running MWPM on the projected surface codes	110
	MWPM under depolarizing noise	110
6.5.3	The Lifting Step	111
6.5.4	Threshold Evaluation	111
	Published references and preliminary evaluation	112
6.6	Restriction Decoder	112
6.6.1	The Restriction Step and DEM Generation	113
6.6.2	Running MWPM on the restricted lattices	113
6.6.3	The Local Lifting Step	114
6.6.4	Threshold Evaluation	114
	Published references and preliminary evaluation	116
6.7	Möbius Decoder	117
6.7.1	Implementation and Configuration	117
6.7.2	Threshold Evaluation	117
	Published references and preliminary evaluation	118
6.8	Concatenated MWPM Decoder	118
6.8.1	Implementation and Configuration	119
6.8.2	Threshold Evaluation	119
	Published references and preliminary evaluation	120
6.9	Correlated matching decoder	120
6.9.1	Implementation and Configuration	120
6.9.2	Threshold Evaluation	121
	Published references and preliminary evaluation	121
7	Comparative analysis and discussion	123
7.1	Determination of error thresholds (p_{th}) for 4.8.8 color codes	124

7.2	Logical failure rate (p_L) scaling as a function of physical error rate (p) and code distance (d)	125
7.3	Sub-threshold scaling analysis and theoretical suppression limits	127
7.4	Computational complexity and empirical decoding time .	128
7.5	Final evaluation	132
8	Potential improvements and future work	135
8.1	Potential improvements	135
8.2	Future work	136
List of Figures		V
Bibliography		XI

List of Figures

1.1	Mechanism of a classical 3-bit repetition code.	5
1.2	A parity check bit appended to a 3-bit message.	5
1.3	Geometric representation of a qubit on the Bloch sphere.	7
1.4	Encoding circuit for the bit-flip code.	7
1.5	Syndrome measurement for bit-flip code.	8
1.6	Encoding circuit for the 3-qubit phase-flip code.	9
1.7	Full encoding circuit for the 9-qubit Shor code.	10
2.1	Evolution and representations of the surface code.	25
2.2	The three valid 2D configurations for topological color codes.	27
2.3	Logical observables in a toric color code.	28
2.4	Planar topology of a 4.8.8 lattice color code.	31
2.5	Construction of Bivariate Bicycle Codes.	34
4.1	A visual representation of the Minimum Weight Perfect Matching problem.	43
4.2	Visual representation of the Union-find decoder cluster-growth strategy.	47
4.3	Visual representation of the Belief Propagation (BP) algorithm numerical updates.	49
4.4	Syndrome hypergraph for local patch of a color code and its three projections.	54
4.5	Projection method on the dual graph	55

4.6	Projection decoding on the dual graph	56
4.7	Restriction decoding on the dual graph	59
4.8	Color code representations for a $d = 7$ 4.8.8 lattice	60
4.9	Successful decoding of Projection and Restriction decoders in the bulk.	61
4.10	Failure of the Projection and Restriction decoders at the lattice boundary.	62
4.11	Möbius strip construction from 2D color code restrictions	63
4.12	Graph structures used in the Concatenated MWPM decoder.	66
4.13	Decoding flow of the Concatenated MWPM Red sub-routine.	68
4.14	Restriction and Correlated Decoding Step-by-step	70
5.1	Simulation 3: Comparison of the results of simulations 1 and 2.	79
5.2	Simulation 4: Impact of circuit-level and phenomenological noise.	80
5.3	Simulation 5: Effect of increasing the distance under a complex noise model.	81
5.4	Simulation 6: Performance analysis of the Shor 9-qubit code.	82
5.5	Simulation 7: Visualizing the Threshold Theorem.	86
6.1	Evolution of the 4.8.8 Color Code lattice.	98
6.2	Complete quantum circuit for the 4.8.8 Color Code. . . .	100
6.3	Simulation 2b: Performance comparison of the Min-Sum and Product-Sum BP update rules.	105
6.4	Simulation 2b: Comparison of different OSD post- processing strategies.	106
6.5	Simulation 2b: Effect of the OSD search order.	107
6.6	Simulation 2: BP+OSD decoder thresholds.	107
6.7	Simulation 2: BP+OSD decoder execution times.	108
6.8	Simulation 3: Projection decoder thresholds.	112
6.9	Simulation 3: Projection decoder execution times.	112
6.10	Lifting as a MWPM problem	115
6.11	Simulation 4: Restriction decoder thresholds.	116
6.12	Simulation 4: Restriction decoder execution times.	116
6.13	Simulation 5: Möbius decoder thresholds.	118

6.14	Simulation 5: Möbius decoder execution times.	118
6.15	Simulation 6: Concatenated MWPM decoder thresholds. .	119
6.16	Simulation 6: Concatenated MWPM decoder execution times.	120
6.17	Simulation 7: Correlated matching decoder thresholds. . .	121
6.18	Simulation 7: Correlated matching decoder execution times.	122
7.1	Empirical error thresholds for the evaluated decoders. . .	125
7.2	Mitigation ratio comparison at $d = 9$ and $d = 23$	126
7.3	Logical error scaling below threshold.	129
7.4	Extrapolated required distances for target logical error rates.	129
7.5	Empirical decoding time scaling.	130
7.6	Extrapolated required decoding times with hardware budget.	131

Bibliography

- [1] Michael A Nielsen and Isaac L Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.
- [2] Richard P Feynman. “Simulating physics with computers”. In: *International Journal of Theoretical Physics* 21.6/7 (1982).
- [3] Peter W Shor. “Algorithms for quantum computation: discrete logarithms and factoring”. In: *Proceedings 35th Annual Symposium on Foundations of Computer Science*. IEEE. 1994, pp. 124–134.
- [4] Peter W. Shor. “Scheme for reducing decoherence in quantum computer memory”. In: *Physical Review A* 52.4 (1995), R2493–R2496. DOI: [10.1103/PhysRevA.52.R2493](https://doi.org/10.1103/PhysRevA.52.R2493). URL: <https://link.aps.org/doi/10.1103/PhysRevA.52.R2493>.
- [5] Andrew M. Steane. “Error correcting codes in quantum theory”. In: *Physical Review Letters* 77.5 (1996), pp. 793–797. DOI: [10.1103/PhysRevLett.77.793](https://doi.org/10.1103/PhysRevLett.77.793). URL: <https://link.aps.org/doi/10.1103/PhysRevLett.77.793>.
- [6] A Yu Kitaev. “Fault-tolerant quantum computation by anyons”. In: *Annals of Physics* 303.1 (2003), pp. 2–30.
- [7] H. Bombin and M. A. Martin-Delgado. “Topological Quantum Distillation”. In: *Physical Review Letters* 97.18 (Oct. 2006). ISSN: 1079-7114. DOI: [10.1103/physrevlett.97.180501](https://doi.org/10.1103/physrevlett.97.180501). URL: <http://dx.doi.org/10.1103/PhysRevLett.97.180501>.
- [8] Craig Gidney. “Stim: a fast stabilizer circuit simulator”. In: *Quantum* 5 (2021), p. 497. DOI: [10.22331/q-2021-07-06-497](https://doi.org/10.22331/q-2021-07-06-497).

- [9] William K. Wootters and Wojciech H. Zurek. “A single quantum cannot be cloned”. In: *Nature* 299.5886 (1982), pp. 802–803. DOI: [10.1038/299802a0](https://doi.org/10.1038/299802a0).
- [10] Emanuel Knill and Raymond Laflamme. “Theory of quantum error-correcting codes”. In: *Physical Review A* 55.2 (1997), p. 900. DOI: [10.1103/PhysRevA.55.900](https://doi.org/10.1103/PhysRevA.55.900). URL: <https://doi.org/10.1103/PhysRevA.55.900>.
- [11] Daniel Gottesman. “Stabilizer codes and quantum error correction”. arXiv preprint quant-ph/9705052. PhD thesis. California Institute of Technology, 1997. URL: <https://arxiv.org/abs/quant-ph/9705052>.
- [12] Dorit Aharonov and Michael Ben-Or. “Fault-tolerant quantum computation with constant error”. In: *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*. 1997, pp. 176–188. DOI: [10.1145/258533.258579](https://doi.org/10.1145/258533.258579).
- [13] Bryan Eastin and Emanuel Knill. “Restrictions on transversal encoded quantum gate sets”. In: *Physical Review Letters* 102.11 (2009), p. 110502.
- [14] Sergey Bravyi and Alexei Kitaev. “Universal quantum computation with ideal Clifford gates and noisy ancillas”. In: *Physical Review A* 71.2 (2005), p. 022316.
- [15] Craig Gidney and Martin Ekerå. “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits”. In: *Quantum* 5 (Apr. 2021), p. 433. ISSN: 2521-327X. DOI: [10.22331/q-2021-04-15-433](https://doi.org/10.22331/q-2021-04-15-433). URL: <http://dx.doi.org/10.22331/q-2021-04-15-433>.
- [16] H. Bombin and M. A. Martin-Delgado. “Optimal resources for topological two-dimensional stabilizer codes: Comparative study”. In: *Physical Review A* 76.1 (July 2007). ISSN: 1094-1622. DOI: [10.1103/PhysRevA.76.012305](https://doi.org/10.1103/PhysRevA.76.012305). URL: <http://dx.doi.org/10.1103/PhysRevA.76.012305>.
- [17] David S. Wang, Austin G. Fowler, and Lloyd C. L. Hollenberg. “Surface code quantum computing with error rates over 1%”. In: *Phys. Rev. A* 83 (2 Feb. 2011), 020302(R). DOI: [10.1103/PhysRevA.83.020302](https://doi.org/10.1103/PhysRevA.83.020302). URL: <https://link.aps.org/doi/10.1103/PhysRevA.83.020302>.

- [18] Andrew J. Landahl, Jonas T. Anderson, and Patrick R. Rice. *Fault-tolerant quantum computing with color codes*. 2011. arXiv: [1108.5738](https://arxiv.org/abs/1108.5738) [quant-ph]. URL: <https://arxiv.org/abs/1108.5738>.
- [19] Austin G. Fowler. “Two-dimensional color-code quantum computation”. In: *Physical Review A* 83.4 (Apr. 2011). ISSN: 1094-1622. DOI: [10.1103/PhysRevA.83.042310](https://doi.org/10.1103/PhysRevA.83.042310). URL: <http://dx.doi.org/10.1103/PhysRevA.83.042310>.
- [20] N. Lacroix et al. “Scaling and logic in the colour code on a superconducting quantum processor”. In: *Nature* 645.8081 (May 2025), pp. 614–619. ISSN: 1476-4687. DOI: [10.1038/s41586-025-09061-4](https://doi.org/10.1038/s41586-025-09061-4). URL: <http://dx.doi.org/10.1038/s41586-025-09061-4>.
- [21] Sergey Bravyi, David Poulin, and Barbara Terhal. “Tradeoffs for Reliable Quantum Information Storage in 2D Systems”. In: *Physical Review Letters* 104.5 (Feb. 2010). ISSN: 1079-7114. DOI: [10.1103/PhysRevLett.104.050503](https://doi.org/10.1103/PhysRevLett.104.050503). URL: <http://dx.doi.org/10.1103/PhysRevLett.104.050503>.
- [22] Sergey Bravyi et al. “High-threshold and low-overhead fault-tolerant quantum memory”. In: *Nature* 627.8005 (2024), pp. 778–782.
- [23] Eric Dennis et al. “Topological quantum memory”. In: *Journal of Mathematical Physics* 43.9 (2002), pp. 4452–4505.
- [24] Jack Edmonds. “Paths, trees, and flowers”. In: *Canadian Journal of mathematics* 17 (1965), pp. 449–467.
- [25] Oscar Higgott. “PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching”. In: *ACM Transactions on Quantum Computing* 3.3 (2022), pp. 1–16.
- [26] Nicolas Delfosse and Naomi H. Nickerson. “Almost-linear time decoding algorithm for topological codes”. In: *Quantum* 5 (Dec. 2021), p. 595. ISSN: 2521-327X. DOI: [10.22331/q-2021-12-02-595](https://doi.org/10.22331/q-2021-12-02-595). URL: <http://dx.doi.org/10.22331/q-2021-12-02-595>.
- [27] Robert G. Gallager. “Low-density parity-check codes”. In: *Information Theory, IRE Transactions on* 8 (1962), pp. 21–28. ISSN: 0096-1000. DOI: [10.1109/TIT.1962.1057683](https://doi.org/10.1109/TIT.1962.1057683). URL: http://ieeexplore.ieee.org/xpl/freeabs_all.jsp?arnumber=1057683.
- [28] J. Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. San Mateo, CA: Morgan Kaufmann, 1988.

- [29] Joschka Roffe et al. “Decoding across the quantum low-density parity-check code landscape”. In: *Physical Review Research* 2.4 (Dec. 2020). ISSN: 2643-1564. DOI: [10.1103/physrevresearch.2.043423](https://doi.org/10.1103/physrevresearch.2.043423). URL: <http://dx.doi.org/10.1103/PhysRevResearch.2.043423>.
- [30] Nicolas Delfosse. “Decoding color codes by projection onto surface codes”. In: *Physical Review A* 89.1 (Jan. 2014). ISSN: 1094-1622. DOI: [10.1103/physreva.89.012317](https://doi.org/10.1103/physreva.89.012317). URL: <http://dx.doi.org/10.1103/PhysRevA.89.012317>.
- [31] Aleksander Kubica and Nicolas Delfosse. “Efficient color code decoders in $d \geq 2$ dimensions from toric code decoders”. In: *Quantum* 7 (Feb. 2023), p. 929. ISSN: 2521-327X. DOI: [10.22331/q-2023-02-21-929](https://doi.org/10.22331/q-2023-02-21-929). URL: <http://dx.doi.org/10.22331/q-2023-02-21-929>.
- [32] Aleksander Marek Kubica. “The color code: A topological quantum computer”. PhD thesis. California Institute of Technology, 2018. URL: <https://thesis.caltech.edu/10955/>.
- [33] Kaavya Sahay and Benjamin J. Brown. “Decoder for the Triangular Color Code by Matching on a Möbius Strip”. In: *PRX Quantum* 3.1 (Jan. 2022). ISSN: 2691-3399. DOI: [10.1103/prxquantum.3.010310](https://doi.org/10.1103/prxquantum.3.010310). URL: <http://dx.doi.org/10.1103/PRXQuantum.3.010310>.
- [34] Seok-Hyung Lee, Andrew Li, and Stephen D. Bartlett. “Color code decoder with improved scaling for correcting circuit-level noise”. In: *Quantum* 9 (Jan. 2025), p. 1609. ISSN: 2521-327X. DOI: [10.22331/q-2025-01-27-1609](https://doi.org/10.22331/q-2025-01-27-1609). URL: <http://dx.doi.org/10.22331/q-2025-01-27-1609>.
- [35] Asmae Benhemou et al. “Minimising surface-code failures using a color-code decoder”. In: *Quantum* 9 (Feb. 2025), p. 1632. ISSN: 2521-327X. DOI: [10.22331/q-2025-02-17-1632](https://doi.org/10.22331/q-2025-02-17-1632). URL: <http://dx.doi.org/10.22331/q-2025-02-17-1632>.
- [36] Yantong Liu, Junjie Wu, and Lingling Lao. *The correlated matching decoder for the 4.8.8 color code*. 2025. arXiv: [2511.13192](https://arxiv.org/abs/2511.13192) [quant-ph]. URL: <https://arxiv.org/abs/2511.13192>.
- [37] Daniel Gottesman. “The Heisenberg representation of quantum computers”. In: *Group22: Proceedings of the XXII International Col-*

- loquium on Group Theoretical Methods in Physics*. arXiv preprint quant-ph/9807006. International Press. 1999, pp. 32–43.
- [38] Scott Aaronson and Daniel Gottesman. “Improved simulation of stabilizer circuits”. In: *Physical Review A* 70.5 (2004), p. 052328. DOI: [10.1103/PhysRevA.70.052328](https://doi.org/10.1103/PhysRevA.70.052328).
 - [39] Cirq Developers. *Cirq*. <https://zenodo.org/doi/10.5281/zenodo.4062499>. DOI: 10.5281/ZENODO.4062499. Python package for writing, manipulating, and running quantum circuits on quantum computers and simulators. Aug. 2025.
 - [40] Joschka Roffe. *ldpc: A Python package for decoding classical and quantum LDPC codes*. 2022. DOI: [10.5281/zenodo.10023405](https://doi.org/10.5281/zenodo.10023405). URL: <https://github.com/quantumsimulations/ldpc>.
 - [41] Ching-Feng Kung, Kao-Yueh Kuo, and Ching-Yi Lai. “On Belief Propagation Decoding of Quantum Codes with Quaternary Reliability Statistics”. In: *2023 12th International Symposium on Topics in Coding (ISTC)*. IEEE, Sept. 2023, pp. 1–5. DOI: [10.1109/istc57237.2023.10273527](https://doi.org/10.1109/istc57237.2023.10273527). URL: <http://dx.doi.org/10.1109/ISTC57237.2023.10273527>.
 - [42] Ashley M. Stephens. *Efficient fault-tolerant decoding of topological color codes*. 2014. arXiv: [1402.3037 \[quant-ph\]](https://arxiv.org/abs/1402.3037). URL: <https://arxiv.org/abs/1402.3037>.
 - [43] Craig Gidney. *Chromobius: A Möbius Color Code Decoder*. GitHub repository. 2024. URL: <https://github.com/quantumlib/chromobius>.
 - [44] Seok-Hyung Lee. *ConcatMatching: Generalised concatenated matching decoder for arbitrary stabiliser codes*. <https://github.com/seokhyung-lee/ConcatMatching>. 2024.
 - [45] Seok-Hyung Lee. *color-code-stim*. <https://github.com/seokhyung-lee/color-code-stim>. 2024.
 - [46] Yue Wu, Namitha Liyanage, and Lin Zhong. “Micro Blossom: Accelerated Minimum-Weight Perfect Matching Decoding for Quantum Error Correction”. In: *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. ASPLOS ’25. ACM, Mar. 2025, pp. 639–654. DOI: [10.1145/3676641.3716005](https://doi.org/10.1145/3676641.3716005). URL: <http://dx.doi.org/10.1145/3676641.3716005>.

- [47] Namitha Liyanage et al. “FPGA-Based Distributed Union-Find Decoder for Surface Codes”. In: *IEEE Transactions on Quantum Engineering* 5 (2024), pp. 1–18. ISSN: 2689-1808. DOI: [10.1109/tqe.2024.3467271](https://doi.org/10.1109/tqe.2024.3467271). URL: <http://dx.doi.org/10.1109/TQE.2024.3467271>.
- [48] Rajeev and Acharya et al. “Quantum error correction below the surface code threshold”. In: *Nature* 638.8052 (Dec. 2024), pp. 920–926. ISSN: 1476-4687. DOI: [10.1038/s41586-024-08449-y](https://doi.org/10.1038/s41586-024-08449-y). URL: <http://dx.doi.org/10.1038/s41586-024-08449-y>.